



Projeto e Análise de Algoritmos

Árvores: ABB, AVL, Tries, Rubro-Negras

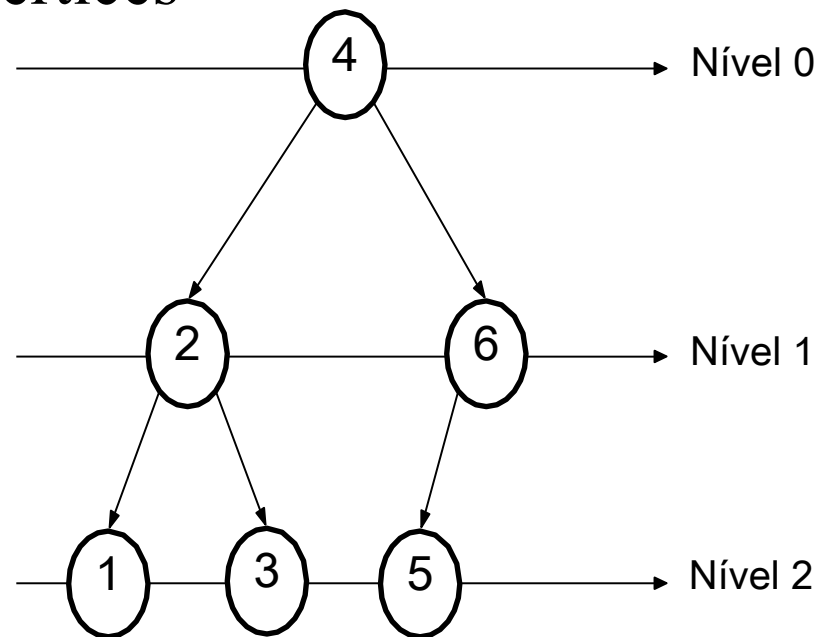
André Tavares da Silva
andre.silva@udesc.br

Introdução à Árvores Binárias

- Estruturas de dados tradicionais
 - Listas representam dados de maneira sequencial
 - Árvores são adequadas para representação hierárquica
 - Árvores possui uma definição recursiva
 - Cada nó da árvore forma uma subárvore
- Uma árvore é composta por
 - Um conjunto de nós
 - Um dos nós é denominado raiz
 - Cada nó pode ter múltiplos filhos
 - Os nós que não possuem filhos são chamados de folhas

Introdução à Árvores Binárias

- Estrutura de uma árvore
 - Nós representam os vértices da árvore
 - Arestas conectam dois vértices
 - Níveis da árvore



Introdução à Árvores Binárias

- Maneiras para representar visualmente uma árvore

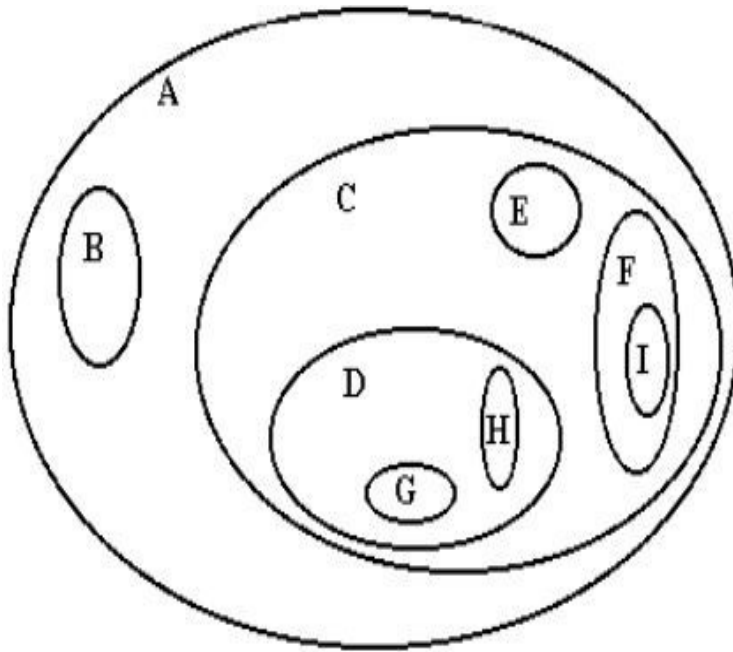
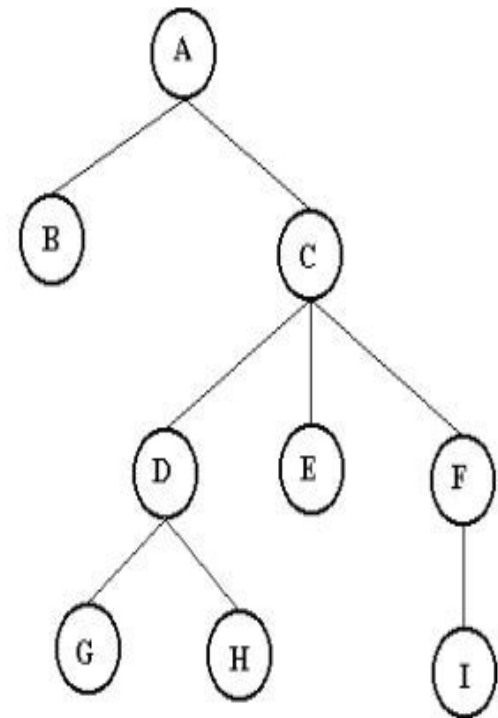


Diagrama de inclusão

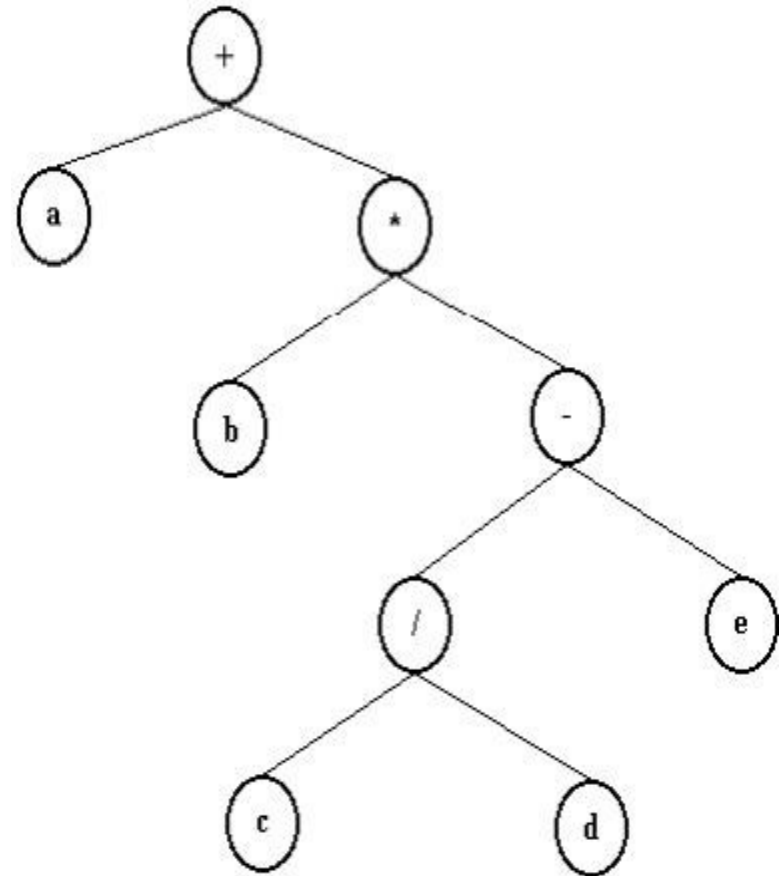


Representação hierárquica

Introdução à Árvores Binárias

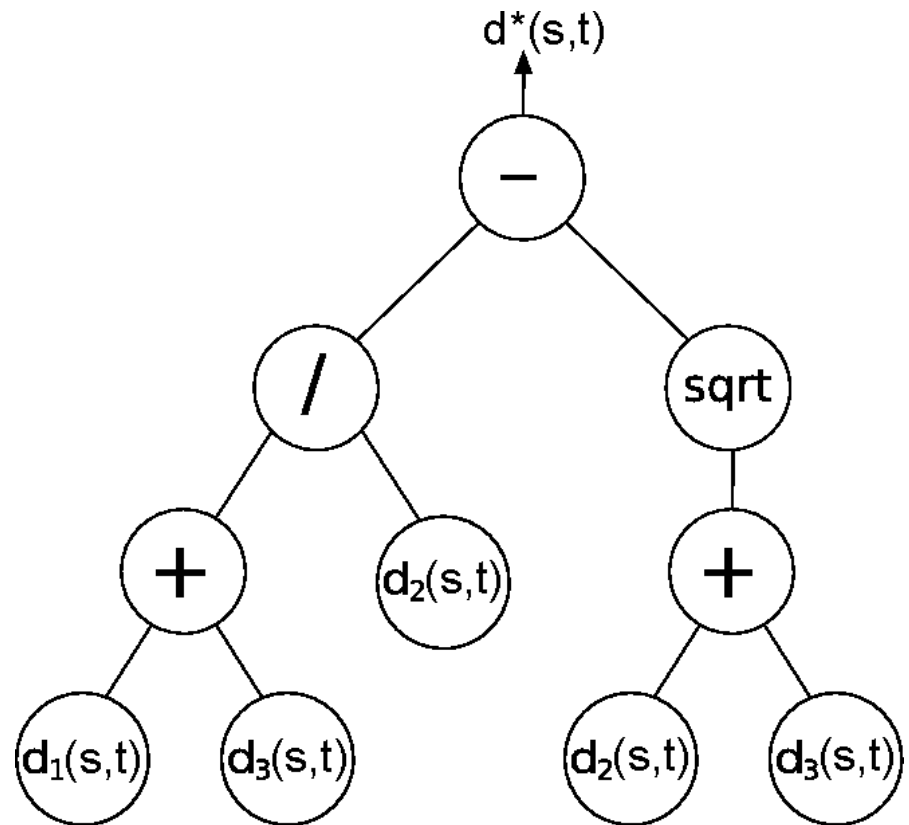
- Exemplo de uso de árvores para representar expressões aritméticas

`( a + ( b * ( c / d - e ) ) )`



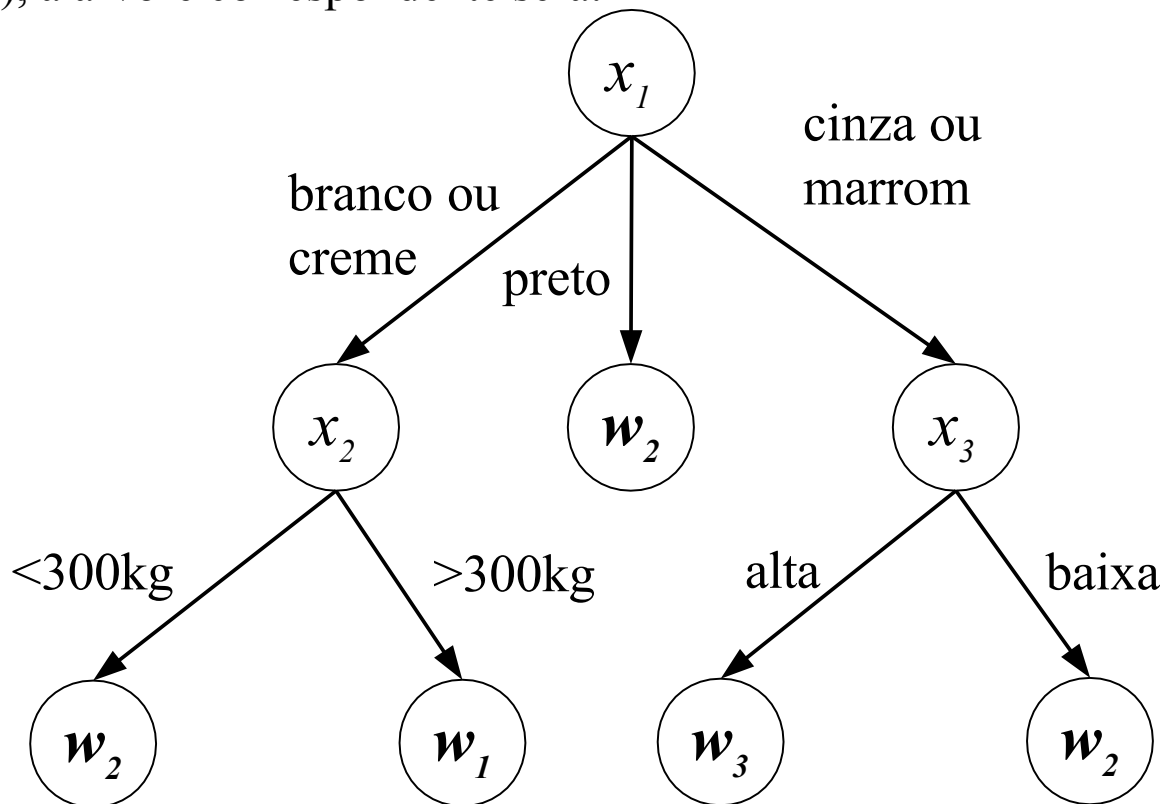
Exemplo: Programação Genética

- A programação genética procura funções de combinação evoluindo uma população P de indivíduos por iteração durante várias gerações.
- O melhor indivíduo gerado pela aplicação de transformações genéticas é definido como função de combinação $d^*(s,t)$.
- Reprodução seleciona os melhores indivíduos e os copia na próxima geração.
- Mutação é uma manipulação aleatória que atua em apenas um indivíduo, selecionando um nó e substituindo a subárvore por outra gerada aleatoriamente.
- Crossover combina o material genético de dois indivíduos trocando uma de suas subárvores.



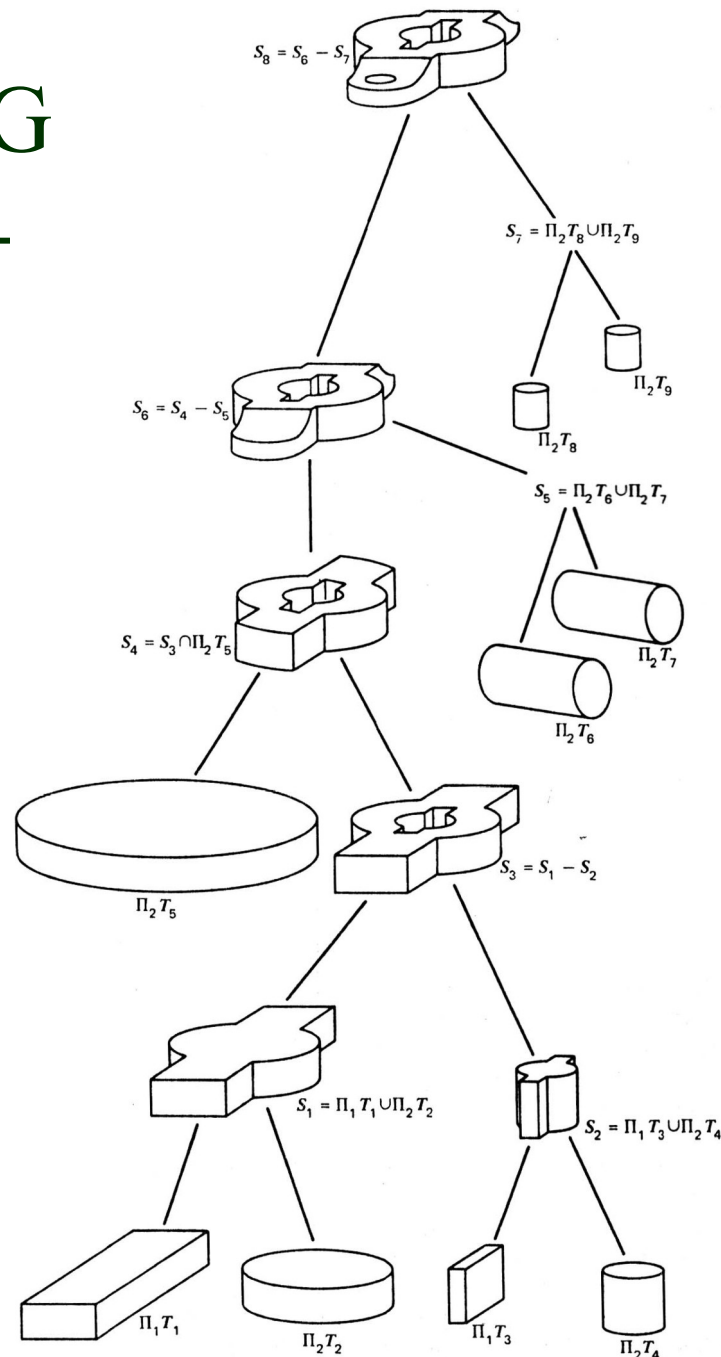
Exemplo: Árvore de Decisão

- Considerando, por exemplo, uma classificação de ursos de três classes distintas: $w_1 = \textit{ursus maritimus}$ (polar), $w_2 = \textit{ursus americanus}$ e $w_3 = \textit{ursus arctos}$ (pardo).
- De acordo com três características (x_1 =cor, x_2 =peso, e x_3 =possibilidade de atacar humanos), a árvore correspondente será:



Exemplo: CSG

- A partir de primitivas geométricas (cilindros, cones, esferas, cubos,...), transformações geométricas (escala, rotação e translação) e operações booleanas (união, intersecção e diferença), são criados objetos geométricos por esta técnica.
- Um modelo CSG é codificado por uma árvore binária.
- Os nós internos contêm operações booleanas regularizadas;
- Arestas podem armazenar as transformações geométricas;
- Folhas contêm objetos primitivos (tipicamente, quádricas)



Introdução à Árvores

- Conceitos básicos
 - Cada nó pode possuir um único nó pai
 - Cada nó possui uma coleção de nós filhos
 - Esta coleção pode ser vazia
 - Árvores binárias possuem, no máximo, dois nós filhos
 - O nó raiz não possui nó pai
 - Os nós folhas possuem uma coleção vazia de filhos
 - Grau de um nó representa a quantidade de ligações com nós vizinhos
 - Grau de saída de um nó representa a quantidade de filhos do nó em questão
 - Grau de uma árvore representa a maior quantidade de filhos entre os nós
 - Uma floresta é uma coleção de árvores

Introdução à Árvores

- Conceitos básicos (cont.)
 - Caminho
 - É a uma sequência de nós distintos ligados pela relação de pai e filho
 - Existe apenas um caminho da raiz para qualquer nó
 - O comprimento de um caminho refere-se ao número de nós contidos
 - Profundidade
 - Comprimento do maior caminho da raiz até o nó folha mais distante
 - Profundidade é uma medida de altura da árvore
 - Largura
 - Maior número de filhos dos nós pertencentes ao mesmo nível

Introdução à Árvores

- Conceitos básicos (cont.)
 - Árvore cheia de grau D
 - Todos os nós possuem o mesmo número de filhos, exceto os nós folha
 - Árvore cheia de grau 2
 - Todos os nós possuem dois filhos, exceto os nós folha
 - São chamadas de árvores binárias cheias (perfeitamente balanceadas)
 - Pode ser implementadas sequencialmente por vetor
 - Exemplo de árvore binária para representação de expressões aritméticas
 - Nós folhas representam os operandos (números)
 - Nós internos representam os operadores

Árvores Binárias

- Representação em C

```
typedef struct no {  
    struct no* pai;           // ponteiro para o nó pai  
    struct no* esquerda;     // ponteiro para o nó filho da esquerda  
    struct no* direita;      // ponteiro para o nó filho da direita  
    float v;                  // conteúdo de um nó arbitrário da árvore  
} Arvore;
```

```
typedef struct arvore {  
    struct no* raiz;  
} Arvore;
```

Árvores Binárias

- Operações básicas
 - Criar uma estrutura de árvore
 - Inserir um novo elemento na árvore
 - Remover o elemento da árvore
 - Verificar se a árvore está vazia
 - Liberar memória da árvore
 - Percorrer os caminhos da árvore
 - Buscar um elemento na árvore

Árvores Binárias

- Criar uma árvore binária

```
Arvore* cria() {  
    Arvore *arvore;  
    arvore = malloc(sizeof(Arvore)); arvore->raiz = NULL;  
  
    return arvore;  
}
```

- Criar uma árvore binária

```
int vazia(Arvore* arvore) {  
    return (arvore->raiz == NULL);  
}
```

Árvores Binárias

- Adicionar um elemento na árvore

```
No* adiciona(Arvore* arvore, No* pai, float valor) {  
    No *no = malloc(sizeof(No));  
    no->pai = pai;  
    no->esquerda = NULL;  
    no->direita = NULL;  
    no->valor = valor;  
  
    if (pai == NULL) {  
        arvore->raiz = no;  
    }  
  
    return no;  
}
```

Árvores Binárias

- Remover um elemento na árvore

```
void remove(Arvore* arvore, No* no) {  
    if (no->esquerda != NULL) remove(arvore, no->esquerda);  
    if (no->direita != NULL) remove(arvore, no->direita);  
    if (no->pai == NULL) {  
        arvore->raiz = NULL;  
    } else {  
        if (no->pai->esquerda == no)  
            no->pai->esquerda = NULL;  
        else  
            no->pai->direita = NULL;  
    }  
    free(no);  
}
```


- Percorrer a árvore

```
void percorrer(No* no) {  
    if (no != NULL) {  
        printf("%f", no->valor);  
        percorrer(no->esquerda);  
        percorrer(no->direita);  
    }  
}
```

Árvores Binárias

- Busca em árvore
 - Estratégia para visitar os nós de uma árvore
 - Busca em profundidade
 - Elementos são visitados intercalando os níveis da árvore
 - O resultado da busca a partir da raiz forma múltiplos caminhos
 - Busca em largura
 - Elementos são visitados no mesmo nível da árvore
 - Em seguida é realizada a mesma busca no próximo nível da árvore
 - Precisa ser conhecido quais nós pertencem a cada nível

Árvores Binárias

- Busca em profundidade
 - A busca ocorre visitando os nós filhos da esquerda para a direita
 - O processo ocorre iterativamente a cada subárvore explorada
 - Estratégias de busca em árvore binária: *pré-ordem*, *em-ordem* e *pós-ordem*
 - *Pré-ordem*
 - Raiz, esquerda, direita
 - *Em-ordem*
 - Esquerda, raiz e direita
 - *Pós-ordem*
 - Esquerda, direita e raiz

Árvores Binárias

- Árvores ordenadas
 - Os valores dos nós são dispostos de forma hierárquica e ordenados
 - O nó da esquerda possui um valor (chave) menor que o nó atual
 - O nó da direita possui um valor (chave) maior que o nó atual
- Altura e largura máxima são as dimensões da árvore
 - Esforço computacional de busca é proporcional às dimensões da árvore
 - Isto é, o esforço computacional depende da altura e largura da árvore

Árvores Binárias

- Balanceamento de árvores
 - Árvore degenerada
 - Ocorre quando cada nó possui uma única subárvore associada
 - Neste caso a árvore representa uma estrutura linear
 - Percorrer nesta árvore corresponde a complexidade N (linear)
 - Árvore cheia
 - Possui um balanceamento perfeito
 - Todos os nós possuem dois filhos ou são folhas
 - Percorrer nesta árvore corresponde a complexidade $\log N$ (logarítmica)

Árvores Binárias

- Estrutura de uma árvore binária genérica

```
typedef struct no {  
    struct no* pai;           //ponteiro para o nó pai  
    struct no* esquerda;      //ponteiro para o nó filho a esquerda  
    struct no* direita;       //ponteiro para o nó filho a direita  
    void* valor;              //conteúdo genérico do nó  
  
} No;  
  
typedef struct arvore {  
    struct no* raiz; //raiz da árvore  
    int tamanho;      //tamanho do valor dos nós (em bytes)  
} Arvore;
```

Complexidade de Árvores Binárias

- A relação entre o número de nós n e a altura da árvore vem de:

$$n = \sum_{k=0}^h 2^k$$

- Sendo assim, aplicando logaritmos de ambos os lados:

$$h = \log_2(n+1) - 1$$

- nos permite escrever que a altura mínima da árvore é **$O(\log_2 n)$** . Caso a árvore seja degenerada, a altura será de **$O(n)$** .

Árvores Binárias

- Como tornar genérica a operação de pesquisa?
- Usando **templates** em programação orientada a objetos
- Técnica baseada em *callback* usando **ponteiro de função**
 - Permite separação entre
 - Comportamento para percorrer os elementos da estrutura de dados
 - Ação realizada sobre cada elemento visitado na iteração
 - Função para percorrer é genérica
 - A parte específica é o que será feito a cada elemento visitado
 - Isto é, a operação de *callback*

Árvores Binárias

- Implementação ocorre por meio de ponteiro de funções
 - Nome de uma função representa o endereço da função
- Exemplo
 - Assinatura da função

```
void callback(void* v);
```

- Declaração do ponteiro para armazenar o endereço da função

```
void *(cb)(void*);
```

onde *cb* é ponteiro para funções com mesma assinatura que *callback*

Árvores Binárias

- Recomendações sobre o uso de *callbacks*
 - Evitar uso de variáveis globais
 - Problemas de concorrência
 - Dificuldade para compreensão do código
 - Passar parâmetros genéricos, se for o caso
 - Pode ter, além do ponteiro do elemento atual, um ponteiro para dados
 - Isolar o comportamento genérico específico do *callback*
 - Reduz o acoplamento entre o cliente e a implementação de lista genérica

Árvores Binárias

- Operação de percorrer genérica com suporte a *callback*

```
void percorrer(No* no, void (callback)(void*)) {  
    if (no != NULL) {  
        callback(no->v);  
  
        percorrer(no->esquerda);  
        percorrer(no->direita);  
    }  
}
```

- Uso da operação de *callback*

```
void exhibe(void* v) {  
    printf("%f\n", v);  
}
```

```
int main(int argc, char *argv[]){  
    Arvore *a = cria(); //insere valores na árvore...  
    percorre(a->raiz, exhibe); //passando a função como parâmetro  
}
```

Exercícios

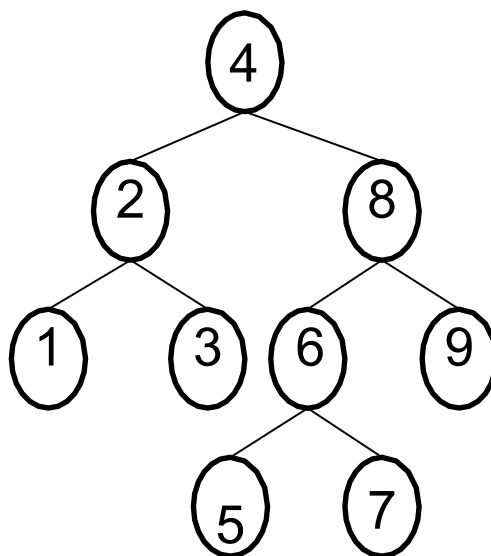
1. Implemente um algoritmo que visite todos os nós de uma árvore binária com uma busca em profundidade *em-ordem*, *pré-ordem* e *pós-ordem*.

○ Exemplo, para a árvore abaixo a saída deve ser:

■ *Pré-ordem*: 4, 2, 1, 3, 8, 6, 5, 7, 9

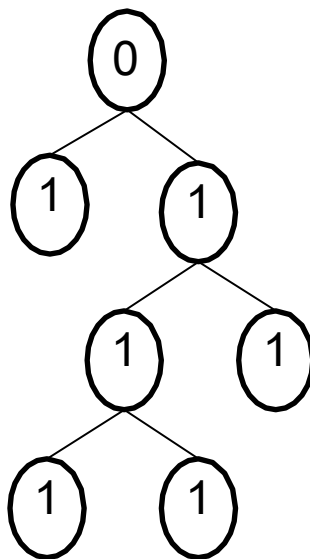
■ *Em-ordem*: 1, 2, 3, 4, 5, 6, 7, 8, 9

■ *Pós-ordem*: 1, 3, 2, 5, 7, 6, 9, 8, 4



Exercícios

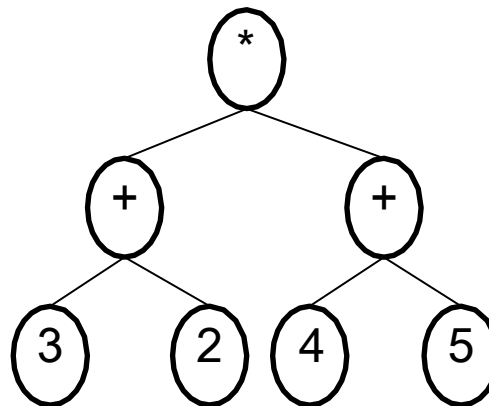
2. Uma árvore *unival* (valor universal) é uma árvore na qual todos os nós abaixo dela possuem o mesmo valor. Dada a raiz de uma árvore binária, conte o número de subárvores *unival*.
- Exemplo, a árvore abaixo possui 5 subárvores *unival*



Exercícios

3. Suponha uma expressão aritmética representada como uma árvore binária. Cada folha é um número inteiro e cada nó interno representa uma operação, como $+$, $-$, $*$ ou $/$. Dada a raiz da árvore, escreva um programa que avalia a expressão.

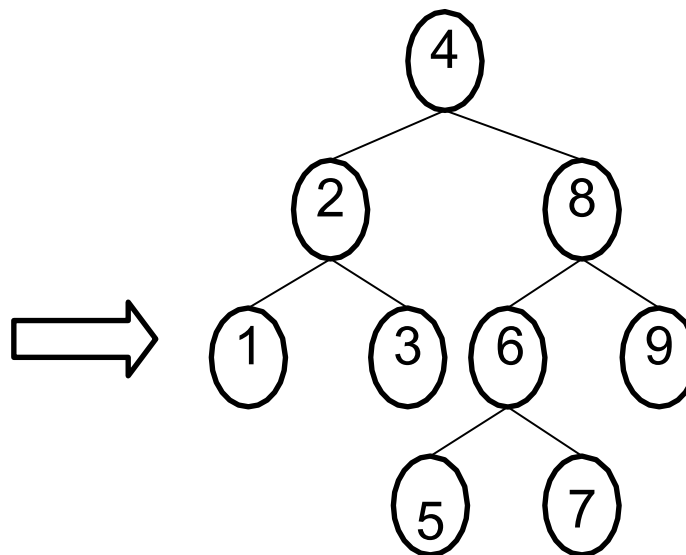
- Exemplo, dada a árvore abaixo a saída é 45 pois $(3 + 2) * (4 + 5)$



Exercícios

4. Implemente uma função para adicionar nós em uma árvore binária que respeite as regras abaixo necessárias para uma pesquisa binária.
- O valor de cada nó deve ser inteiro
 - O nó filho à esquerda possui um valor menor ou igual ao valor de seu nó pai
 - O nó filho à direita possui um valor maior do que o valor de seu nó pai
 - Exemplo: deve ser gerada a árvore abaixo ao adicionar 4, 2, 1, 3, 8, 9, 6, 5, 7

```
Arvore *a = cria();  
adiciona(a,4);  
adiciona(a,2);  
adiciona(a,1);  
adiciona(a,3);  
adiciona(a,8);  
adiciona(a,9);  
adiciona(a,6);  
adiciona(a,5);  
adiciona(a,7);
```



Árvore Binária de Busca

- Uma árvore binária de busca (abreviada por ABB) serve para o armazenamento de dados no computador e a sua subsequente busca de informações.
- Em uma árvore binária de busca cada nó contém um campo contendo uma chave de busca, podendo haver outras informações, além dos ponteiros esquerda e direita.
- A chave de busca especifica em geral um determinado registro ou informação, como número de identidade, CPF, etc.
- Criada a árvore, é possível localizar qualquer elemento em **$O(\log n)$** dependendo do balanceamento da mesma, podendo ser inseridos novos registros ou removidos para atualizar a árvore.

Árvore Binária de Busca

- Uma árvore de busca binária A é uma árvore binária vazia ou, então, uma árvore binária cuja raiz armazena um item x e tem as seguintes propriedades:
 - Todo item na subárvore esquerda de A é menor ou igual a x .
 - Todo item na subárvore direita de A é maior que x .
 - Cada subárvore de A é uma árvore de busca binária.
- Uma consequência importante dessas propriedades é que a projeção de uma árvore de busca binária sempre produz uma sequência ordenada de itens.

Inserção em ABB

- A função para inserir um novo item numa árvore de busca binária usa um percurso pré-ordem: primeiro ela verifica se a árvore está vazia e, caso esteja, ela coloca o novo item na raiz da árvore; se não, se o novo item for menor ou igual àquele na raiz da árvore, recursivamente, ela o insere na subárvore esquerda; caso contrário, recursivamente, ela o insere na subárvore direita. No código abaixo, como o ponteiro para a árvore pode ser alterado durante uma inserção, ele deve ser passado por referência.

```
void insere(float valor) {  
    No *no=root, *prev = NULL;  
    while(no != NULL) {  
        prev = no;  
        if(valor <= no->valor)  
            no = no->esquerda;  
        else  
            no = no->direita;  
    }  
    if(root == NULL) root = new No(valor);  
    else if(valor < prev->valor) prev->esquerda = new No(valor);  
    else prev->direita = new No(valor);  
}
```

Consulta em ABB

- Consultar se determinado novo nó existe em uma árvore binária de busca é uma tarefa similar à inserção de um novo nó. Basicamente, o que temos que fazer é percorrer os nós da árvore usando o seguinte conjunto de passos: primeiro compare o valor buscado com a raiz; se o valor é menor do que a raiz: vá para a subárvore da esquerda; se o valor é maior do que a raiz: vá para a subárvore da direita; aplique o método até que a raiz seja igual ao valor buscado.

```
int busca(float valor) {  
    No *no=root, *prev = NULL;  
    while(no != NULL) {  
        prev = no;  
        if(valor <= no->valor)  
            no = no->esquerda;  
        else  
            no = no->direita;  
    }  
    if(root == NULL) return 0; // Árvore vazia  
    else if(valor == prev->valor) return 1;  
    Return 0; // elemento não localizado  
}
```

Remoção em ABB

- A remoção em árvore de busca binária é mais difícil do que a inserção e a busca. Primeiro vamos considerar a remoção de um item máximo da árvore de busca binária. Para isso, partindo da raiz da árvore, seguimos sempre o ponteiro à direita, enquanto ele não for NULL. Quando chegarmos a um nó cujo ponteiro à direita é NULL, estaremos no nó que guarda um item máximo da árvore. Então, basta remover esse nó e devolver o item que ele armazena.
- O nó que guarda um item máximo da árvore de busca binária não pode ter filho à direita (senão, ele não seria máximo), mas ele pode ter um filho à esquerda. Portanto, quando esse nó é removido, o ponteiro que o apontava deve passar a apontar seu filho à esquerda (caso ele não tenha filho à esquerda, o ponteiro que o apontava ficará nulo).

Remoção em ABB

- Agora, vamos considerar a remoção de um item x , que está na raiz da árvore de busca binária A . Então, há três casos possíveis: Se o nó apontado por A não tem filhos, então ele deve ser desalocado e o ponteiro A deve ser anulado (isto é, a árvore fica vazia). Se o nó apontado por A tem um único filho, então ele deve ser desalocado e o ponteiro A deve passar a apontar esse filho. Se o nó apontado por A tem dois filhos, então o item nesse nó deve ser substituído por um item máximo removido de sua subárvore esquerda.
- Para entender a estratégia adotada nesse último caso, lembre que, numa árvore de busca binária A , cada item na subárvore esquerda de A é menor que todo item na subárvore direita de A . Portanto, substituindo o item na raiz de A por um item máximo removido de sua subárvore esquerda, podemos garantir que a árvore resultante da remoção do item A ainda será uma árvore de busca binária.

Remoção em ABB

- Para generalizar essa ideia, precisamos considerar que o item a ser removido pode estar em qualquer nó da árvore, não apenas na raiz. Nesse caso, antes de removê-lo, precisamos encontrá-lo na árvore. Para isso, basta adaptar a lógica da função de busca em árvore.

```
void remove (float valor) {  
    No *no=root, *prev = NULL;  
    int filho_esq = 1;  
    while (no != NULL) {  
        prev = no;  
        if (valor <= no->valor) {  
            no = no->esquerda;  
            filho_esq = 1;  
        } else {  
            no = no->direita;  
            filho_esq = 0;  
        }  
    }  
    if (no == NULL)  
        return; // não localizado ou árvore vazia  
    // se folha:  
    else if ((no->esquerda == NULL) && (no->esquerda == NULL)) {  
        if(no==root) root = NULL;  
        else if (filho_esq) prev->esquerda = NULL;  
        else prev->direita == NULL;  
    }  
    // (continua...)
```

```
    // continuação:  
    } else if (no->esquerda == NULL) {  
        // se não tem filho a esquerda, substitui pela subarvore dir  
        if (filho_esq) prev->esquerda = no->direita;  
        else prev->direita = no->direita;  
    } else if (no->direita == NULL) {  
        // se não tem filho a direita, substitui pela subarvore esq  
        if (filho_esq) prev->esquerda = no->esquerda;  
        else prev->direita = no->esquerda;  
    } else {  
        // se possui mais de um filho, procura nó máximo (dir/esq)  
        No *maximo = remmax(no);  
        if (filho_esq) prev->esquerda = maximo;  
        else prev->direita = maximo;  
    }  
}
```

Exercícios

- 1) Supondo que a árvore seja criada inserindo valores em uma sequência aleatória de valores utilizando sempre um mesmo conjunto de dados. A árvore sempre será criada com a mesma estrutura ou poderá resultar em árvores diferentes?
- 2) Uma árvore binária de busca sempre será balanceada?
- 3) Indique um caso ideal em relação à ordem de inserção de nós para criação de uma árvore de busca balanceada. Explique o motivo.
- 4) Informe qual o pior caso ao criar uma árvore de busca. Justifique.

Exercícios de Implementação

- 1) Crie um programa para gerar uma ABB (árvore binária de busca). O programa deve ser capaz de inserir novos elementos, remover elementos da árvore, realizar uma busca e listar todos os elementos da árvore de maneira ordenada.
- 2) Escreva programa deve ser capaz de ler uma árvore binária (qualquer) e informar as seguintes informações:
 - Se a árvore é vazia;
 - Tamanho da árvore (número de nós)
 - Altura ou profundidade da árvore (nível máximo)
 - Largura da árvore em cada nível
 - Se árvore está completa
 - Se a árvore está cheia

Introdução à Árvores Binárias

- Estruturas de dados tradicionais
 - Listas representam dados de maneira sequencial
 - Árvores são adequadas para representação hierárquica
 - Árvores possui uma definição recursiva
 - Cada nó da árvore forma uma subárvore
- Uma árvore é composta por
 - Um conjunto de nós
 - Um dos nós é denominado raiz
 - Cada nó pode ter múltiplos filhos
 - Os nós que não possuem filhos são chamados de folhas

Árvores Binárias

- Balanceamento de árvores
 - Árvore degenerada
 - Ocorre quando cada nível da árvore apresenta um único nó
 - Neste caso a árvore representa uma estrutura linear
 - Uma busca nesta árvore corresponde a complexidade $O(n)$
 - Árvore cheia ou completa
 - Uma árvore cheia possui um balanceamento perfeito
 - Todos os nós possuem dois filhos, exceto os nós folhas
 - Uma busca nesta árvore corresponde a complexidade $O(\log n)$

Árvores Binárias

- Benefícios de uma árvore balanceada
 - Minimizam o número de comparações efetuadas no pior caso
- Para garantir esta propriedade, é necessário
 - Reconstruir a árvore em seu estado ideal a cada alteração de estrutura
 - Operações de inclusão, exclusão ou alteração (quando envolver a chave)
 - Para cada operação acima, pode ser necessário um balanceamento
 - Balanceamento consiste em reequilibrar a árvore em suas dimensões
 - Isto é, largura e altura

Árvores AVL

Árvores AVL

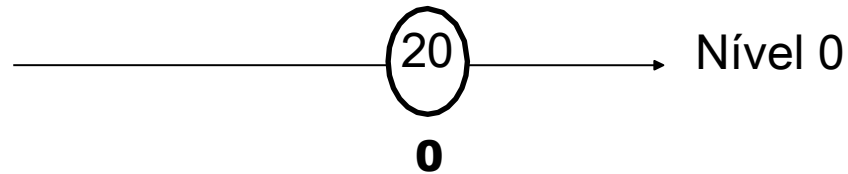
- Árvore AVL é uma árvore binária altamente balanceada
 - Introduzida por Adelson-Velskii e Landis, 1962
 - A cada inserção ou exclusão, é executada uma rotina de balanceamento
 - Visam manter a altura da subárvores à esquerda e à direita equilibradas
 - Formalmente
 - Alturas das subárvores à esq. e dir. diferem-se, no máximo, em uma unidade
 - Para este cálculo, usa-se o fator de balanceamento (FB)

FB = altura da subárvore esquerda – altura da subárvore direita

Árvores AVL

- Exemplo

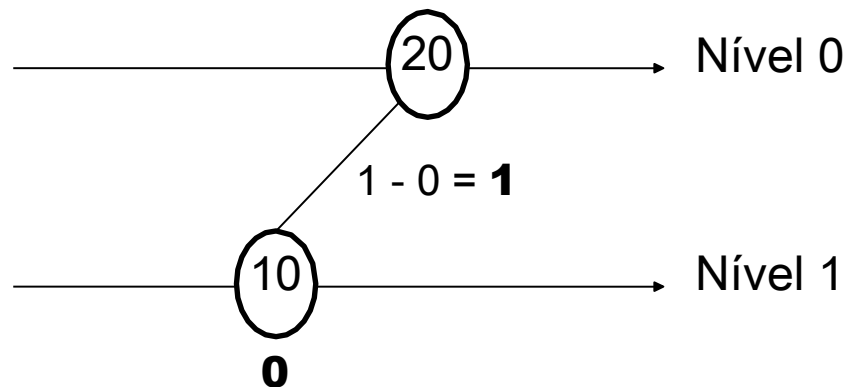
- 20



Árvores AVL

- Exemplo

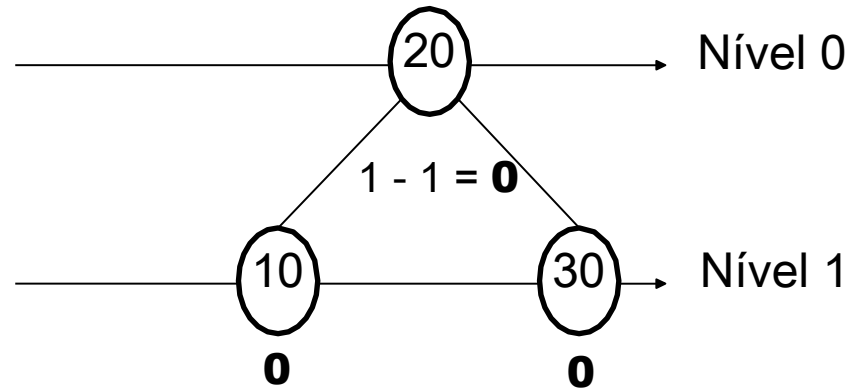
- 20
- 10



Árvores AVL

- Exemplo

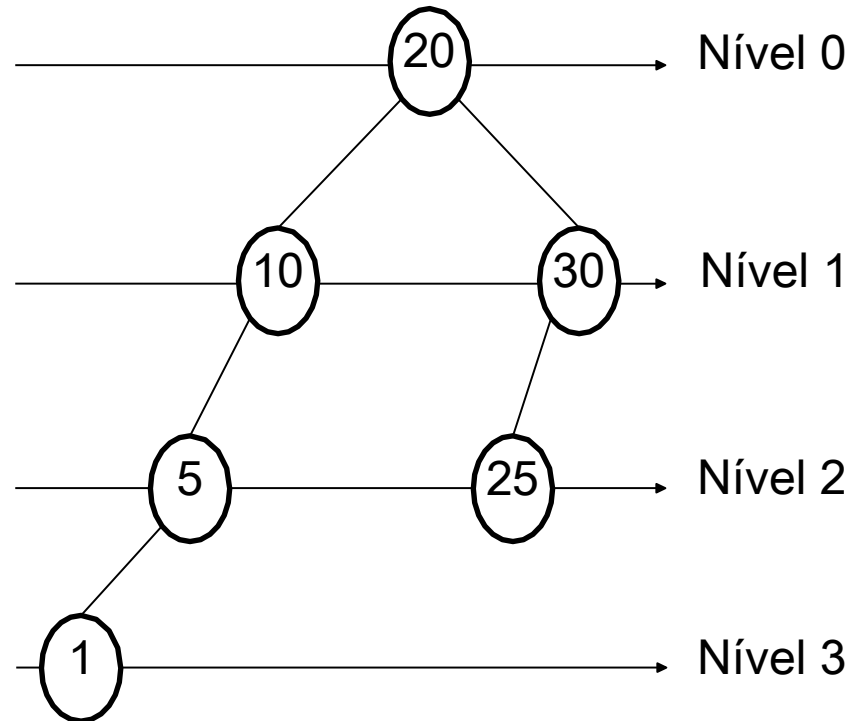
- 20
- 10
- 30



Árvores AVL

- Exemplo

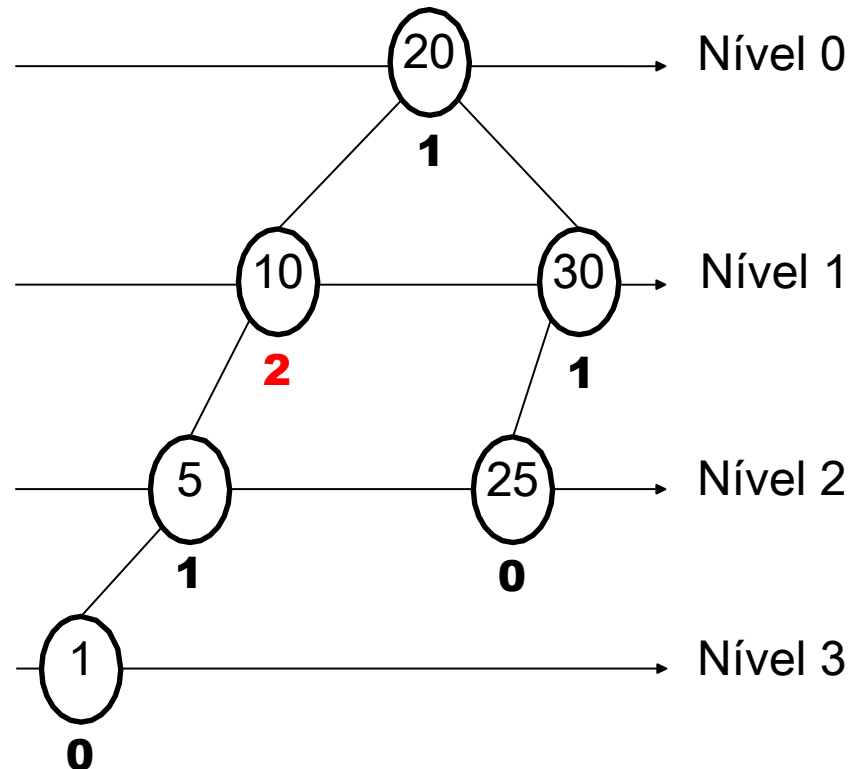
- 20
- 10
- 30
- 5
- 25
- 1



Árvores AVL

- Exemplo

- 20
- 10
- 30
- 5
- 25
- 1



Operações de rotação

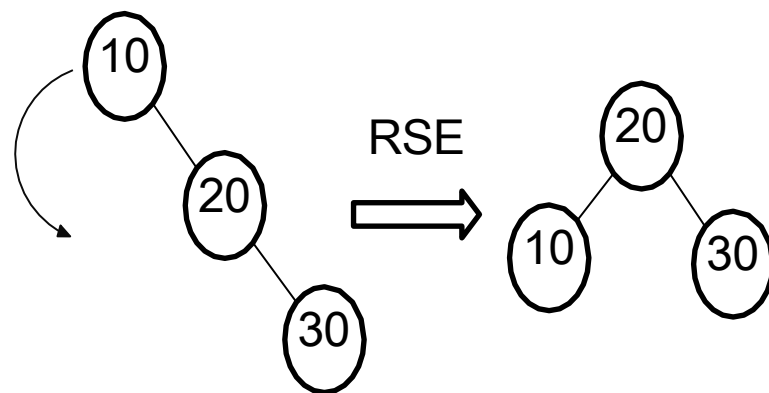
- Quando a árvore está desbalanceada, é possível realizar as seguintes operações para balanceá-la novamente
 - Rotação simples
 - Rotação simples à esquerda (RSE) ou rotação LL
 - Rotação simples à direita (RSD) ou rotação RR
 - Rotação dupla
 - Rotação dupla à esquerda (RDE) ou rotação RL
 - Rotação dupla à direita (RDD) ou rotação LR
- Quando o FB resultar em um valor diferente de 0, -1 ou 1, deverá ser realizada uma das operações de balanceamento

Operações de rotação

- Rotação simples à esquerda (RSE) ou rotação LL
 - Neste caso a subárvore está mais alta (profunda) no lado direito
 - Situação para realização de uma RSE
 - FB do nó desbalanceado for negativo e;
 - Nó à direita também apresentar FB negativo

- Exemplo: RSE(10,20)

- Nó 10 passa a ser filho de 20
- Nó 20 passa a ser pai de 10 e 30

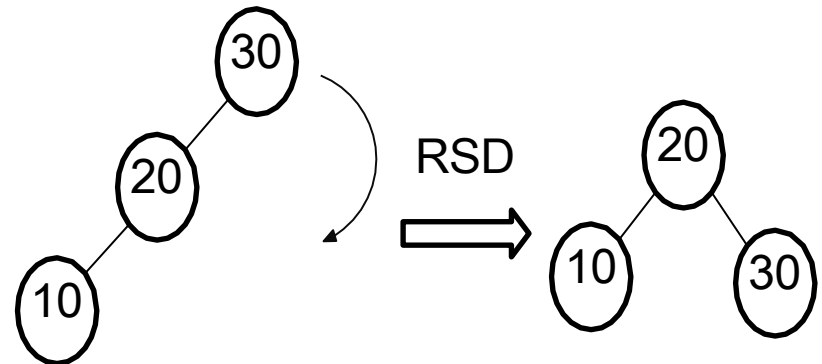


Operações de rotação

- Rotação simples à direita (RSD) ou rotação RR
 - Neste caso a subárvore está mais alta (profunda) no lado esquerdo
 - Situação para realização de uma RSD
 - FB do nó desbalanceado for positivo e;
 - Nó à esquerda também apresentar FB positivo

- Exemplo: RSD(30,20)

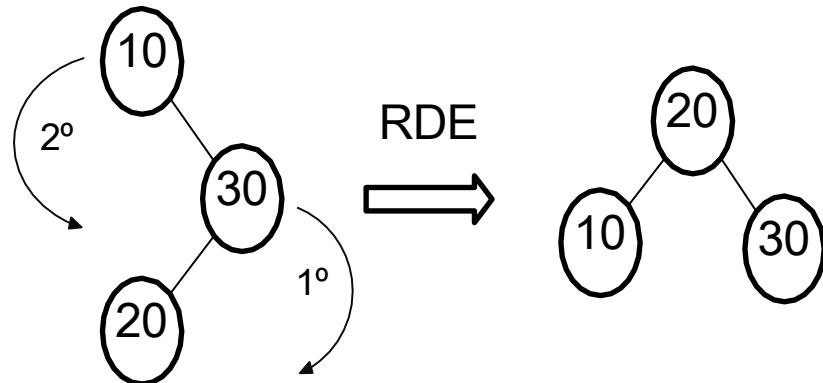
- Nó 30 passa a ser filho de 20
- Nó 20 passa a ser pai de 10 e 30



Operações de rotação

- Rotação dupla à esquerda (RDE) ou rotação RL
 - Neste caso a subárvore está mais alta (profunda) no lado direito
 - Situação para realização de uma RDE
 - FB do nó desbalanceado for negativo e;
 - Nó a direita apresentar FB positivo
- Exemplo: RDE(10,30)

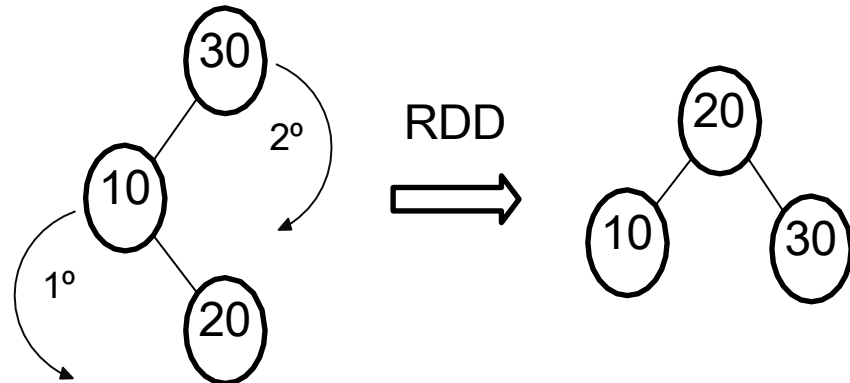
- RSD(30,20)
- RSE(10,20)



Operações de rotação

- Rotação dupla à direita (RDD) ou rotação LR
 - Neste caso a subárvore está mais alta (profunda) no lado esquerdo
 - Situação para realização de uma RDD
 - FB do nó desbalanceado for positivo e;
 - Nó a direita apresentar FB negativo
- Exemplo: RDD(30,10)

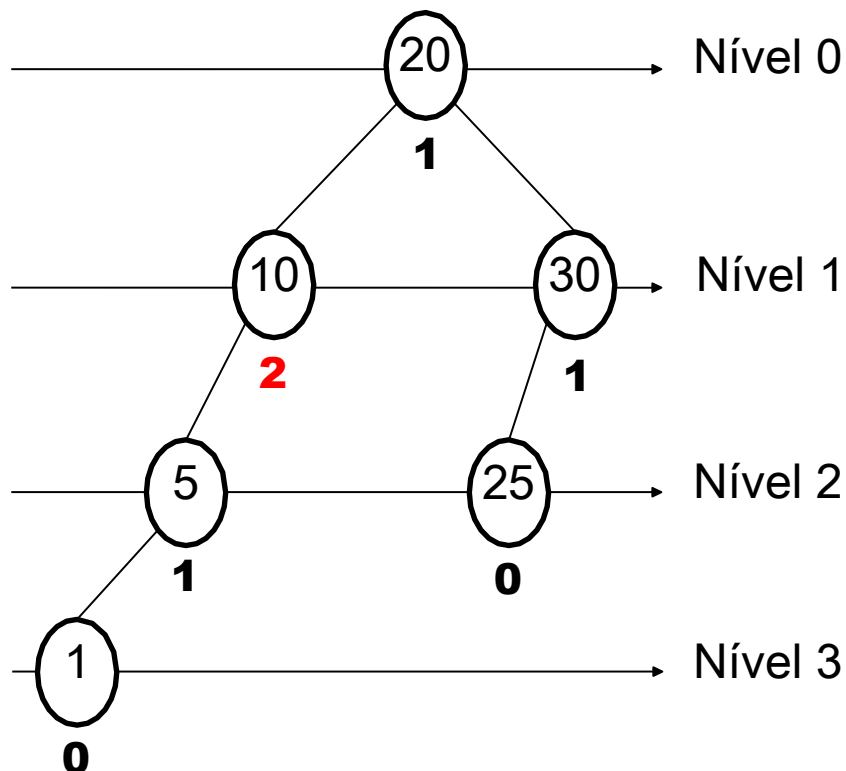
- RSE(10,20)
- RSD(30,20)



Operações de rotação

- Exemplo

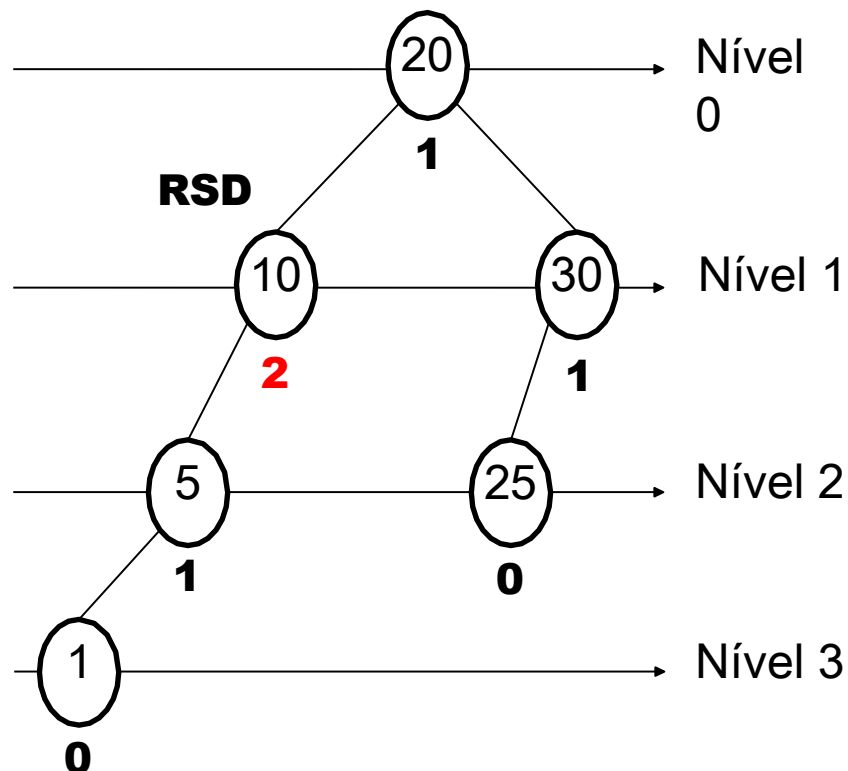
- 20
- 10
- 30
- 5
- 25
- 1



Operações de rotação

- Exemplo

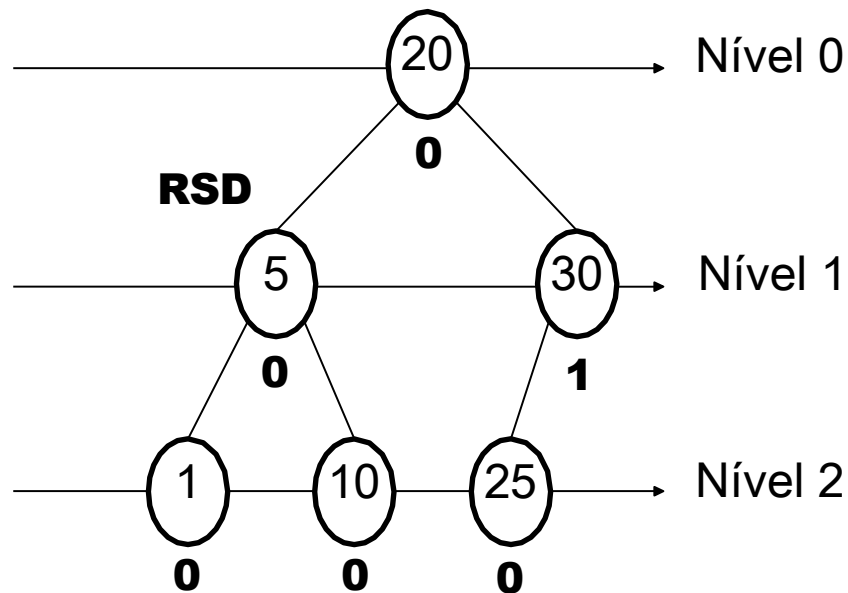
- 20
- 10
- 30
- 5
- 25
- 1



Operações de rotação

- Exemplo

- 20
- 10
- 30
- 5
- 25
- 1



- Altura do nó

```
int altura(No* no) {  
    int esquerda=0, direita=0;  
  
    if (no->esquerda != NULL)  
        esquerda = altura(no->esquerda) + 1;  
  
    if (no->direita != NULL)  
        direita = altura(no->direita) + 1;  
  
    return esquerda > direita ? esquerda : direita;  
}
```

- Fator de balanceamento (FB)

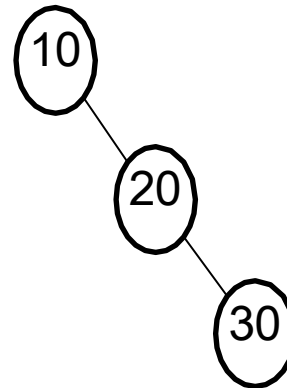
```
int fb(No* no) {  
    int esquerda=0, direita=0;  
  
    if (no->esquerda != NULL)  
        esquerda = altura(no->esquerda) + 1;  
  
    if (no->direita != NULL)  
        direita = altura(no->direita) + 1;  
  
    return esquerda - direita;  
}
```

- Rotação simples à esquerda (RSE)

```
No* rse(No* no) {  
    No* pai = no->pai;  
    No* direita = no->direita;  
  
    no->direita = direita->esquerda;  
    no->pai = direita;  
  
    direita->esquerda = no;  
    direita->pai = pai;  
  
    return direita;  
}
```

- Rotação simples à esquerda (RSE)

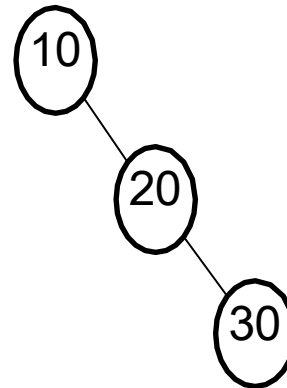
```
No* rse(No* no) {  
    No* pai = no->pai;  
    No* direita = no->direita;  
  
    no->direita = direita->esquerda;  
    no->pai = direita;  
  
    direita->esquerda = no;  
    direita->pai = pai;  
  
    return direita;  
}
```



- pai: NULL
- no: 10
 - pai: NULL
 - esquerda: NULL
 - direita: 20
- direita: 20
 - pai: 10
 - esquerda: NULL
 - direita: 30

- Rotação simples à esquerda (RSE)

```
No* rse(No* no) {  
    No* pai = no->pai;  
    No* direita = no->direita;  
  
    no->direita = direita->esquerda;  
    no->pai = direita;  
  
    direita->esquerda = no;  
    direita->pai = pai;  
  
    return direita;  
}
```



- pai: NULL
- no: 10
 - pai: NULL
 - esquerda: NULL
 - **direita: NULL**
- direita: 20
 - pai: 10
 - esquerda: NULL
 - direita: 30

- Rotação simples à esquerda (RSE)

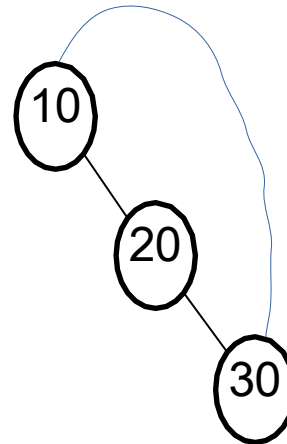
```

No* rse(No* no) {
    No* pai = no->pai;
    No* direita = no->direita;

    no->direita = direita->esquerda;
    no->pai = direita;

    direita->esquerda = no;
    direita->pai = pai;

    return direita;
}
    
```



- pai: NULL
- no: 10
 - **pai: 30**
 - esquerda: NULL
 - direita: NULL
- direita: 20
 - pai: 10
 - esquerda: NULL
 - direita: 30

- Rotação simples à esquerda (RSE)

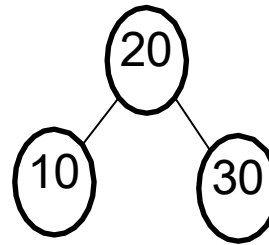
```

No* rse(No* no) {
    No* pai = no->pai;
    No* direita = no->direita;

    no->direita = direita->esquerda;
    no->pai = direita;

    direita->esquerda = no;
    direita->pai = pai;

    return direita;
}
    
```



- pai: NULL
- no: 10
 - pai: 30
 - esquerda: NULL
 - direita: NULL
- direita: 20
 - pai: 10
 - **esquerda: 10**
 - direita: 30

- Rotação simples à esquerda (RSE)

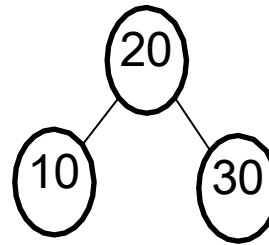
```

No* rse(No* no) {
    No* pai = no->pai;
    No* direita = no->direita;

    no->direita = direita->esquerda;
    no->pai = direita;

    direita->esquerda = no;
    direita->pai = pai;

    return direita;
}
    
```



- pai: **NULL**
- no: 10
 - pai: 30
 - esquerda: NULL
 - direita: NULL
- direita: 20
 - **pai: NULL**
 - esquerda: 10
 - direita: 30

- Rotação simples à direita (RSD)

```
No* rsd(No* no) {  
    No* pai = no->pai;  
    No* esquerda = no->esquerda;  
  
    no->esquerda = esquerda->direita;  
    no->pai = esquerda;  
  
    esquerda->direita = no;  
    esquerda->pai = pai;  
  
    return esquerda;  
}
```

- Rotação simples à direita (RSD)

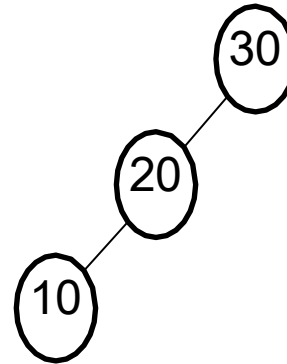
```

No* rsd(No* no) {
    No* pai = no->pai;
    No* esquerda = no->esquerda;

    no->esquerda = esquerda->direita;
    no->pai = esquerda;

    esquerda->direita = no;
    esquerda->pai = pai;

    return esquerda;
}
    
```



- pai: NULL
- no: 30
 - pai: NULL
 - esquerda: 20
 - direita: NULL
- esquerda: 20
 - pai: 30
 - esquerda: 10
 - direita: NULL

Árvores AVL

- Rotação simples à direita (RSD)

```
No* rsd(No* no) {
    No* pai = no->pai;
    No* esquerda = no->esquerda;
```

```
    no->esquerda = esquerda->direita;
```

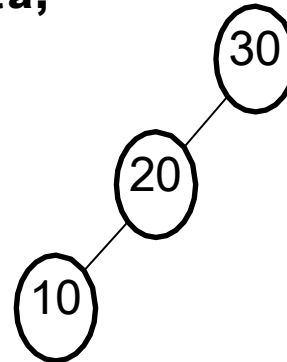
```
    no->pai = esquerda;
```

```
    esquerda->direita = no;
```

```
    esquerda->pai = pai;
```

```
    return esquerda;
```

```
}
```



- pai: NULL
- no: 30
 - pai: NULL
 - **esquerda: NULL**
 - direita: NULL
- esquerda: 20
 - pai: 30
 - esquerda: 10
 - direita: **NULL**

- Rotação simples à direita (RSD)

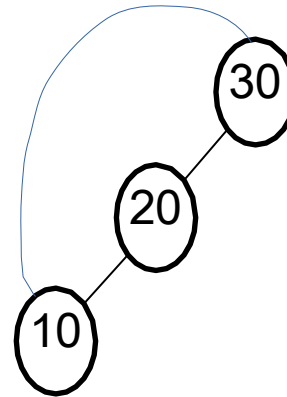
```

No* rsd(No* no) {
    No* pai = no->pai;
    No* esquerda = no->esquerda;

    no->esquerda = esquerda->direita;
    no->pai = esquerda;

    esquerda->direita = no;
    esquerda->pai = pai;

    return esquerda;
}
    
```



- pai: NULL
- no: 30
 - **pai: 20**
 - esquerda: NULL
 - direita: NULL
- esquerda: 20
 - pai: 30
 - esquerda: 10
 - direita: NULL

- Rotação simples à direita (RSD)

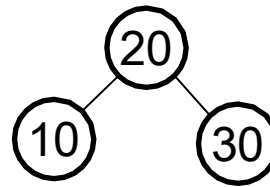
```

No* rsd(No* no) {
    No* pai = no->pai;
    No* esquerda = no->esquerda;

    no->esquerda = esquerda->direita;
    no->pai = esquerda;

    esquerda->direita = no;
    esquerda->pai = pai;

    return esquerda;
}
    
```



- pai: NULL
- no: 30
 - pai: 20
 - esquerda: NULL
 - direita: NULL
- esquerda: 20
 - pai: 30
 - esquerda: 10
 - **direita: 30**

- Rotação simples à direita (RSD)

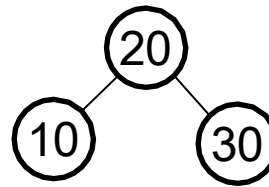
```

No* rsd(No* no) {
    No* pai = no->pai;
    No* esquerda = no->esquerda;

    no->esquerda = esquerda->direita;
    no->pai = esquerda;

    esquerda->direita = no;
    esquerda->pai = pai;

    return esquerda;
}
    
```



- pai: **NULL**
- no: **30**
 - pai: **20**
 - esquerda: **NULL**
 - direita: **NULL**
- esquerda: **20**
 - **pai: NULL**
 - esquerda: **10**
 - direita: **30**

- Rotação dupla à esquerda (RDE)

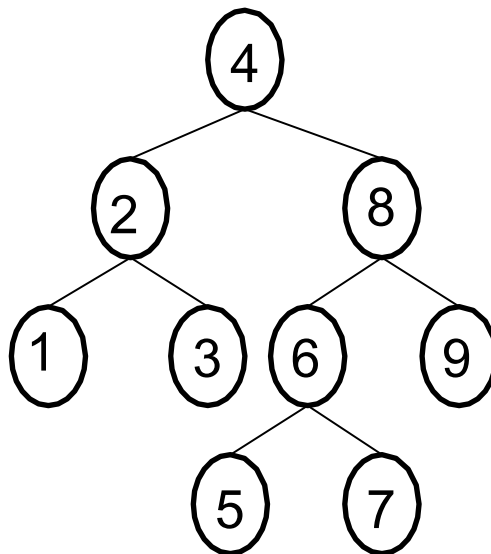
```
No* rde(No* no) {  
    no->direita = rsd(no->direita);  
    return rse(no);  
}
```

- Rotação dupla à direita (RDD)

```
No* rdd(No* no) {  
    no->esquerda = rse(no->esquerda);  
    return rsd(no);  
}
```

Exercícios

1. Implemente uma árvore AVL e adicione os nós de modo que a árvore apresente a respectiva topologia abaixo.



2. Implemente a operação de remoção de nós em uma árvore AVL e valide removendo o nó 6 do exercício anterior.

Heap Sort

Heap Sort

É um arranjo, onde os dados estão organizados de forma que podem ser acessados como se estivessem armazenados em uma árvore binária.

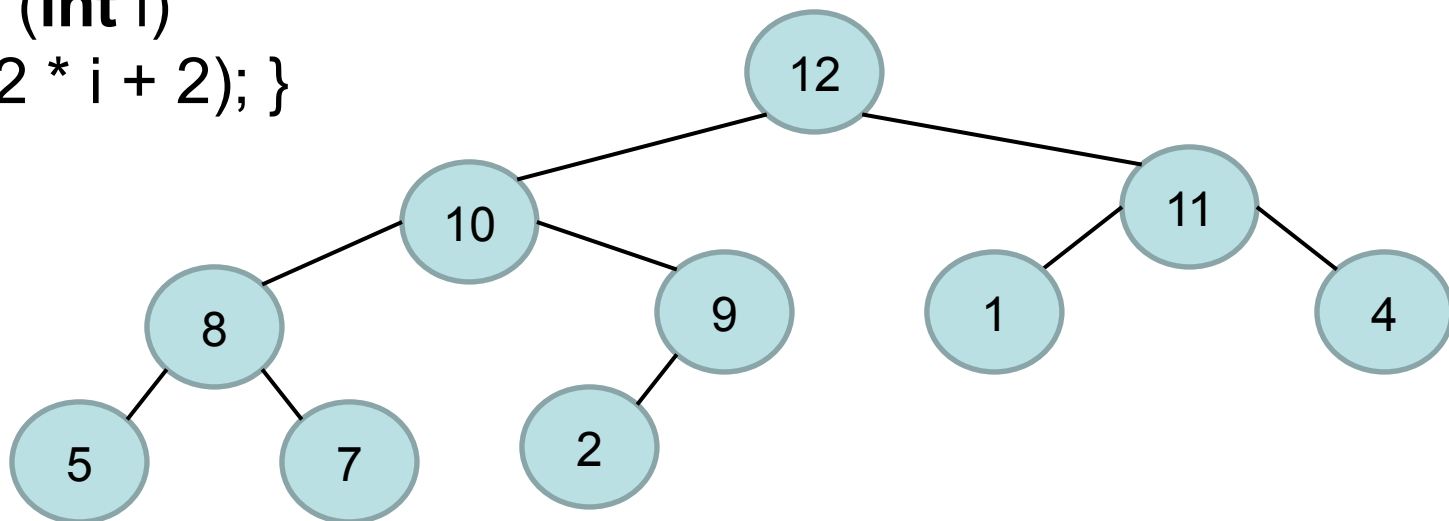
No caso de um *heap máximo*, os elementos armazenado em uma sub-árvore serão sempre menores que o elemento armazenado na raiz. Essa árvore é completa todos seus níveis, com a possível exceção do nível mais baixo.

Heap Sort

0	1	2	3	4	5	6	7	8	9
12	10	11	8	9	1	4	5	7	2

```
int esquerda (int i)  
{ return (2 * i + 1); }
```

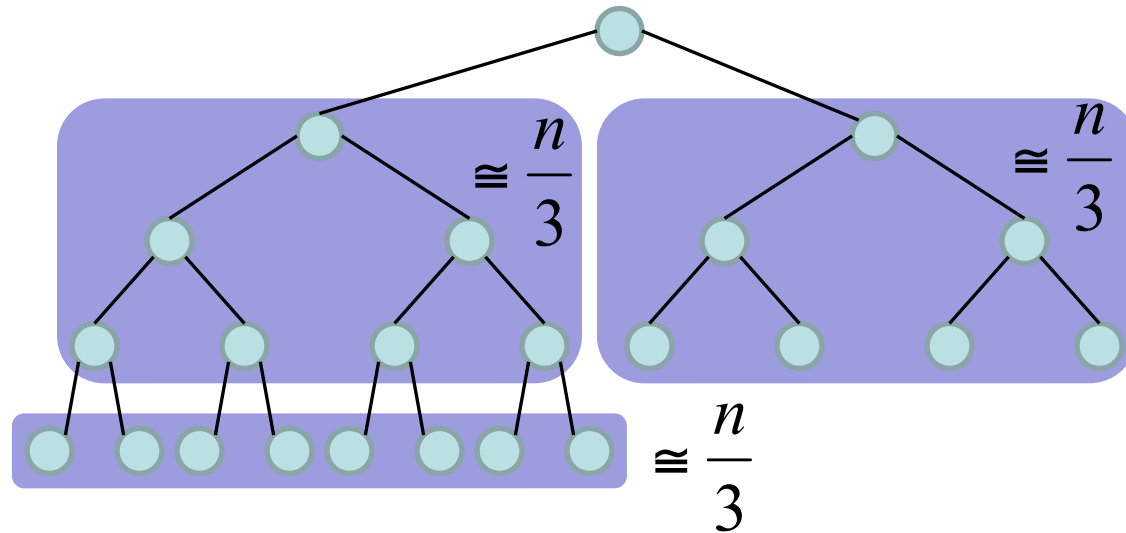
```
int direita (int i)  
{ return (2 * i + 2); }
```



Heap Sort

```
// n = tamanho // i = índice
void heapify (int *a, int n, int i) {
    e = esquerda( i );
    d = direita( i );
    maior = i;
    if (e < n && a[e] > a[maior])
        maior = e;
    if (d < n && a[d] > a[maior])
        maior = d;
    if ( maior != i ) {
        swap (&a[i], &a[maior]);
        heapify(a, n, maior);
    }
}
```

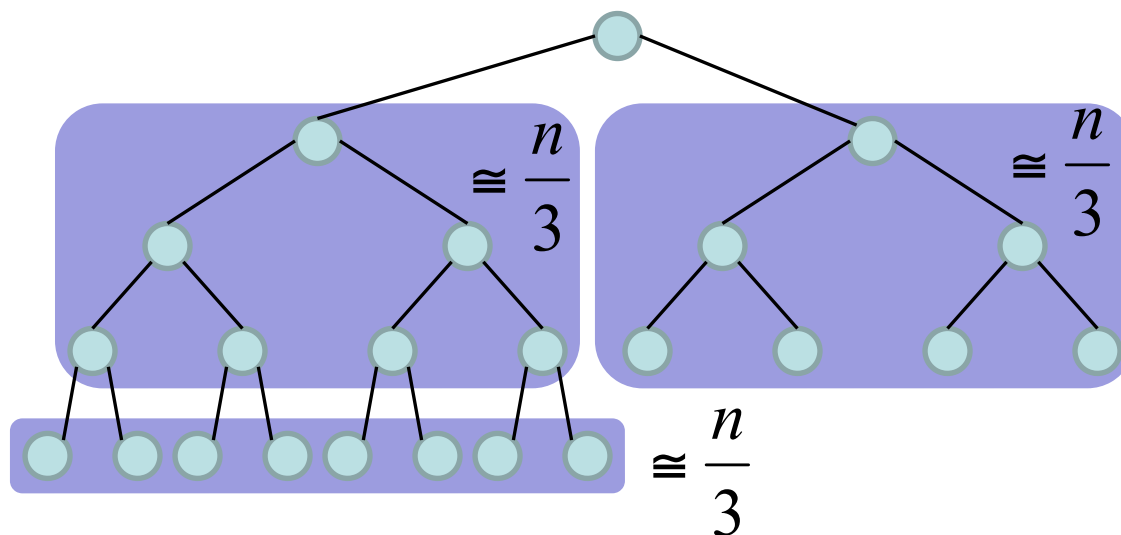
Heap Sort



$$T(n) = T(2n/3) + O(1)$$

$$T(1) = O(1)$$

Heap Sort



Note ainda que existem no máximo: $\left\lceil \frac{n}{2^{h+1}} \right\rceil$ nós de altura h

$n=23 \Rightarrow h=0 : 12$ (folha)

$h=1 : 6$

$h=2 : 3$

$h=3 : 2$

(Obs: altura de baixo p/ cima)

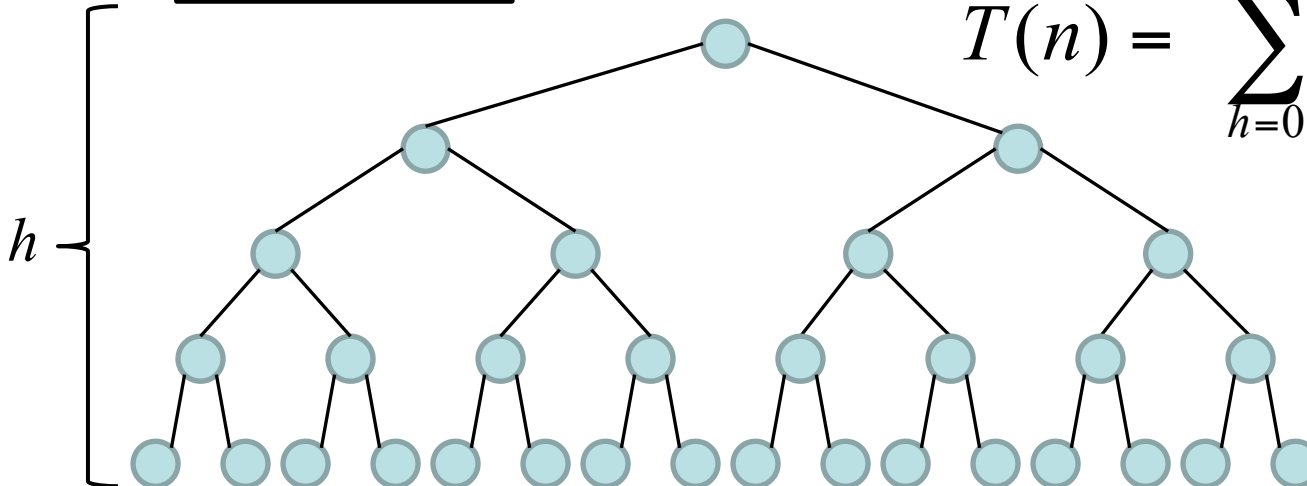
Heap Sort

```
void buildHeap ( int *a, int n ) {
    int i;
    for (i = n/2; i >= 0; i--)      //O(n)
        heapify(a, n, i);
    //O(log n)
}
```

heapify varia com a altura do nó!

$O(n \log n)$

$$T(n) = \sum_{h=0}^{\lfloor \log_2 n \rfloor} [(n / 2^{h+1}) O(h)]$$



Heap Sort

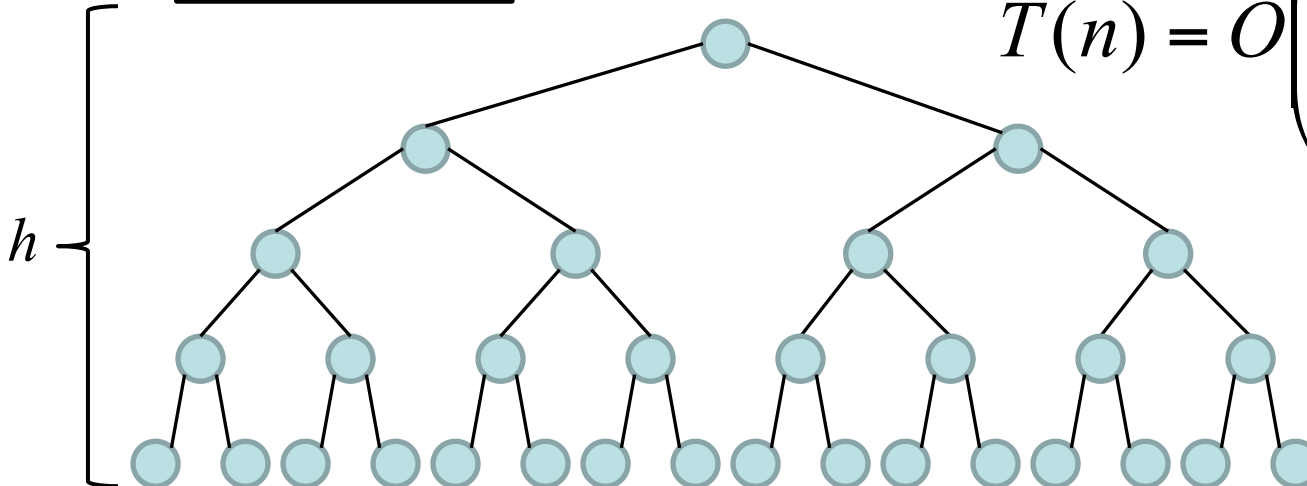
```
void buildHeap ( int *a, int n ) {
    int i;
    for (i = n/2; i >= 0; i--)      //O(n)
        heapify(a, n, i);
    //O(log n)
}
```

heapify varia com a altura do nó!

$O(n \log n)$

$$T(n) = O\left(n \sum_{h=0}^{\lceil \log_2 n \rceil} (h / 2^h)\right)$$

$\Theta(n)$



Heap Sort

```
void heapSort( int *a, int n) {  
    buildHeap( a , n );           //  $\Theta(n)$   
    for (i=n-1; i>0; i--) {       //  $\Theta(n)$   
        swap(&a[0], &a[i]);       //  $\Theta(1)$   
        heapify(a, i, 0);         //  $\Theta(\log n)$   
    }  
}
```

$\Theta(n \log n)$

Árvores Rubro-Negras

Árvores Rubro-Negras

- Árvore rubro-negra é uma árvore de busca binária auto balanceada
 - Assim como em árvore AVL, a rubro-negra possui altura **$O(\log n)$**
 - No entanto, o pior caso de tempo na prática é mais eficiente que a AVL
 - Operações de busca, inserção e remoção ocorrem em tempo **$O(\log n)$**
 - Para alcançar este desempenho, a árvore se mantém aproximadamente balanceada ao inserir ou remover nós
- A árvore rubro-negra foi inventada em 1972 por Rudolf Bayer
 - Inicialmente foi chamada de árvores binárias B simétricas
 - Popularizou com este nome após artigo de Guibas e Sedgewick em 1978

Árvores Rubro-Negras

- Comparação com árvore AVL
 - Em teoria possuem a mesma complexidade computacional
 - Inserção, remoção e busca é $O(\log n)$
 - Na prática a árvore AVL é
 - Mais rápida na operação de busca
 - Mais lenta nas operações de inserção e remoção de nós
 - A árvore AVL possui o balanceamento mais rígido do que a rubro-negra
 - Isto acelera a operação de busca
 - Porém, exige mais esforço computacional para inserção e remoção de nós

Aplicação de Árvores Rubro-Negras

- As árvores rubro-negra são de uso mais geral do que as árvores AVL
 - Bibliotecas em Java
 - `java.util.TreeMap`
 - `java.util.TreeSet`
 - Bibliotecas C++ STL
 - `set`
 - `map`
 - `multiset`
 - `multimap`
 - Linux kernel
 - Scheduler (vruntime)
 - Filesystem (ext3/ext4)
 - Virtual Memory Areas (VMAs)

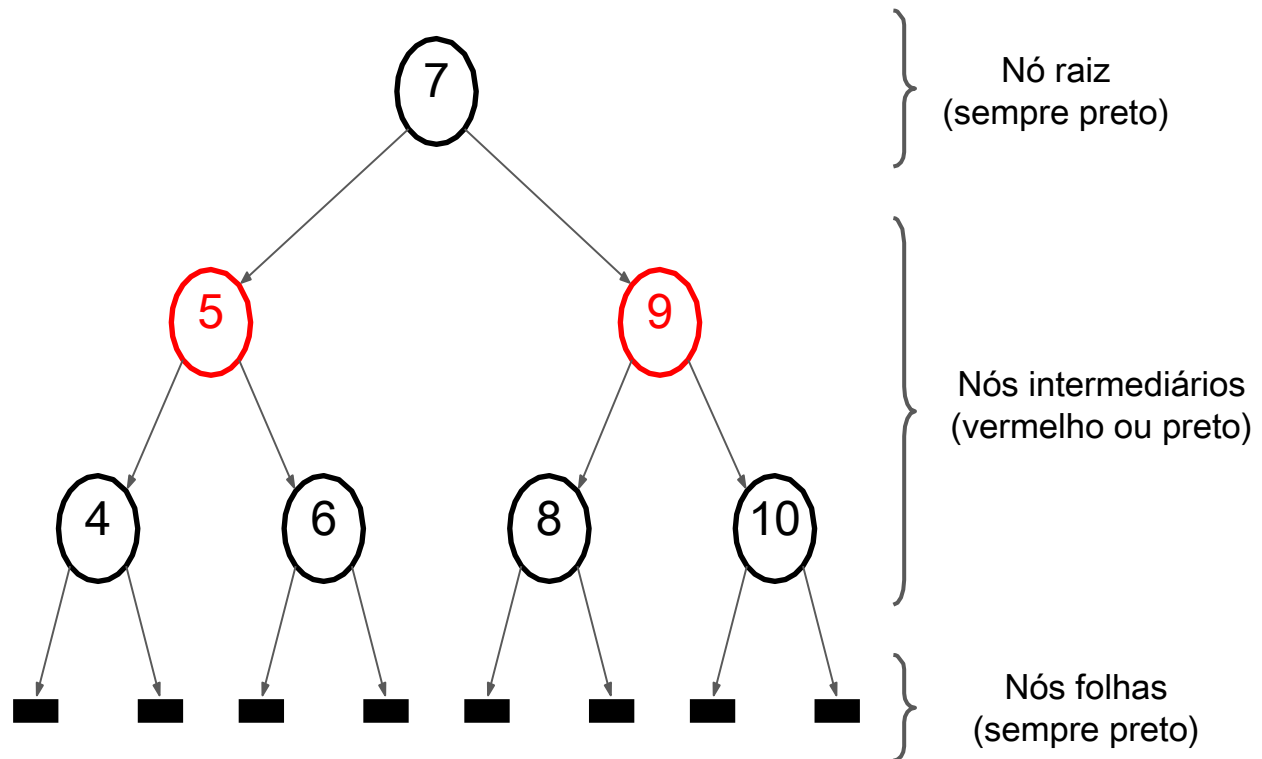
Propriedades da Árvores Rubro-Negras

- Todo nó da árvore possui um atributo adicional que define a cor
 - A cor do nó será utilizada para o balanceamento dos nós
 - Ou seja, as operações de balanceamento baseiam-se na cor dos nós
- Uma árvore rubro-negra deve respeitar as seguintes as regras
 - Regra 1: Todo nó possui cor vermelha ou é preta
 - Regra 2: a raiz sempre será preta
 - Regra 3: todo nó folha possui cor preta
 - Regra 4: se um nó é vermelho então ambos os seus filhos são pretos
 - Regra 5: para cada nó, todos os caminhos deste nó até os nós folha da sua subárvore possuem o mesmo número de nós pretos

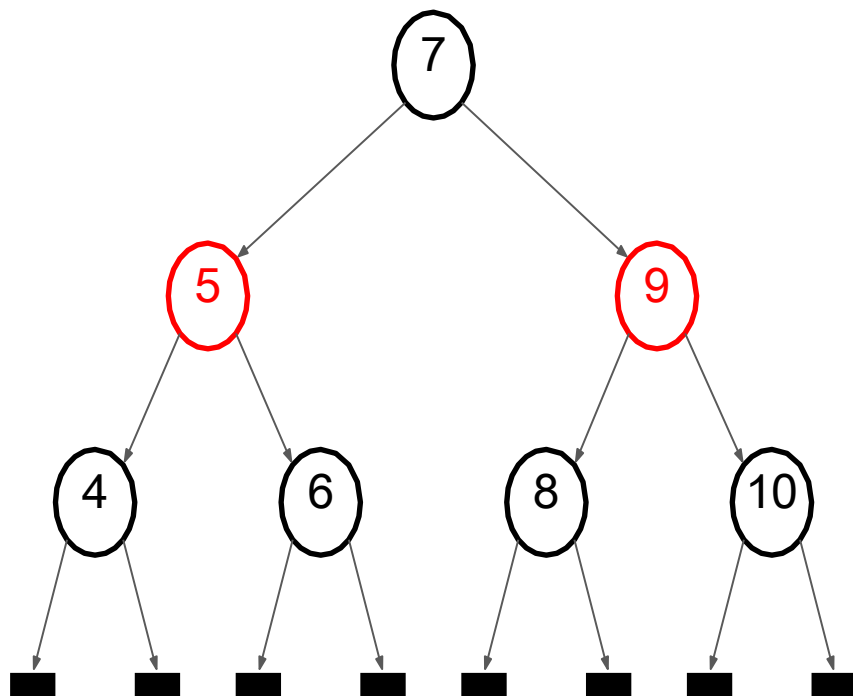
Propriedades da Árvores Rubro-Negras

- Ao restringir a maneira que os nós podem ser coloridos do caminho da raiz até qualquer folhas, assegura-se que
 - Nenhum caminho será maior que o dobro do comprimento de outro
 - A árvore será aproximadamente balanceada
- Em árvores rubro-negra, os nós folha são irrelevantes e sem dados
 - É comum os nós folhas serem representados com ponteiros nulos
 - Porém algumas operações são simplificadas ao explicitar os nós folha
 - Usa-se um nó sentinela e todos os ponteiros para as folhas apontam para ele

Coloração dos nós

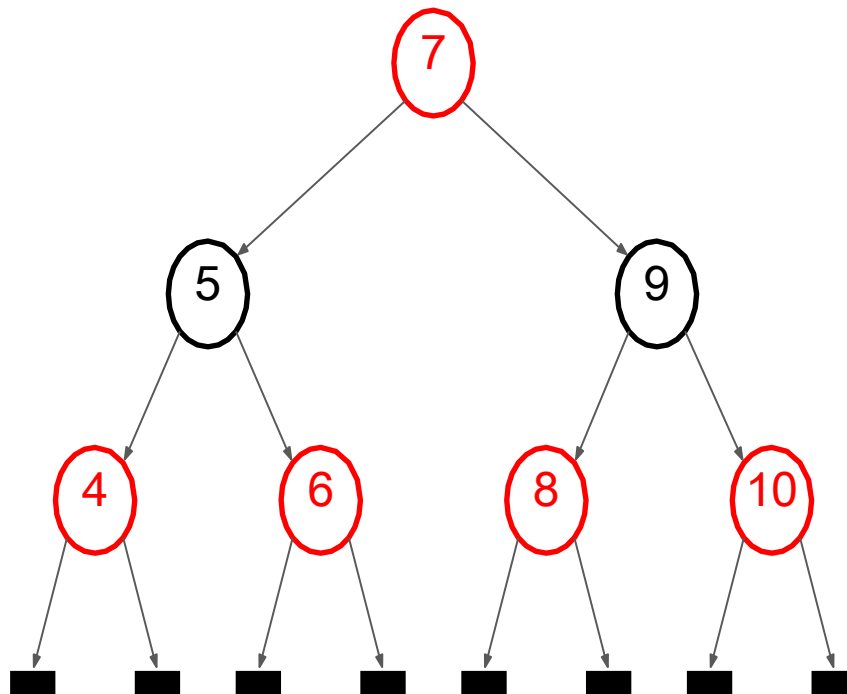


Coloração dos nós



- ✓ Regra 1: Todo nó é vermelho ou é preto
- ✓ Regra 2: a raiz sempre será preto
- ✓ Regra 3: todo nó folha é preto
- ✓ Regra 4: se um nó é vermelho então ambos os seus filhos são pretos
- ✓ Regra 5: todos os caminhos de um nó até as folhas possuem o mesmo número de nós pretos

Coloração dos nós



Regra 1: Todo nó é vermelho ou é preto



Regra 2: a raiz sempre será preto



Regra 3: todo nó folha é preto

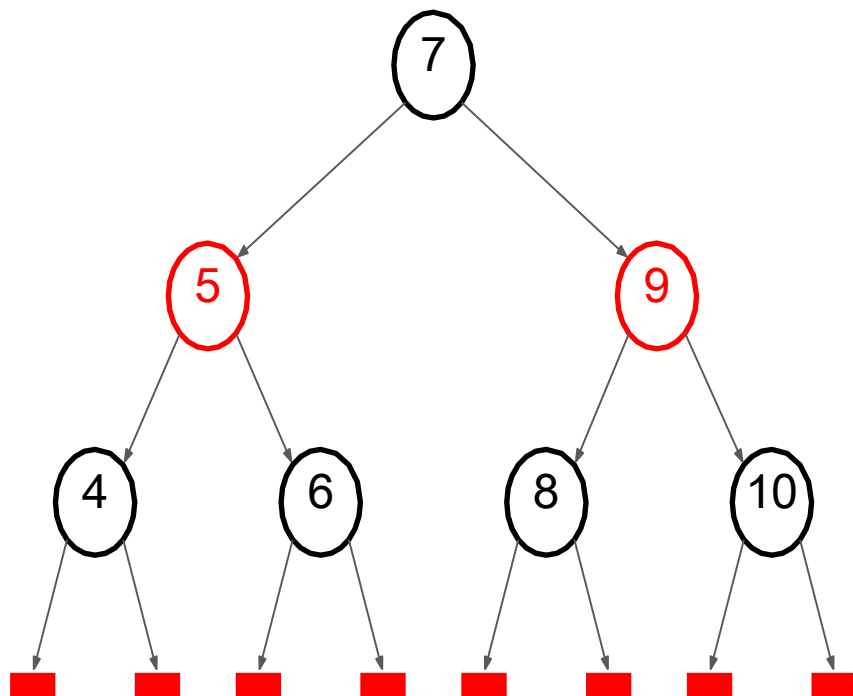


Regra 4: se um nó é vermelho então ambos os seus filhos são pretos



Regra 5: todos os caminhos de um nó até as folhas possuem o mesmo número de nós pretos

Coloração dos nós



Regra 1: Todo nó é vermelho ou é preto



Regra 2: a raiz sempre será preto



Regra 3: todo nó folha é preto

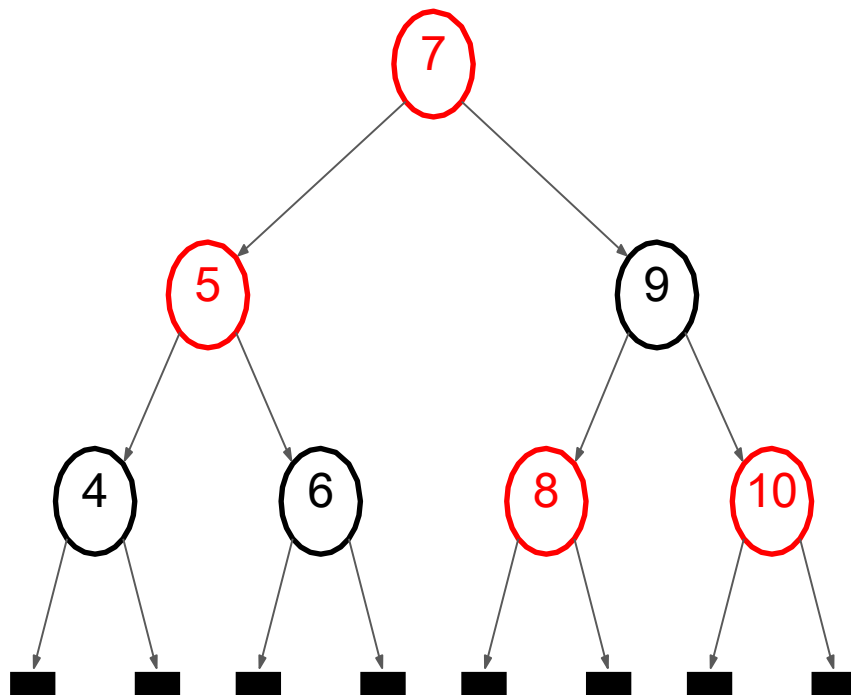


Regra 4: se um nó é vermelho então ambos os seus filhos são pretos



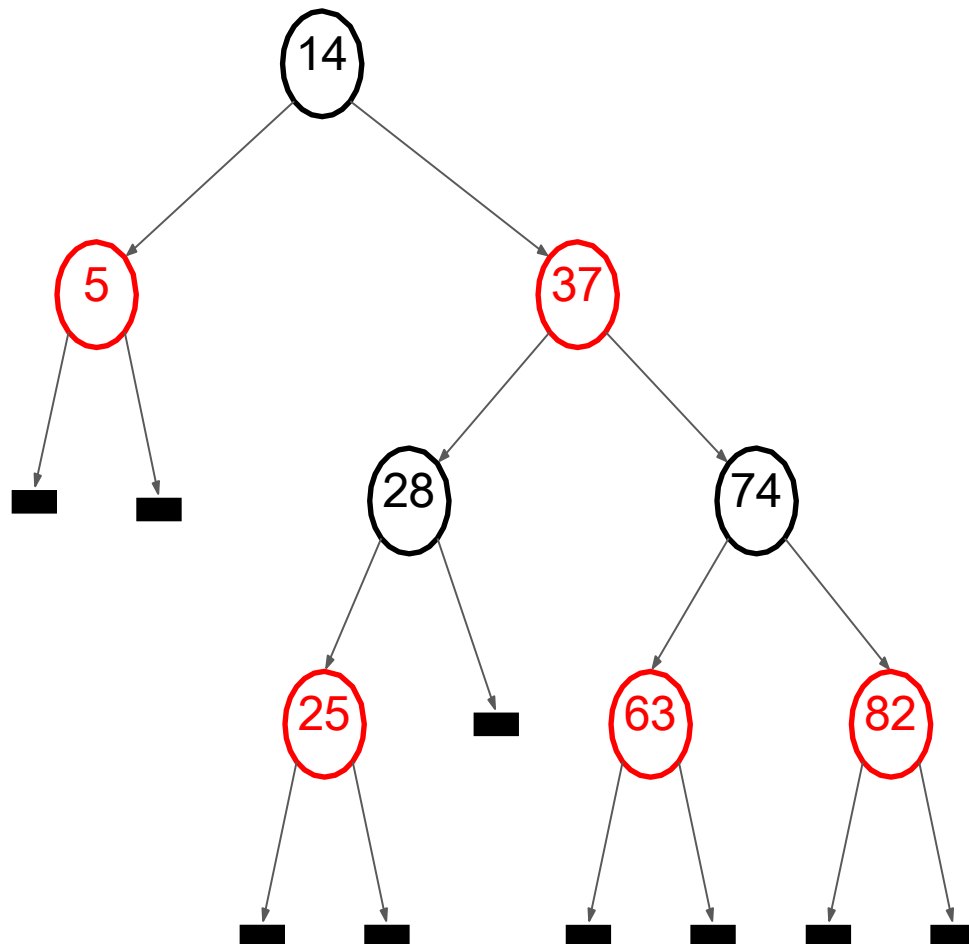
Regra 5: todos os caminhos de um nó até as folhas possuem o mesmo número de nós pretos

Coloração dos nós



- ✓ Regra 1: Todo nó é vermelho ou é preto
- ✓ Regra 2: a raiz sempre será preto
- ✓ Regra 3: todo nó folha é preto
- ✗ Regra 4: se um nó é vermelho então ambos os seus filhos são pretos
- ✓ Regra 5: todos os caminhos de um nó até as folhas possuem o mesmo número de nós pretos

Coloração dos nós



Regra 1: Todo nó é vermelho ou é preto



Regra 2: a raiz sempre será preto



Regra 3: todo nó folha é preto



Regra 4: se um nó é vermelho então ambos os seus filhos são pretos



Regra 5: todos os caminhos de um nó até as folhas possuem o mesmo número de nós pretos

Estrutura da Árvore Rubro-Negra

- Representação em C

```
typedef enum coloracao { Vermelho, Preto } Cor;
```

```
typedef struct no {  
    struct no* pai;  
    struct no* esquerda;  
    struct no* direita;  
    Cor cor; //cor do nó (Vermelho ou Preto)  
    int valor;  
} No;
```

Operação em Árvore Rubro-Negra

- Adicionar novos elementos
 - Cria um novo nó
 - Atribui a cor vermelha ao nó
 - Associa o nó à chave que deseja ser adicionada
 - Localiza onde o novo nó que será adicionado
 - Realiza uma busca binária para localizar o nó
 - Realiza o balanceamento prevendo o nó recém inserido
 - Corrige possíveis violações das regras da árvore rubro-negra

Balanceamento da árvore rubro-negra

- Existem duas operações para rebalancear uma árvore rubro-negra
 - Rotação a esquerda ou à direita
 - São operações locais que alteram alguns ponteiros dos nós relacionados
 - Recoloração de nós envolvidos
 - A fim de evitar dois nós vizinhos com cor vermelha
- O rebalanceamento pode ocorrer em 3 casos distintos
 - Visam corrigir possíveis violações das regras da árvore rubro-negra

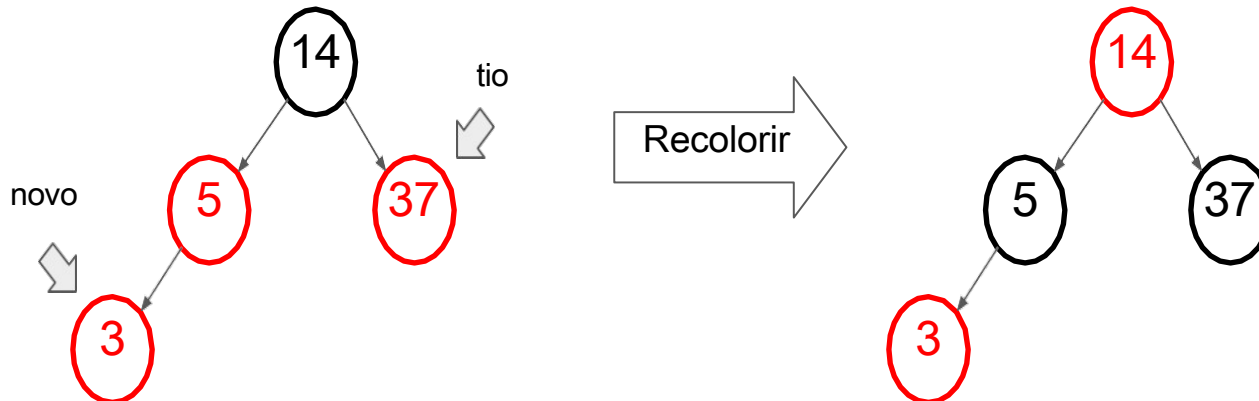
Balanceamento da árvore rubro-negra

- Caso “0”: árvore vazia
 - Ocorre após adicionar o nó raiz, a cor do nó será vermelha
 - Este caso viola a regra 2 (o nó raiz precisa ser preto)
 - Para corrigir, deve ser atribuída a cor preta à raiz da árvore
 - Exemplo
 - Adicionar o nó 14
 - Recolorir nó 14



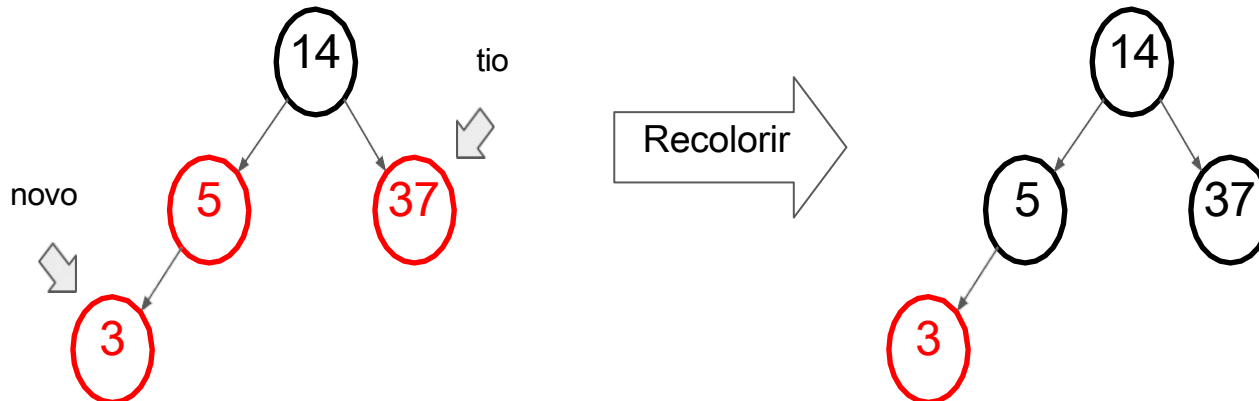
Balanceamento da árvore rubro-negra

- Caso 1: nós pai e filho vermelhos com **tio vermelho**
 - Ocorre após adicionar um nó como filho de outro de cor vermelha
 - Este caso viola a regra 4 (nó vermelho possui ambos os filhos pretos)
 - Para corrigir, deve ser atribuída a cor preta à raiz da árvore
 - Exemplo
 - Adicionar o nó 3
 - Recolorir nós 5 e 37



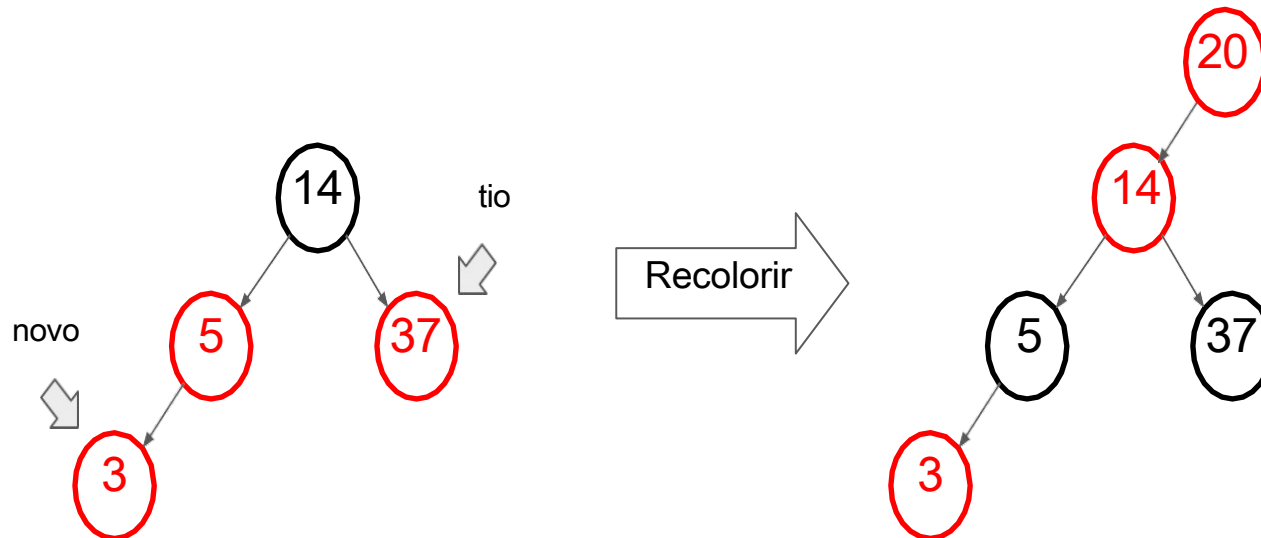
Balanceamento da árvore rubro-negra

- Caso 1: nós pai e filho vermelhos com **tio vermelho** (cont.)
 - Exemplo
 - Adicionar o nó 3
 - Recolorir nós 5 e 37
 - E se o nó 14 for raiz da árvore?
 - Nesse caso, basta recolorir o nó raiz



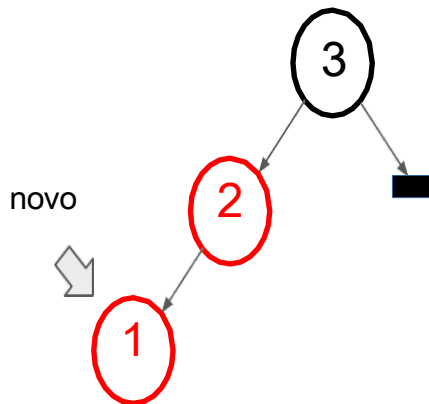
Balanceamento da árvore rubro-negra

- Caso 1: nós pai e filho vermelhos com **tio vermelho** (cont.)
 - Exemplo
 - Adicionar o nó 3
 - Recolorir nós 5 e 37
 - E se o nó 14 for filho de um nó vermelho?
 - O processo deve ser feito iterativamente nos níveis anteriores da árvore
 - A mudança de cor de um nó pode afetar os níveis anteriores



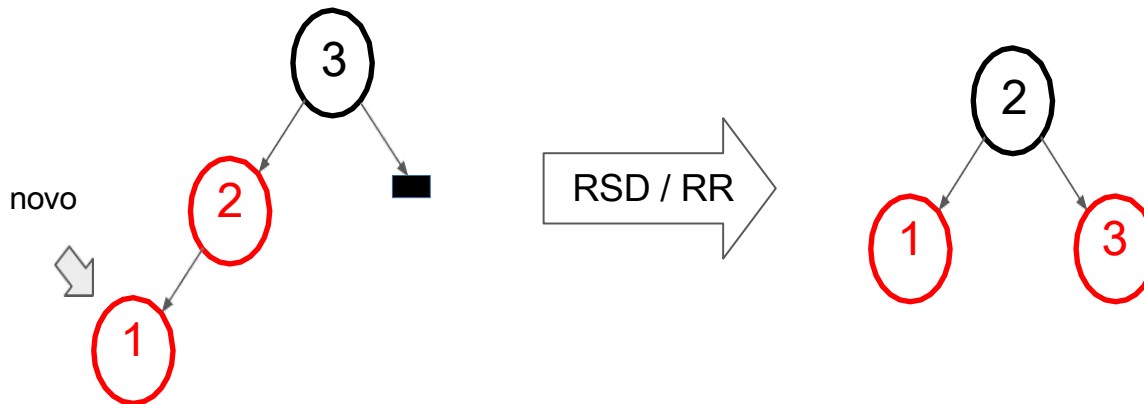
Balanceamento da árvore rubro-negra

- Caso 2: nós pai e filho vermelhos com **tio preto**
 - Quando forma uma “linha reta” entre pai e avô: exige uma **rotação simples** (como na árvore AVL)
 - Pai fica preto
 - Avô fica vermelho
 - Rotação simples à esquerda/direita



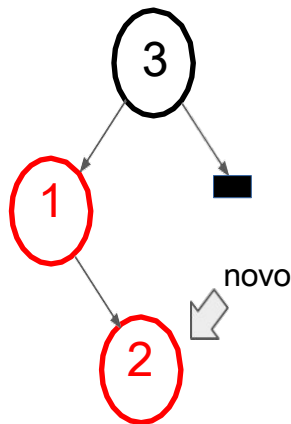
Balanceamento da árvore rubro-negra

- Caso 2: nós pai e filho vermelhos com **tio preto**
 - Quando forma uma “linha reta” entre pai e avô: exige uma **rotação simples** (como na árvore AVL)
 - Pai fica preto
 - Avô fica vermelho
 - Rotação simples à esquerda/direita



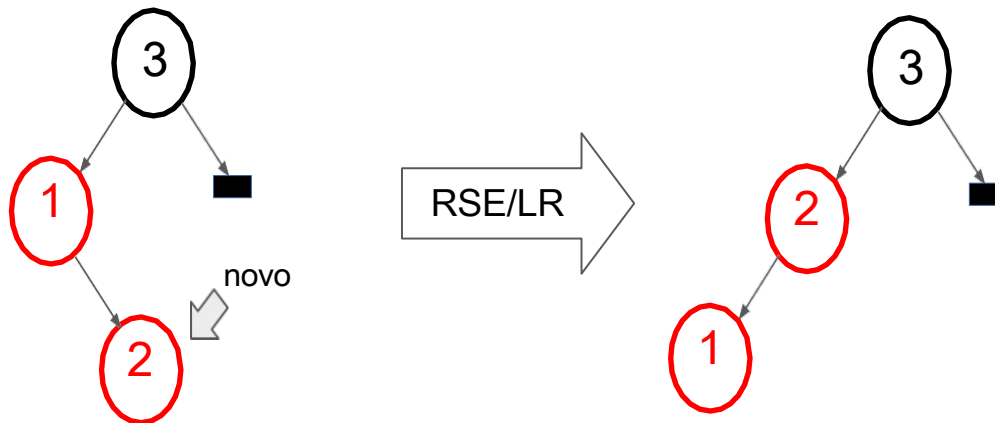
Balanceamento da árvore rubro-negra

- Caso 3: nós pai e filho vermelhos com **tio preto**
 - Quando forma um “zigue-zague” (ou ângulo): exige uma **rotação dupla** (como na árvore AVL)
 - Rotação simples do pai à esquerda/direita
 - Pai fica preto
 - Avô fica vermelho
 - Rotação do avô para a direita/esquerda



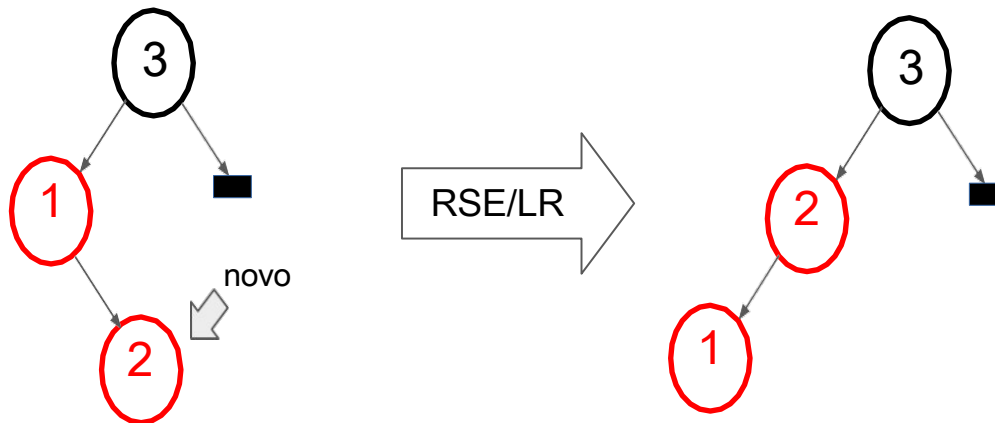
Balanceamento da árvore rubro-negra

- Caso 3: nós pai e filho vermelhos com **tio preto**
 - Quando forma um “zigue-zague” (ou ângulo): exige uma **rotação dupla** (como na árvore AVL)
 - Rotação simples do pai à esquerda/direita
 - Pai fica preto
 - Avô fica vermelho
 - Rotação do avô para a direita/esquerda



Balanceamento da árvore rubro-negra

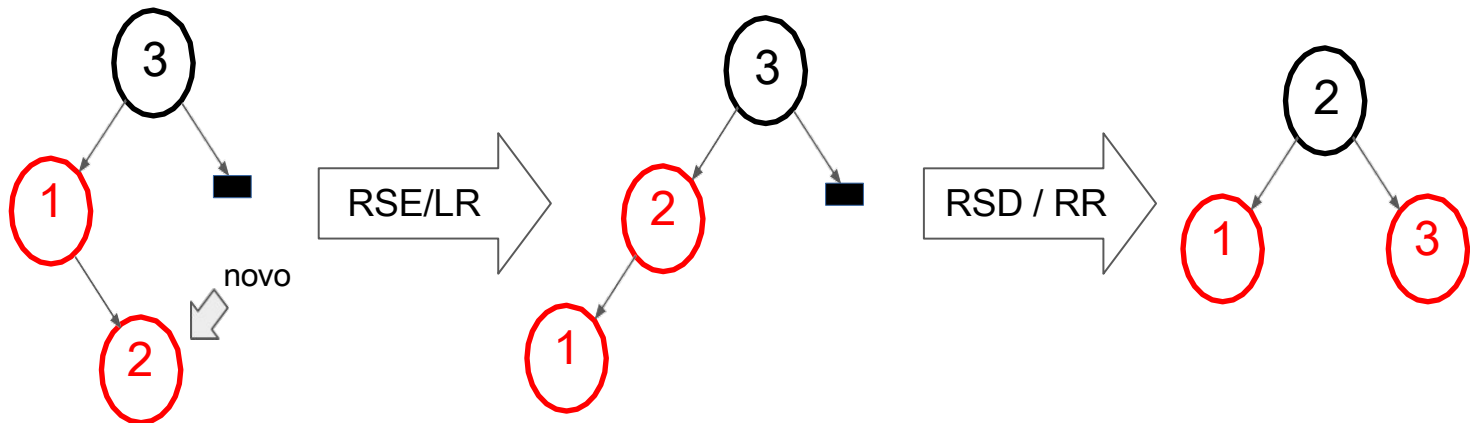
- Caso 3: nós pai e filho vermelhos com **tio preto**
 - Quando forma um “zigue-zague” (ou ângulo): exige uma **rotação dupla** (como na árvore AVL)
 - Rotação simples do pai à esquerda/direita
 - Pai fica preto
 - Avô fica vermelho
 - Rotação do avô para a direita/esquerda



Caso 2!

Balanceamento da árvore rubro-negra

- Caso 3: nós pai e filho vermelhos com **tio preto**
 - Quando forma um “zigue-zague” (ou ângulo): exige uma **rotação dupla** (como na árvore AVL)
 - Rotação simples do pai à esquerda/direita
 - Pai fica preto
 - Avô fica vermelho
 - Rotação do avô para a direita/esquerda



Balanceamento da árvore rubro-negra

- Balanceamento

```
void balancear(Arvore* arvore, No* no) {
    while (no->pai->cor == Vermelho) { //Garante que todos os níveis foram balanceados
        if (no->pai == no->pai->pai->esquerda) {
            No *tio = no->pai->pai->direita;

            if (tio->cor == Vermelho) {
                tio->cor = Preto; //Resolve caso 2
                no->pai->cor = Preto;
                no->pai->pai->cor = Vermelho;
                no = no->pai->pai; //Vai para o nível anterior
            } else {
                if (no == no->pai->direita) {
                    no = no->pai; //Vai para o nível anterior
                    rotacionarEsquerda(arvore, no); //Resolve caso 3
                } else {
                    no->pai->cor = Preto; //Resolve caso 4
                    no->pai->pai->cor = Vermelho;
                    rotacionarDireita(arvore, no->pai->pai);
                }
            }
        } else {
            //Repete o mesmo código do bloco if, invertendo o lado dos direita e esquerda
        }
    }
    arvore->raiz->cor = Preto; //Resolve caso 1
}
```

Balanceamento da árvore rubro-negra

- Rotação à esquerda

```
void rotacionarEsquerda(Arvore* arvore, No* no) {
    No* direita = no->direita;
    no->direita = direita->esquerda;

    if (direita->esquerda != NULL)
        direita->esquerda->pai = no; //Se houver filho à esquerda em direita, ele será pai do nó

    direita->pai = no->pai; //Ajusta no pai do nó à direita

    if (no->pai == NULL)
        arvore->raiz = direita; //Se nó for raiz, o nó direita será a nova raiz da árvore
    else if (no == no->pai->esquerda)
        no->pai->esquerda = direita; //Corrige relação pai-filho do novo pai (esquerda)
    else
        no->pai->direita = direita; //Corrige relação pai-filho do novo pai (direita)

    direita->esquerda = no; //Corrige relação pai-filho entre o nó pivô e o nó à direita
    no->pai = direita;
}
```

Balanceamento da árvore rubro-negra

- Rotação à direita

```
void rotacionarDireita(Arvore* arvore, No* no) {
    No* esquerda = no->esquerda;
    no->esquerda = esquerda->direita;

    if (esquerda->direita != NULL)
        esquerda->direita->pai = no; //Se houver filho à direita em esquerda, ele será pai do nó

    esquerda->pai = no->pai; //Ajusta no pai do nó à esquerda

    if (no->pai == NULL)
        arvore->raiz = esquerda; //Se nó for raiz, o nó esquerda será a nova raiz da árvore
    else if (no == no->pai->esquerda)
        no->pai->esquerda = esquerda; //Corrige relação pai-filho do novo pai (esquerda)
    else
        no->pai->direita = esquerda; //Corrige relação pai-filho do novo pai (direita)

    esquerda->direita = no; //Corrige relação pai-filho entre o nó pivô e o nó à esquerda
    no->pai = esquerda;
}
```


Remoção na árvore rubro-negra

- O procedimento para remoção de um nó é similar aos algoritmos utilizados nas árvores ABB e AVL. Primeiro se busca o elemento a ser removido e depois remove ele mantendo a árvore "ordenada";
- Após a remoção, precisamos verificar se as regras da árvore Rubro-Negra ainda continuam válidas.

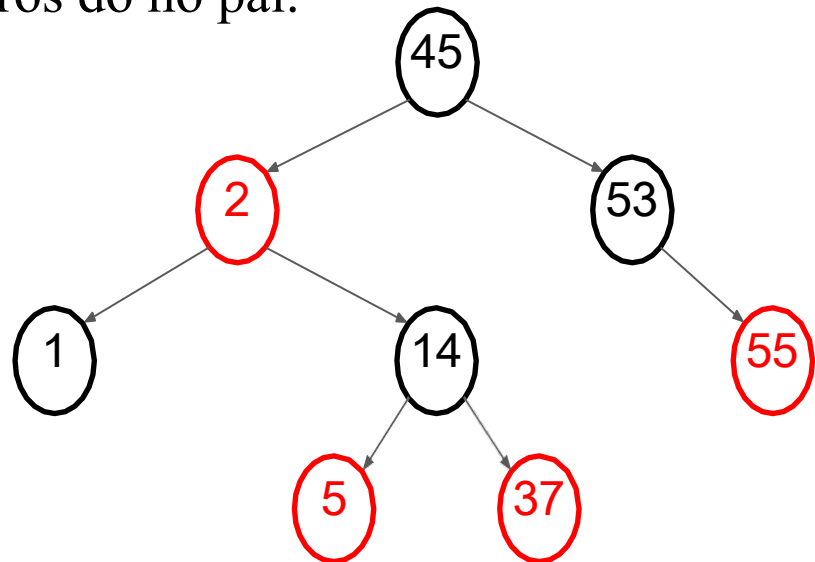
Remoção na árvore rubro-negra

- O procedimento para remoção de um nó é similar aos algoritmos utilizados nas árvores ABB e AVL. Primeiro se busca o elemento a ser removido e depois remove ele mantendo a árvore "ordenada";
- Após a remoção, precisamos verificar se as regras da árvore Rubro-Negra ainda continuam válidas.
- O primeiro caso ocorre quando se remove um nó vermelho que é folha. Neste caso, todas as regras são mantidas sem alteração. Basta, neste caso remover o nó e ajustar os ponteiros do nó pai.

Remoção na árvore rubro-negra

- O procedimento para remoção de um nó é similar aos algoritmos utilizados nas árvores ABB e AVL. Primeiro se busca o elemento a ser removido e depois remove ele mantendo a árvore "ordenada";
- Após a remoção, precisamos verificar se as regras da árvore Rubro-Negra ainda continuam válidas.
- O primeiro caso ocorre quando se remove um nó vermelho que é folha. Neste caso, todas as regras são mantidas sem alteração. Basta, neste caso remover o nó e ajustar os ponteiros do nó pai.

Nesse caso, a remoção dos nós **5**, **37** e **55** não violam as regras da árvore Rubro-Negra.

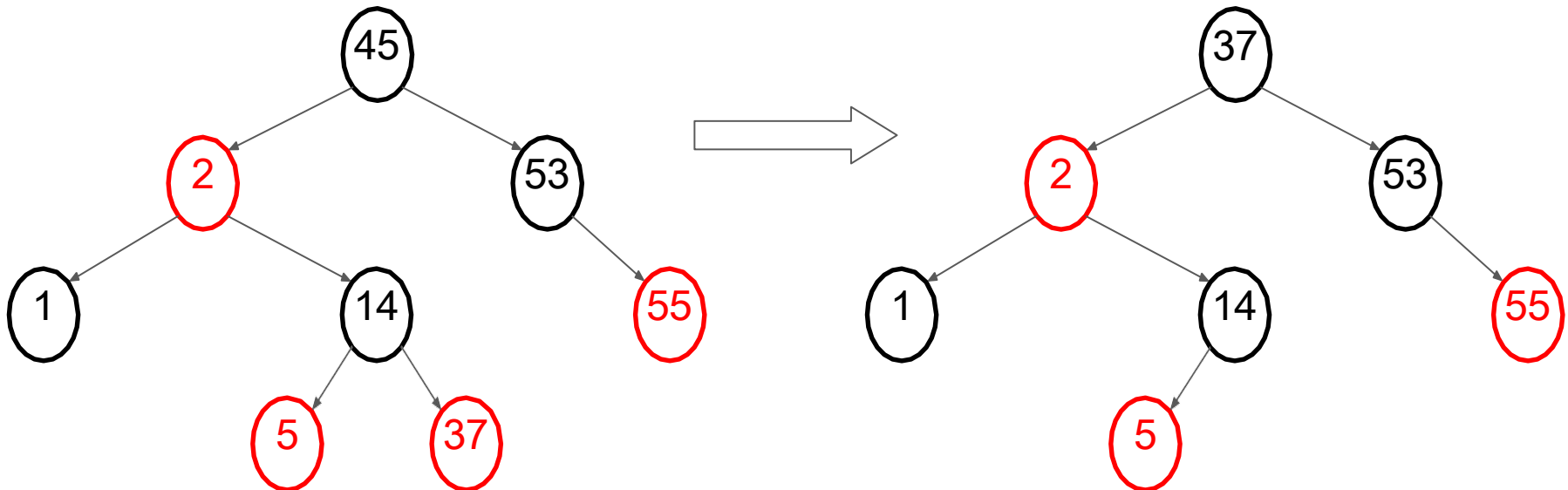


Remoção na árvore rubro-negra

- Lembremos que a remoção de um nó com dois filhos na verdade é apenas uma substituição:
- O maior valor da esquerda (antecessor) ou menor da direita (sucessor) é quem é removido de fato.
- Neste nó é que deve-se verificar as regras de balanceamento após a remoção.

Remoção na árvore rubro-negra

- Lembremos que a remoção de um nó com dois filhos na verdade é apenas uma substituição:
- O maior valor da esquerda (antecessor) ou menor da direita (sucessor) é quem é removido de fato.
- Neste nó é que deve-se verificar as regras de balanceamento após a remoção. *Ao remover o nó 45, substituímos ele por 37 e removemos o nó folha (vermelho no caso), mantendo a cor preta original.*



Remoção na árvore rubro-negra

- A remoção de um nó preto reduz o número de nós pretos em qualquer caminho que continha o nó que foi removido;
- Casos que não precisamos nos preocupar, pois não causa alteração nas regras da árvore Rubro-Negra:
 - Nó preto substituído por um nó vermelho (lembrando que mantém-se a cor preta na substituição);
 - Nó vermelho substituído por outro nó vermelho;
 - Nó vermelho folha removido (substituído por nó nulo).

Remoção na árvore rubro-negra

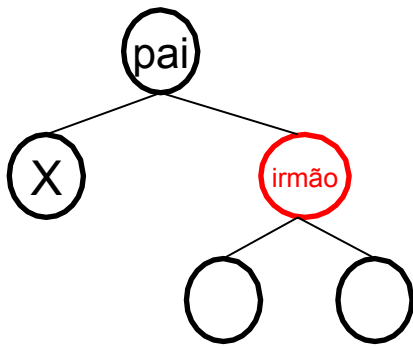
- A remoção de um nó preto reduz o número de nós pretos em qualquer caminho que continha o nó que foi removido;
- Casos que **precisamos** nos preocupar, pois causa alteração nas regras da árvore Rubro-Negra:
 - Nó preto substituído por um nó preto;
 - Nó preto substituído por uma folha nula;

Remoção na árvore rubro-negra

- A remoção de um nó preto reduz o número de nós pretos em qualquer caminho que continha o nó que foi removido;
- Casos que **precisamos** nos preocupar, pois causa alteração nas regras da árvore Rubro-Negra:
 - Nó preto substituído por um nó preto;
 - Nó preto substituído por uma folha nula;
- Se o nó é raiz, o caminho da raiz até as folhas não muda (já que muda na árvore toda).

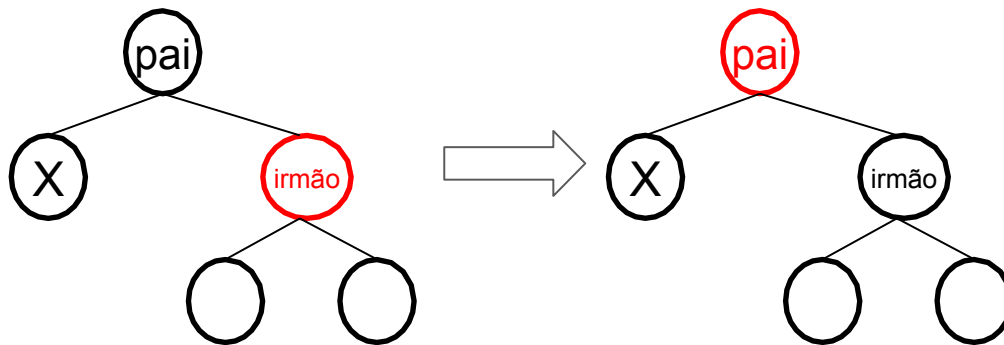
Remoção na árvore rubro-negra

- A remoção de um nó preto reduz o número de nós pretos em qualquer caminho que continha o nó que foi removido;
- Casos 1: irmão é vermelho
 - Pai fica vermelho;
 - Irmão fica preto;
 - Rotaciona pai à esquerda;



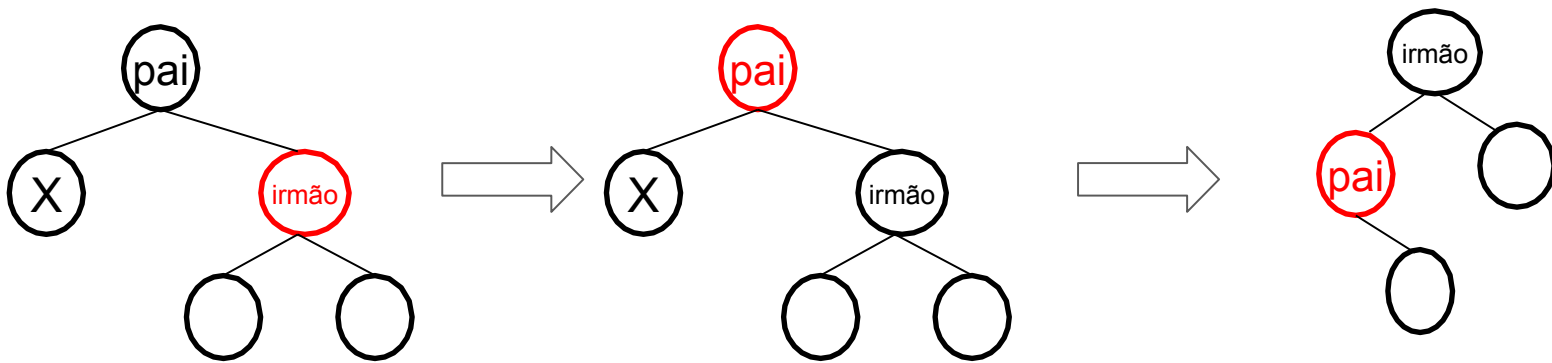
Remoção na árvore rubro-negra

- A remoção de um nó preto reduz o número de nós pretos em qualquer caminho que continha o nó que foi removido;
- Casos 1: irmão é vermelho
 - Pai fica vermelho;
 - Irmão fica preto;
 - Rotaciona pai à esquerda;



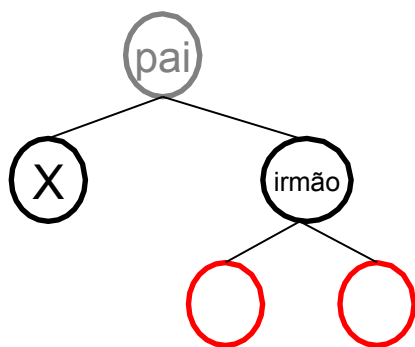
Remoção na árvore rubro-negra

- A remoção de um nó preto reduz o número de nós pretos em qualquer caminho que continha o nó que foi removido;
- Casos 1: irmão é vermelho
 - Pai fica vermelho;
 - Irmão fica preto;
 - Rotaciona pai à esquerda;



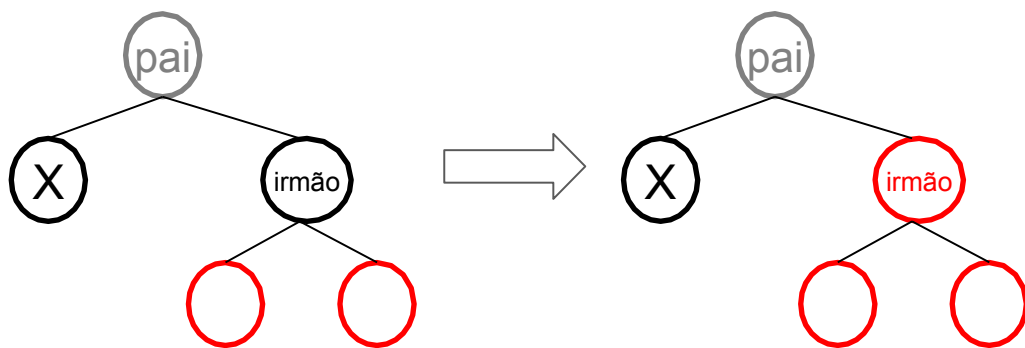
Remoção na árvore rubro-negra

- Casos 2: irmão é preto e filhos do irmão (sobrinhos) são vermelhos
 - Irmão fica vermelho;
 - Nó passa a apontar para o pai (sobe na árvore para testar próximos casos);
 - Se o pai era vermelho, basta trocar para preto;
 - Senão inicia uma nova verificação.



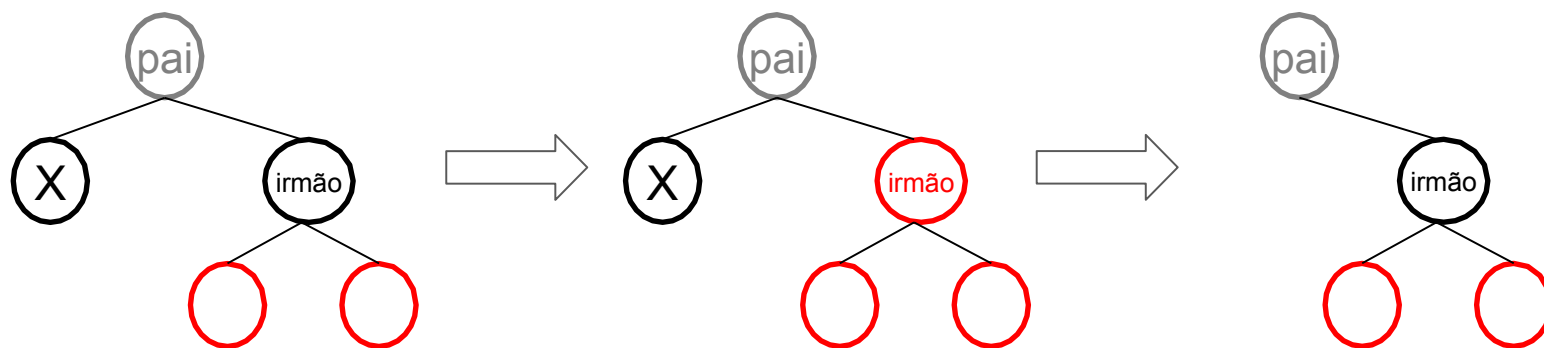
Remoção na árvore rubro-negra

- Casos 2: irmão é preto e filhos do irmão (sobrinhos) são vermelhos
 - Irmão fica vermelho;
 - Nó passa a apontar para o pai (sobe na árvore para testar próximos casos);
 - Se o pai era vermelho, basta trocar para preto;
 - Senão inicia uma nova verificação.



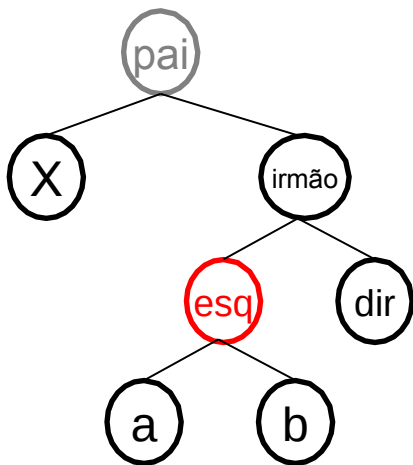
Remoção na árvore rubro-negra

- Casos 2: irmão é preto e filhos do irmão (sobrinhos) são vermelhos
 - Irmão fica vermelho;
 - Nó passa a apontar para o pai (sobe na árvore para testar próximos casos);
 - Se o pai era vermelho, basta trocar para preto;
 - Senão inicia uma nova verificação.



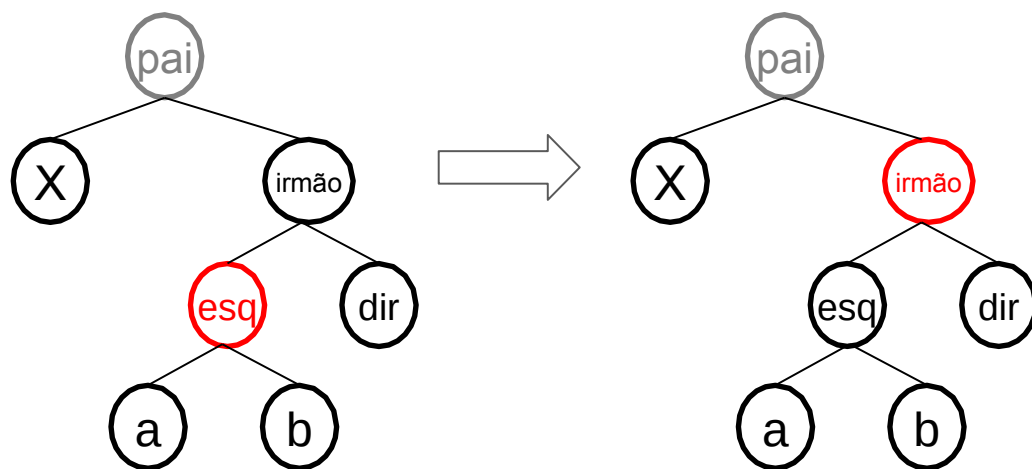
Remoção na árvore rubro-negra

- Casos 3: irmão da direita é preto, filho direito do irmão é preto e filho esquerdo é vermelho
 - Filho esquerdo fica preto;
 - Irmão fica vermelho;
 - Rotação à direita do irmão esquerdo.



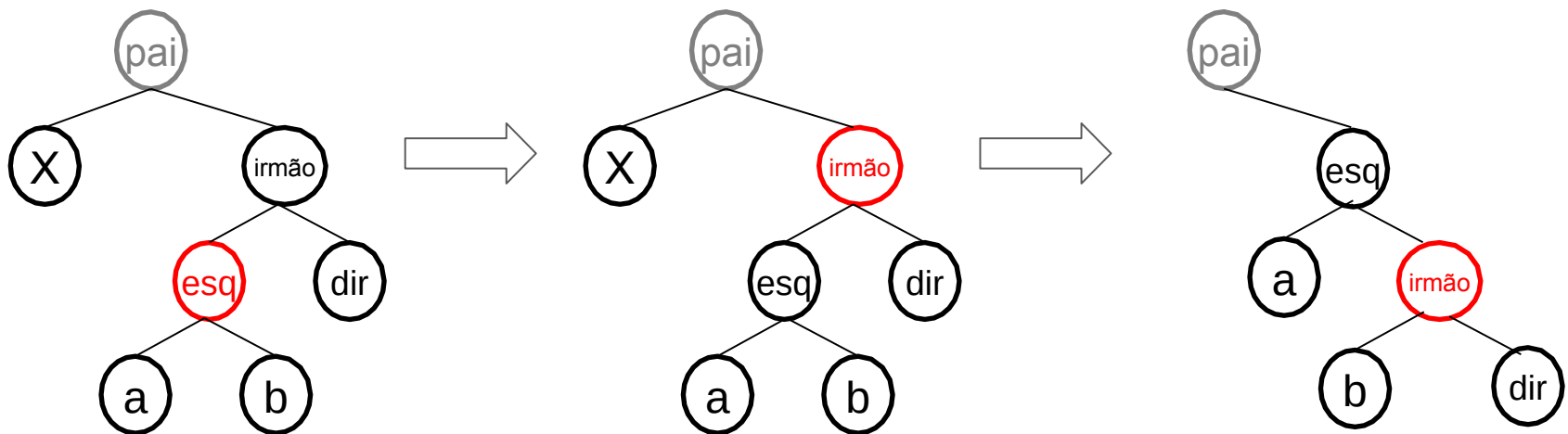
Remoção na árvore rubro-negra

- Casos 3: irmão da direita é preto, filho direito do irmão é preto e filho esquerdo é vermelho
 - Filho esquerdo fica preto;
 - Irmão fica vermelho;
 - Rotação à direita do irmão esquerdo.



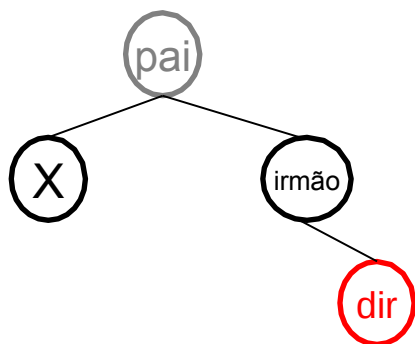
Remoção na árvore rubro-negra

- Casos 3: irmão da direita é preto, filho direito do irmão é preto e filho esquerdo é vermelho
 - Filho esquerdo fica preto;
 - Irmão fica vermelho;
 - Rotação à direita do irmão esquerdo.



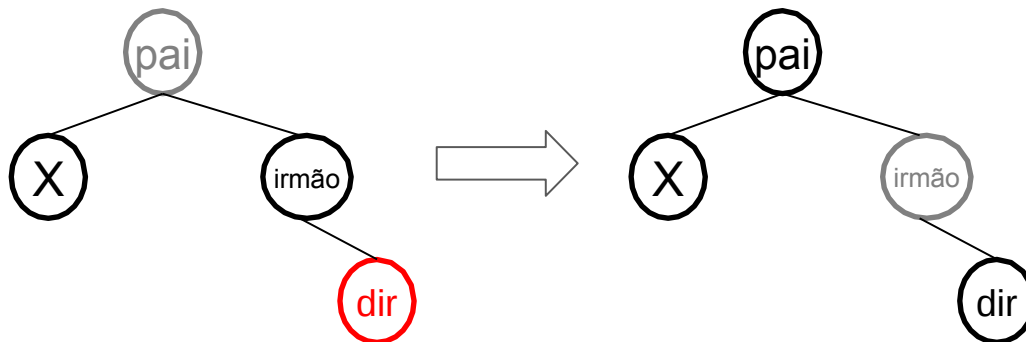
Remoção na árvore rubro-negra

- Casos 4: irmão da direita é preto e filho direito do irmão é vermelho
 - Irmão copia cor do pai
 - Pai e filho direito (dir) ficam pretos
 - Rotação pai à esquerda.
 - Se raiz for vermelha, trocar para preto.



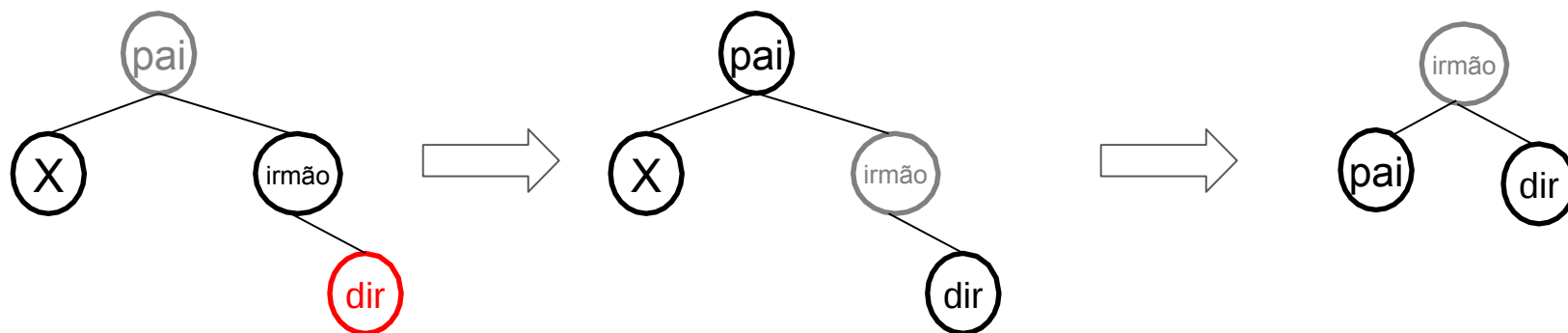
Remoção na árvore rubro-negra

- Casos 4: irmão da direita é preto e filho direito do irmão é vermelho
 - Irmão copia cor do pai
 - Pai e filho direito (dir) ficam pretos
 - Rotação pai à esquerda.
 - Se raiz for vermelha, trocar para preto.



Remoção na árvore rubro-negra

- Casos 4: irmão da direita é preto e filho direito do irmão é vermelho
 - Irmão copia cor do pai
 - Pai e filho direito (dir) ficam pretos
 - Rotação pai à esquerda.
 - Se raiz for vermelha, trocar para preto.

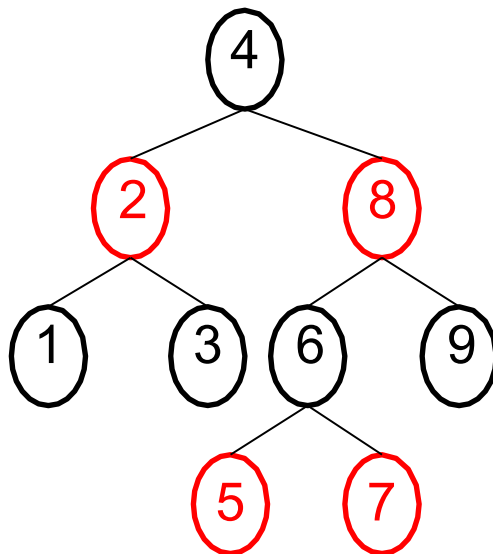


Aplicação de Árvores Rubro-Negras

- As árvores rubro-negra são de uso mais geral do que as árvores AVL
 - Bibliotecas em Java
 - `java.util.TreeMap`
 - `java.util.TreeSet`
 - Bibliotecas C++ STL
 - `set`
 - `map`
 - `multiset`
 - `multimap`
 - Linux kernel
 - Scheduler (vruntime)
 - Filesystem (ext3/ext4)
 - Virtual Memory Areas (VMAs)

Exercícios

1. Implemente uma árvore rubro-negra e adicione os nós de modo que a árvore apresente a respectiva topologia abaixo.



2. Implemente a operação de remoção de nós em uma árvore Rubro-Negra e valide removendo o nó 6 do exercício anterior.

Árvores Trie

Auto Completar e outros problemas

- Objetivo
 - Localizar um dado que corresponde à chave informada
 - A chave e os dados mantidos no conjunto faz parte de um léxico
 - A busca deve considerar uma aproximação de correspondência
- Contexto
 - Conjunto de nomes de pessoas
 - Há nomes com grafias semelhantes
 - Manuel/Manoel
 - Luis/Luiz
 - Podem ocorrer erros na entrada de dados (chave)

Árvores Trie

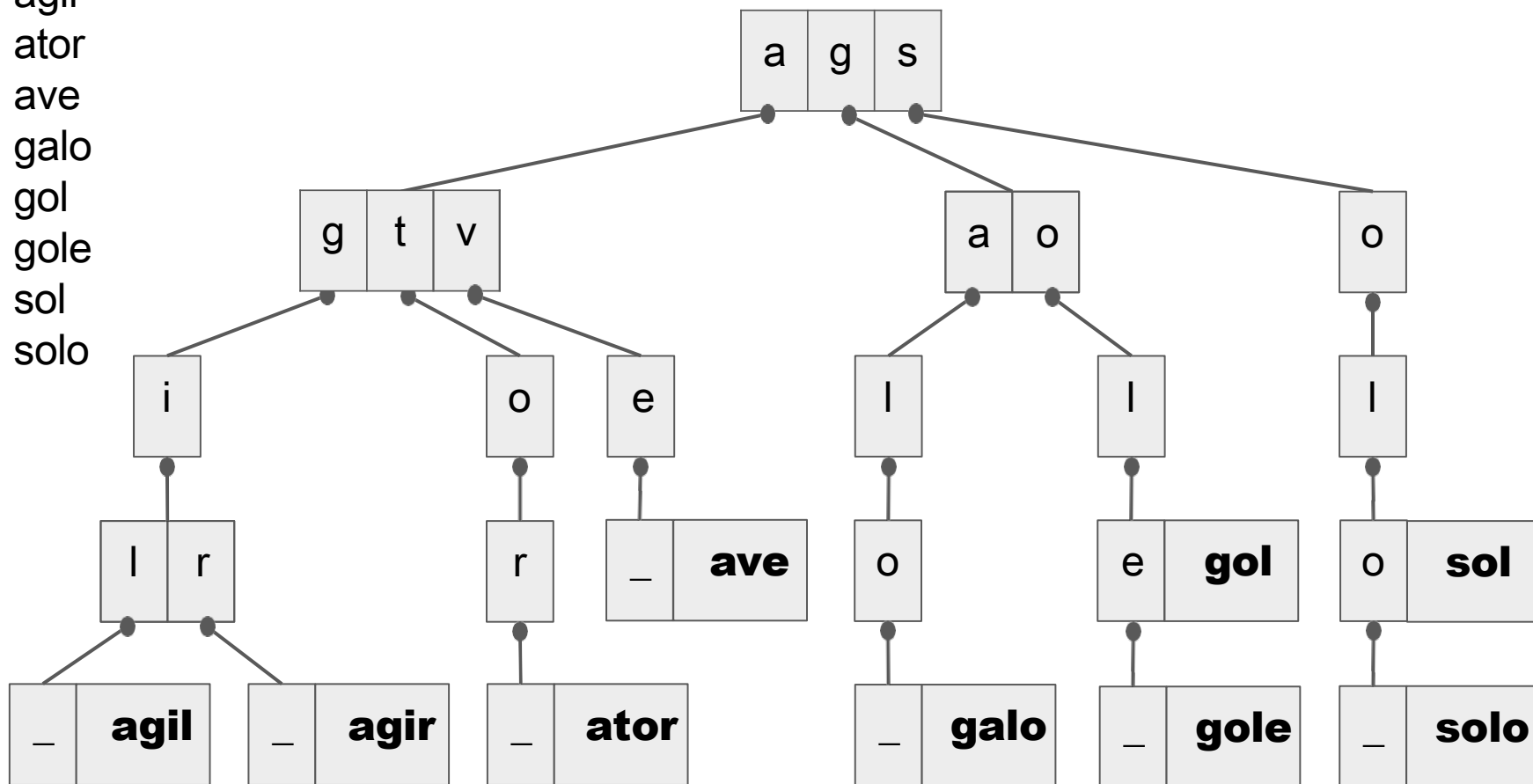
- Introduzida por Edward Fredkin em 1960
 - O nome Trie tem origem em Retrieval (recuperar)
 - No caso do propósito da árvore, recuperação dados
- Estrutura da árvore
 - Cada nó contém informações sobre um ou mais símbolos do léxico
 - O léxico (alfabeto) é finito e pode abranger os símbolos como:
 - $\{ 0, 1, \dots \} \rightarrow$ números
 - $\{ A, B, C, D, \dots \} \rightarrow$ caracteres
 - $\{ \} \rightarrow$ nulo

Árvores Trie

- São adequadas para tratamento lexicográfico
 - Manuseio de dicionário
 - Pesquisas em texto de grande dimensão
 - Construção de índices de documentos baseado em léxico
 - Expressão regular como padrão de pesquisa
- Estratégia
 - Caminho da raiz para qualquer nó representa um prefixo de uma string
 - Todos os descendentes diretos do mesmo pai são agrupados
 - Em um nó folha, o último caractere da palavra sendo procurada deverá apontar para o próprio nó

Árvores Trie

- agir
- agil
- ator
- ave
- galo
- gol
- gole
- sol
- solo



Árvores Trie

- Estratégia (cont.)
 - Cada nível da árvore explorado corresponde ao avanço de um símbolo na chave de pesquisa
 - Cada nó pode conter informação sobre um ou mais símbolos do alfabeto utilizado
 - Uma sequência de filhos em um dado nó pode formar qualquer palavra com base neste léxico (alfabeto)
 - Não existe limite para:
 - Tamanho de uma sequência de filhos de um nó
 - Tamanho de uma chave

Aplicações de Árvores Trie

- Corretor ortográfico
 - As palavras são comparadas com um dicionário armazenado em arquivo
 - Se não são encontradas, indica-se as opções para correção
- Procedimentos
 - Armazenar dicionário em uma trie
 - Percorrer a trie símbolo por símbolo para localizar a palavra testada
 - Caso seja detectado um erro na chave, o algoritmo sugere correções
 - Indicando as palavras do dicionário formadas a partir de um nó

Aplicações de Árvores Trie

- Sugestões possíveis para correção
 - Substituição
 - Avança um caracter na chave e avança um nível na árvore
 - Deleção
 - Avança um nível na árvore
 - Inserção
 - Avança um caracter na chave
 - Transposição
 - Avança um nível na árvore e testa a posição atual da chave
 - Se coincidir, avança um caracter na chave
 - Retrocede um nível na árvore para confirmar a inversão

Aplicações de Árvores Trie

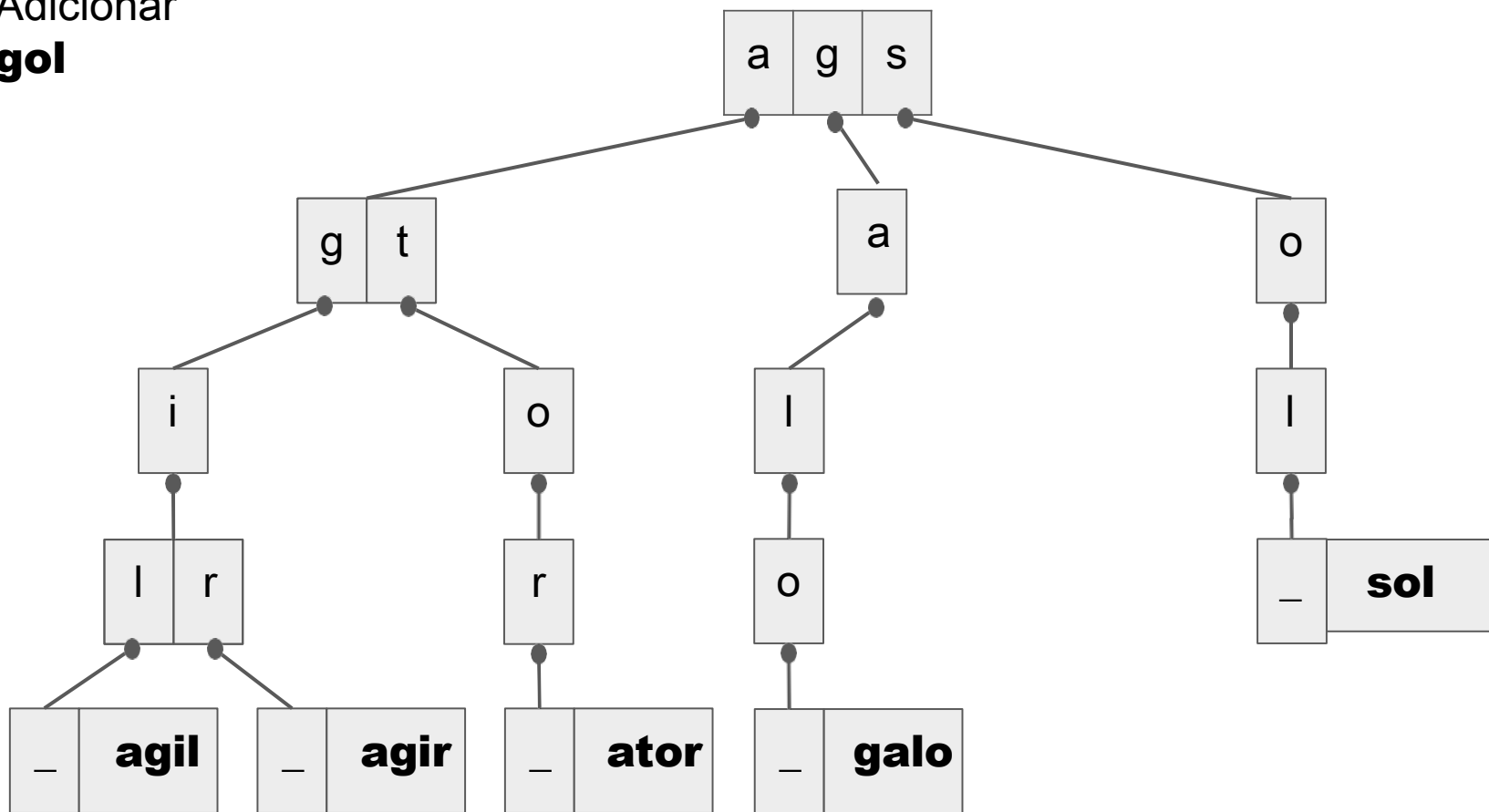
- Auto-preenchimento
 - Armazena palavras mais utilizadas em uma trie
 - A medida que um novo símbolo é adicionado na chave, serão exibidas as opções possíveis de palavras mais utilizadas
- Procedimentos
 - Armazenar dicionário em uma trie
 - Percorrer a trie símbolo por símbolo para localizar a palavra testada
 - Indica as palavras do dicionário formadas a partir de um nó, considerando a frequência de uso delas
 - Requer uma estrutura adicional para manter esta frequência de uso

Operações sobre Árvores Trie

- Adição
 - Realizada uma busca pela palavra a ser inserida
 - Se a palavra já existir na trie, nada é feito
 - Caso contrário, é recuperado o nó até o ponto onde acontece a maior subcadeia da palavra a ser inserida
 - O restante dos seus caracteres (prefixo) serão adicionados na trie a partir daquele nó

Árvores Trie

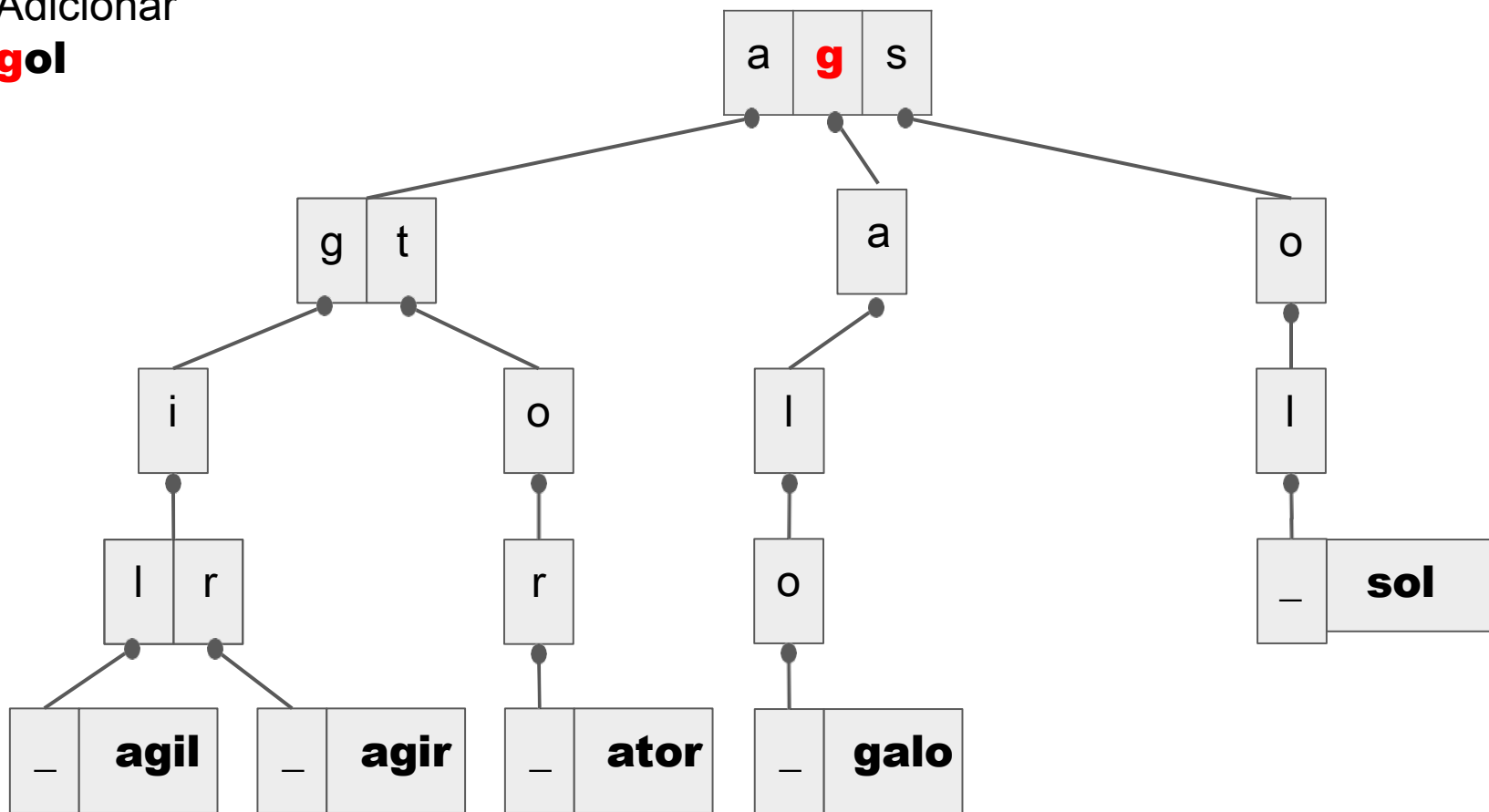
Adicionar
gol



Árvores Trie

Adicionar

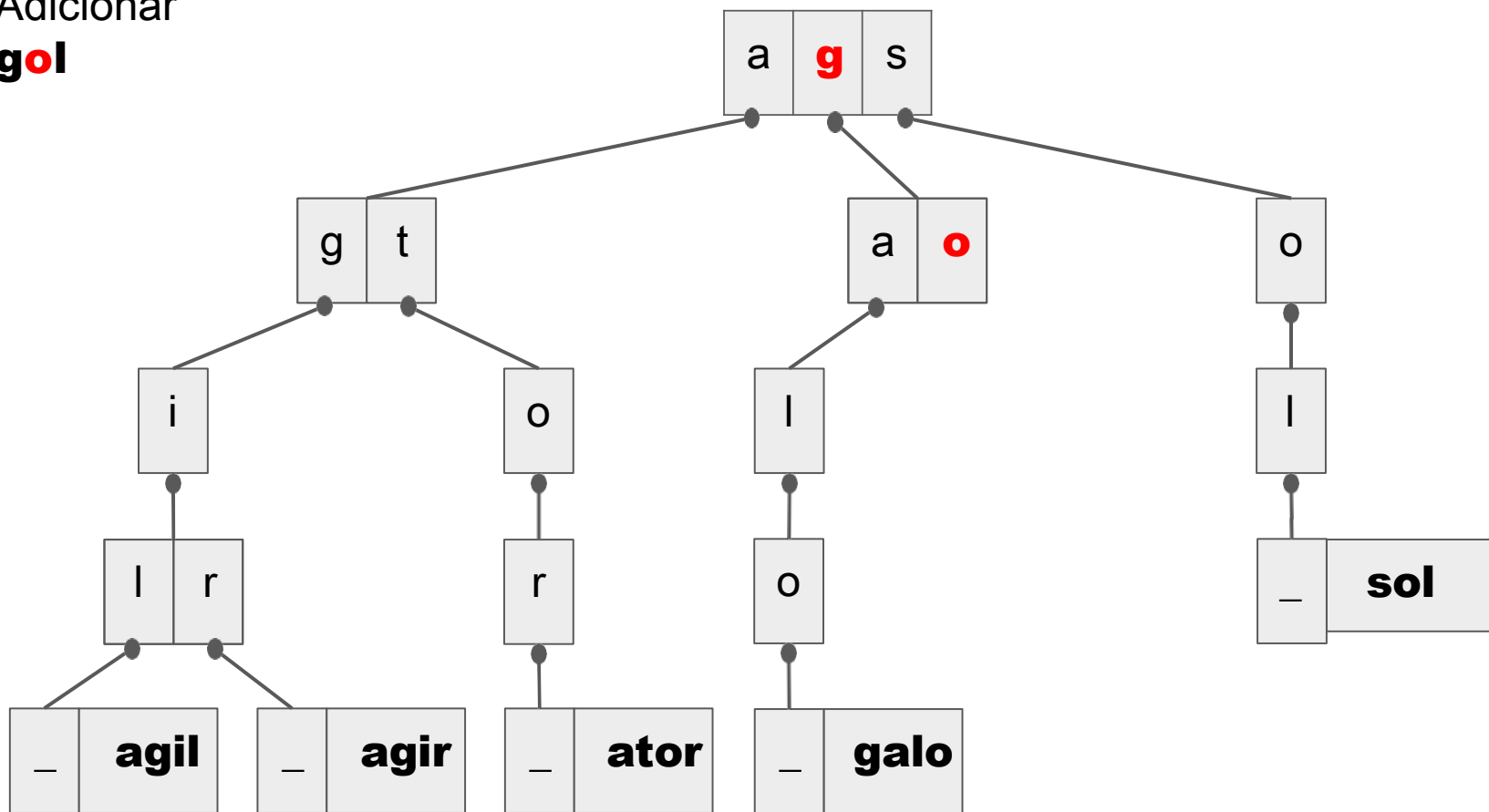
gol



Árvores Trie

Adicionar

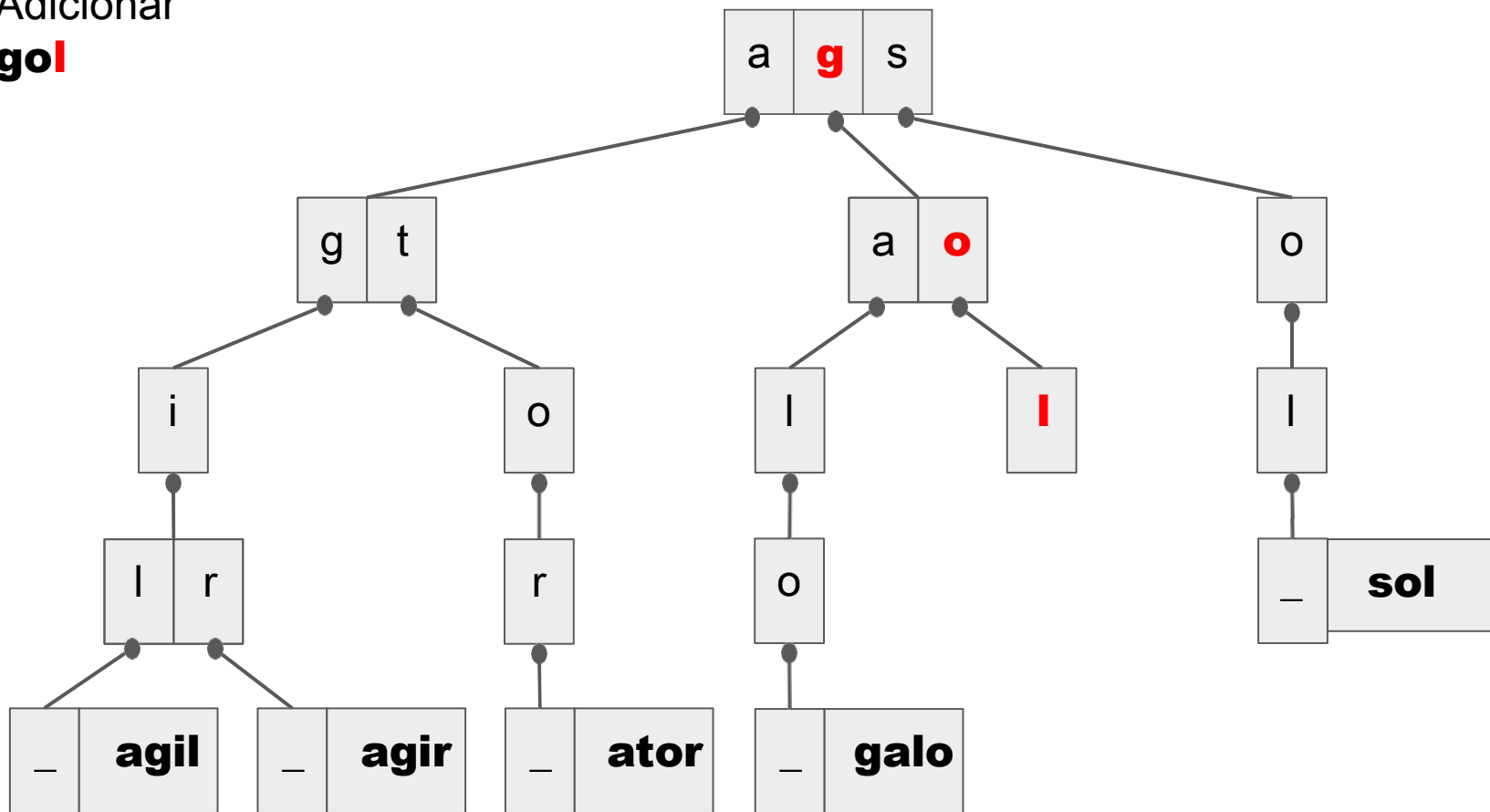
gol



Árvores Trie

Adicionar

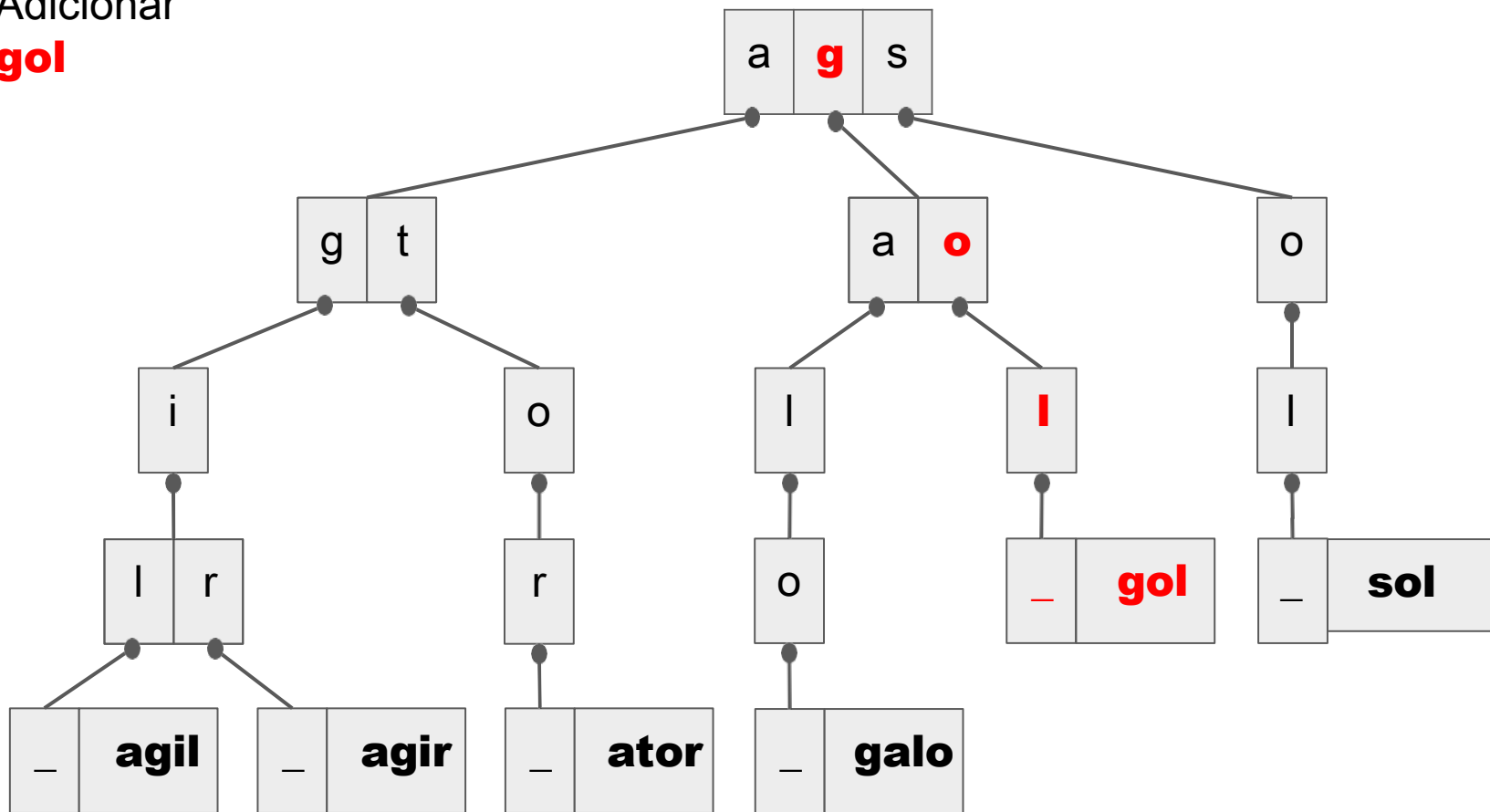
gol



Árvores Trie

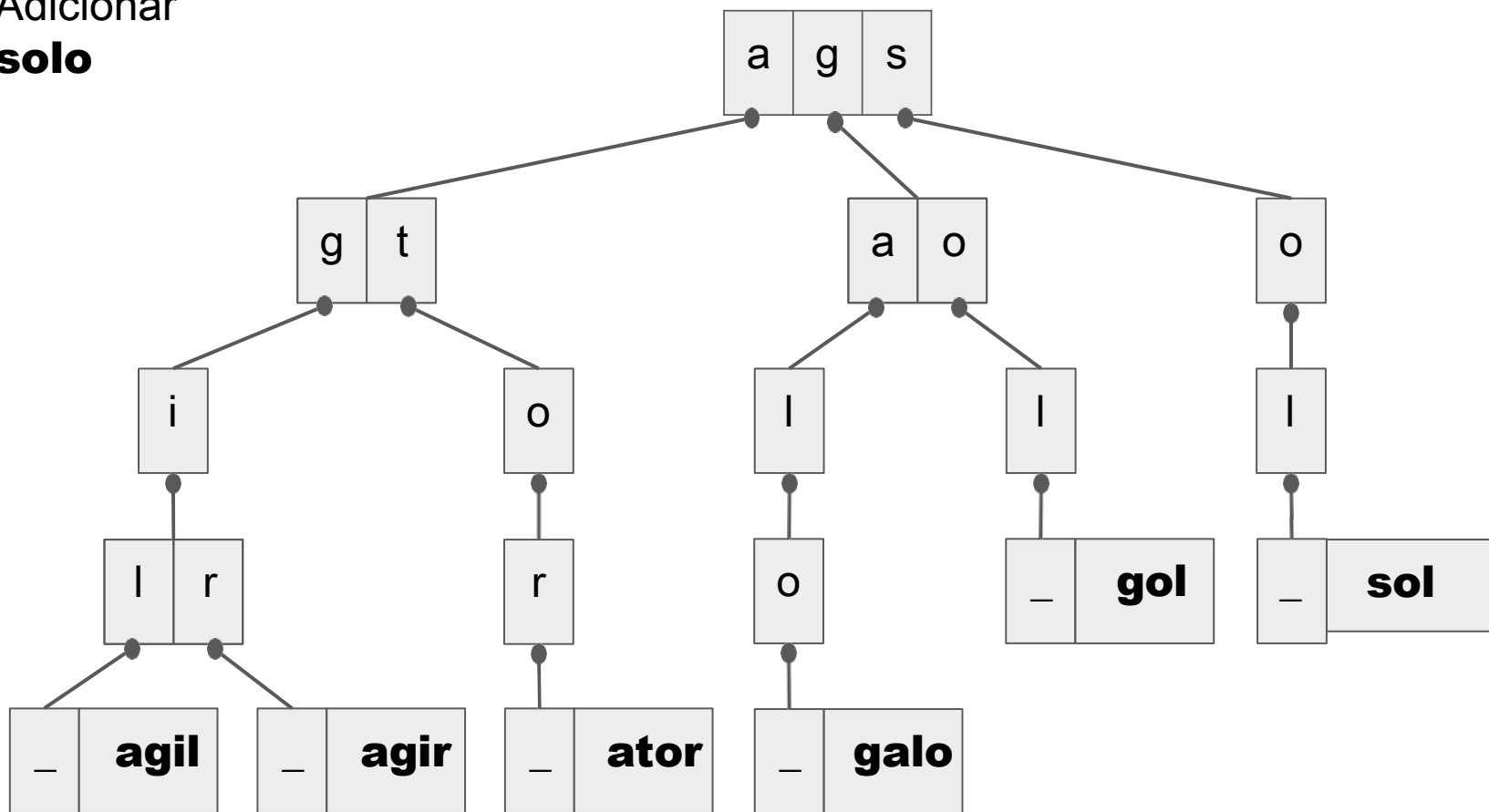
Adicionar

gol



Árvores Trie

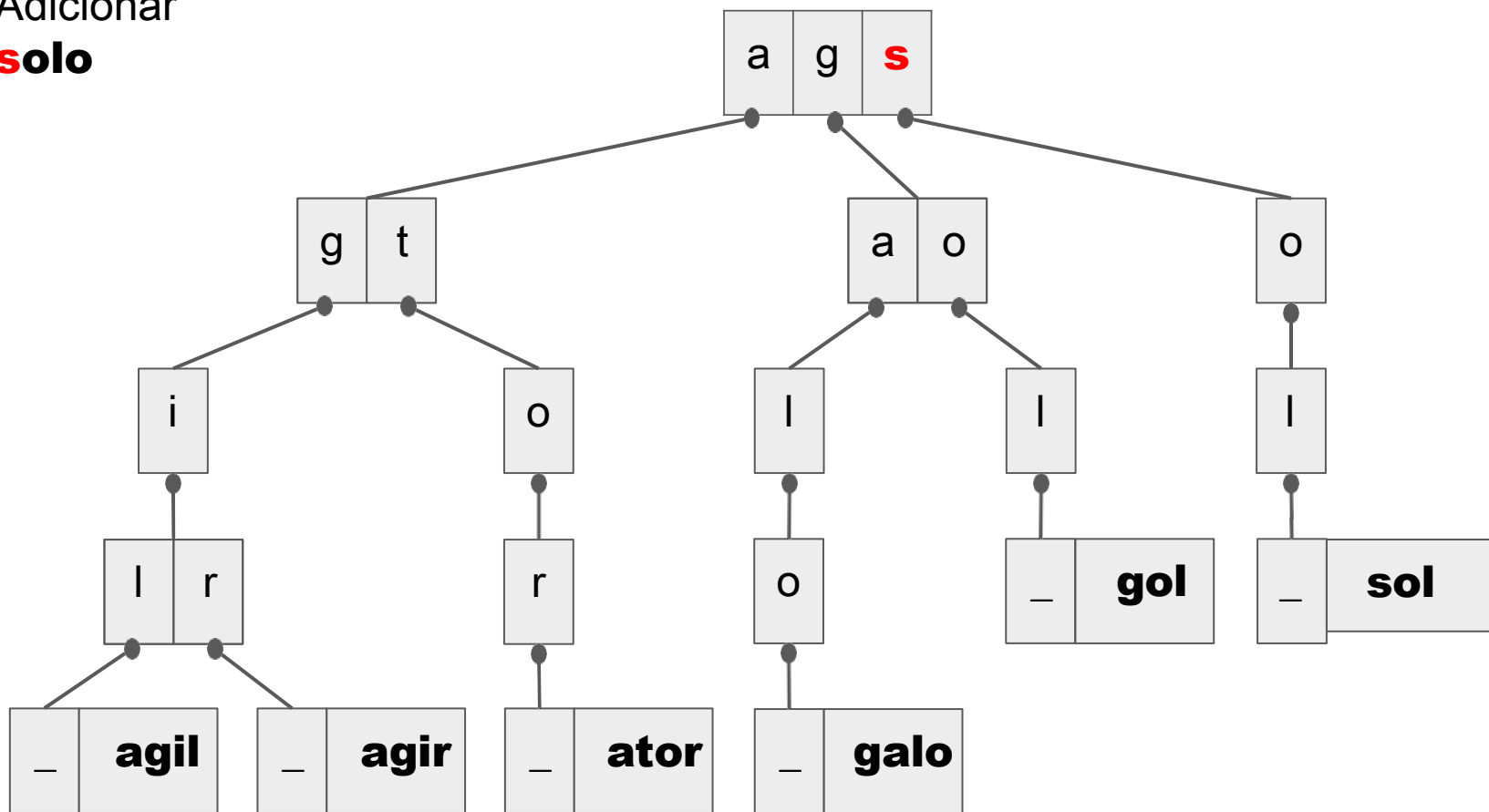
Adicionar
solo



Árvores Trie

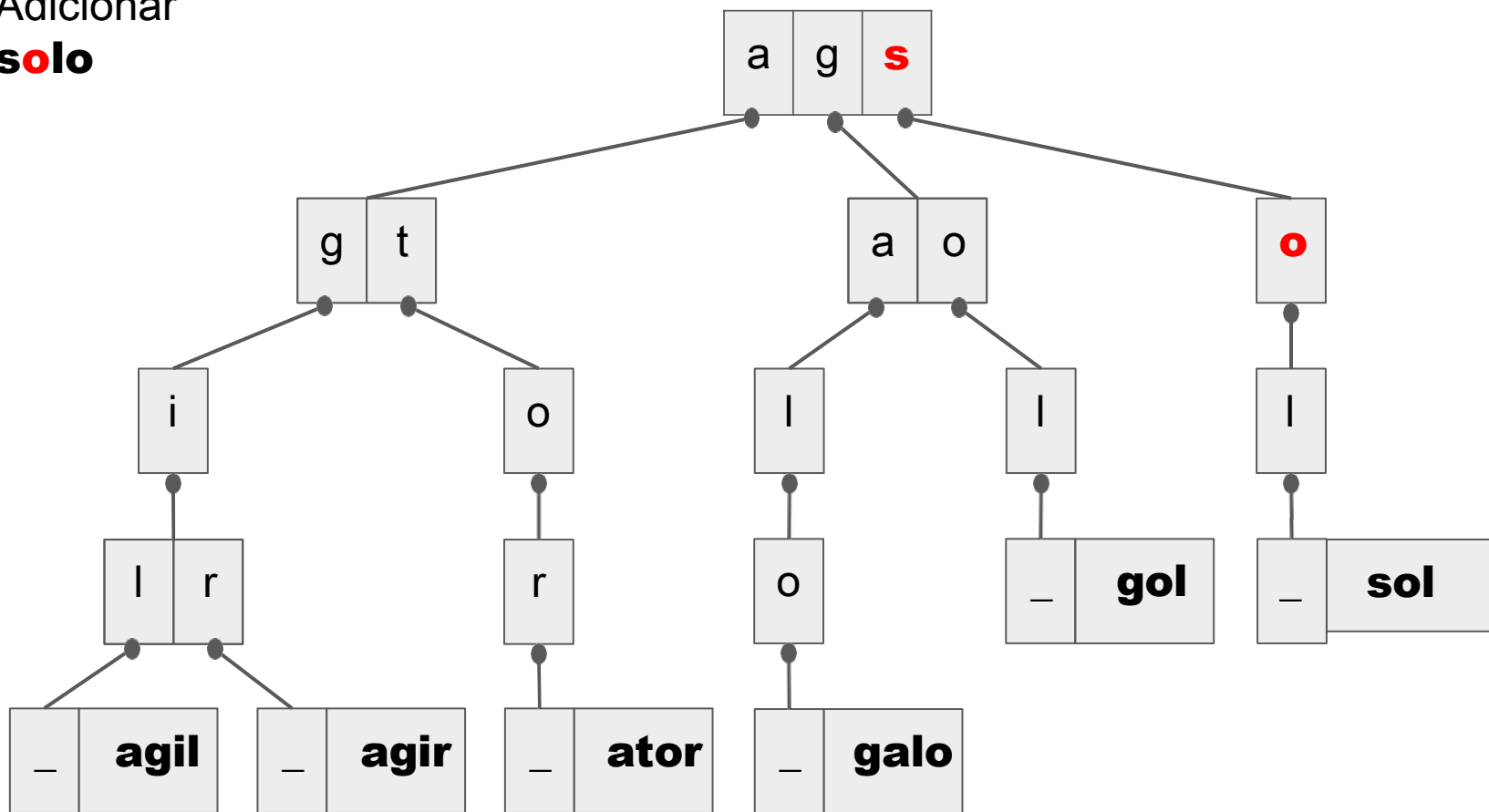
Adicionar

solo



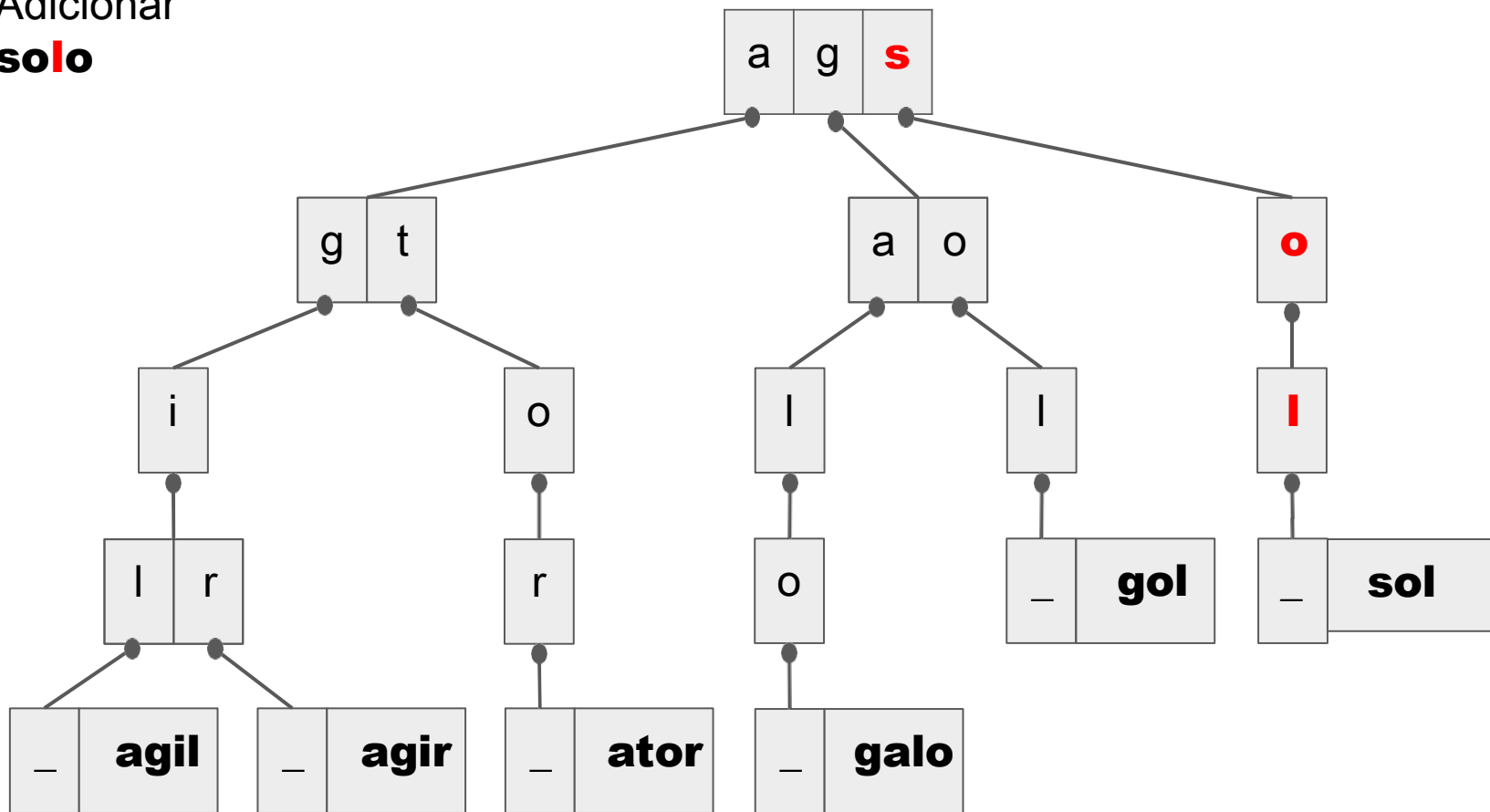
Árvores Trie

Adicionar
solo



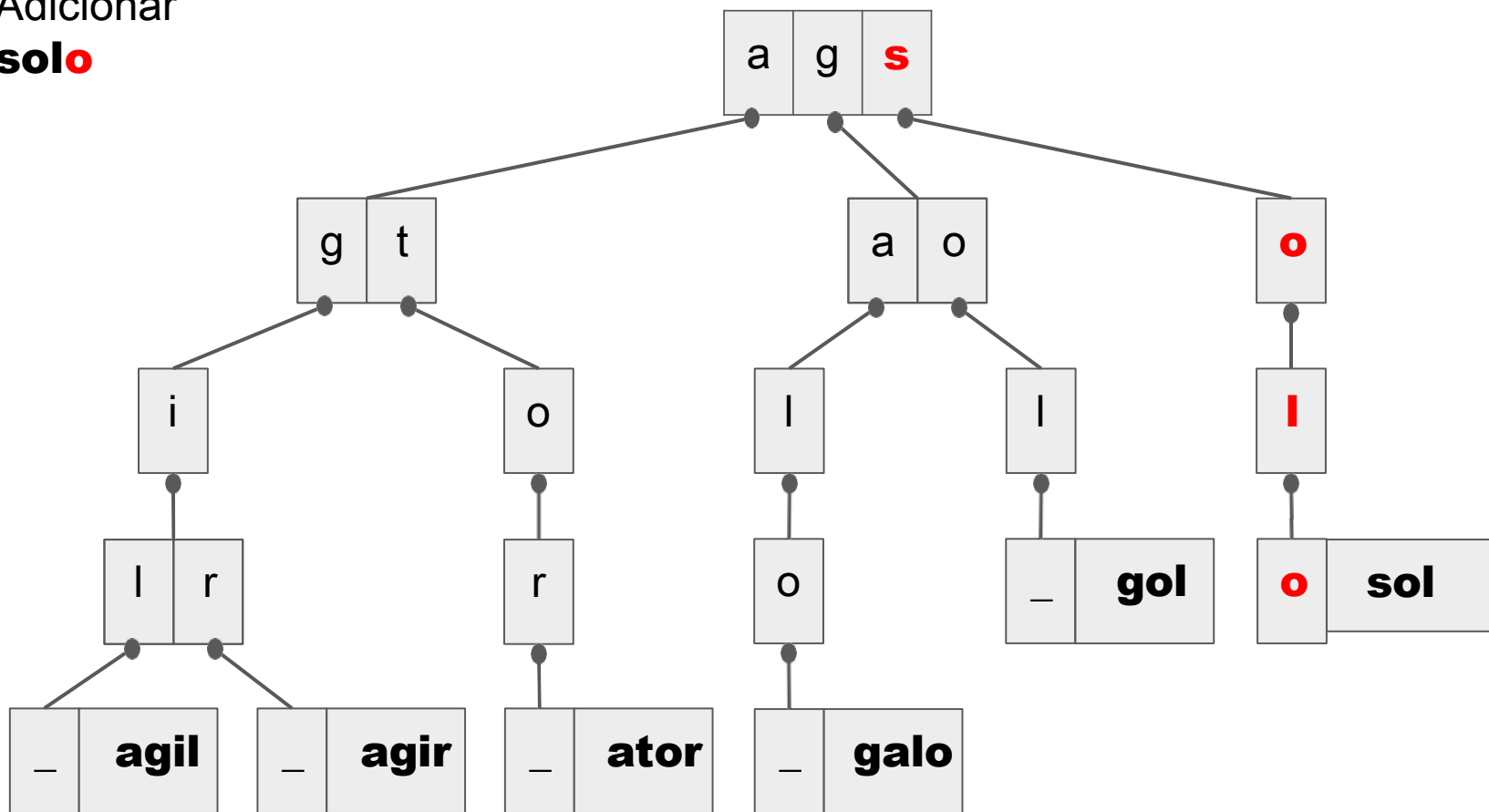
Árvores Trie

Adicionar
solo



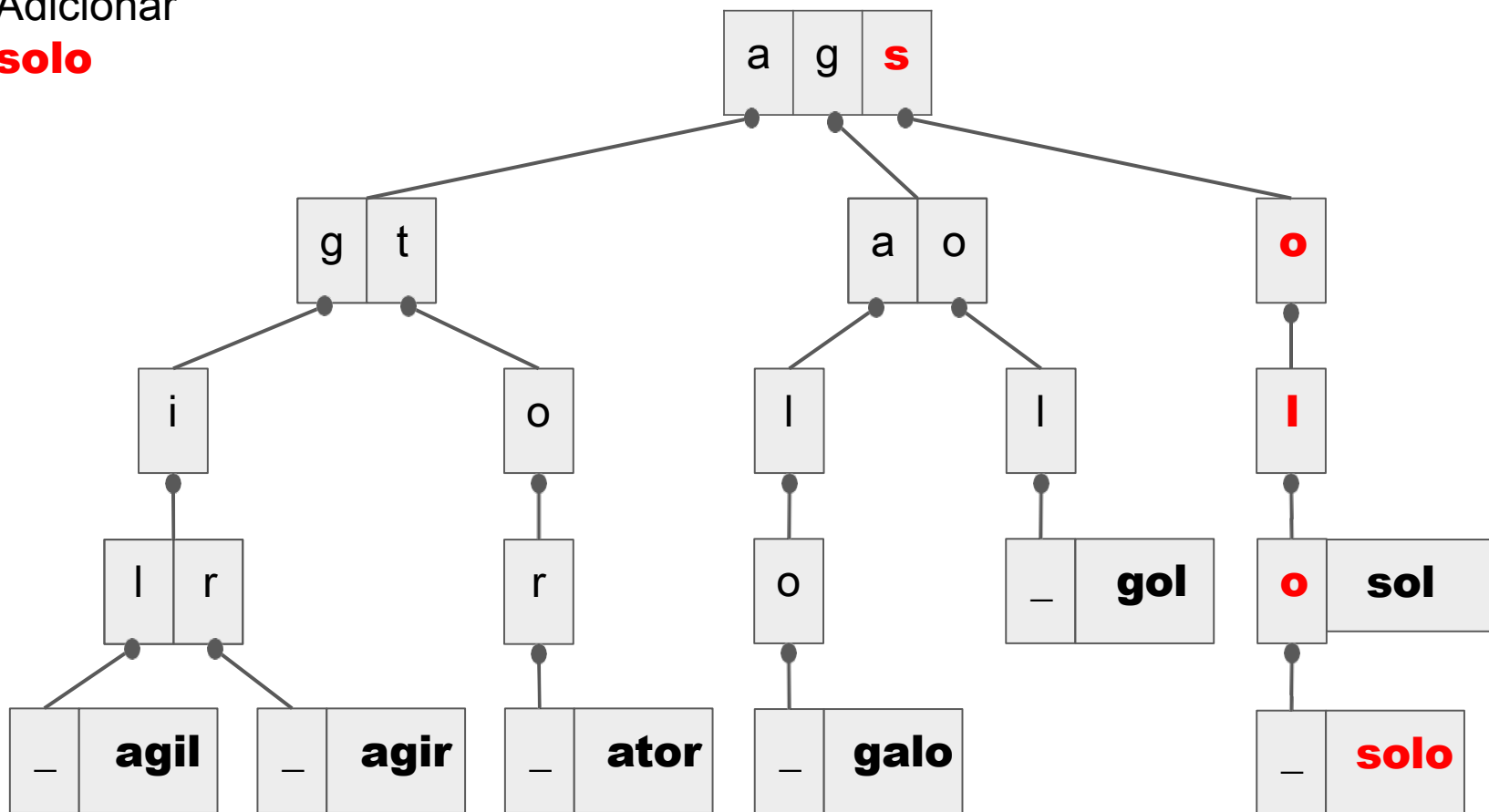
Árvores Trie

Adicionar
sol



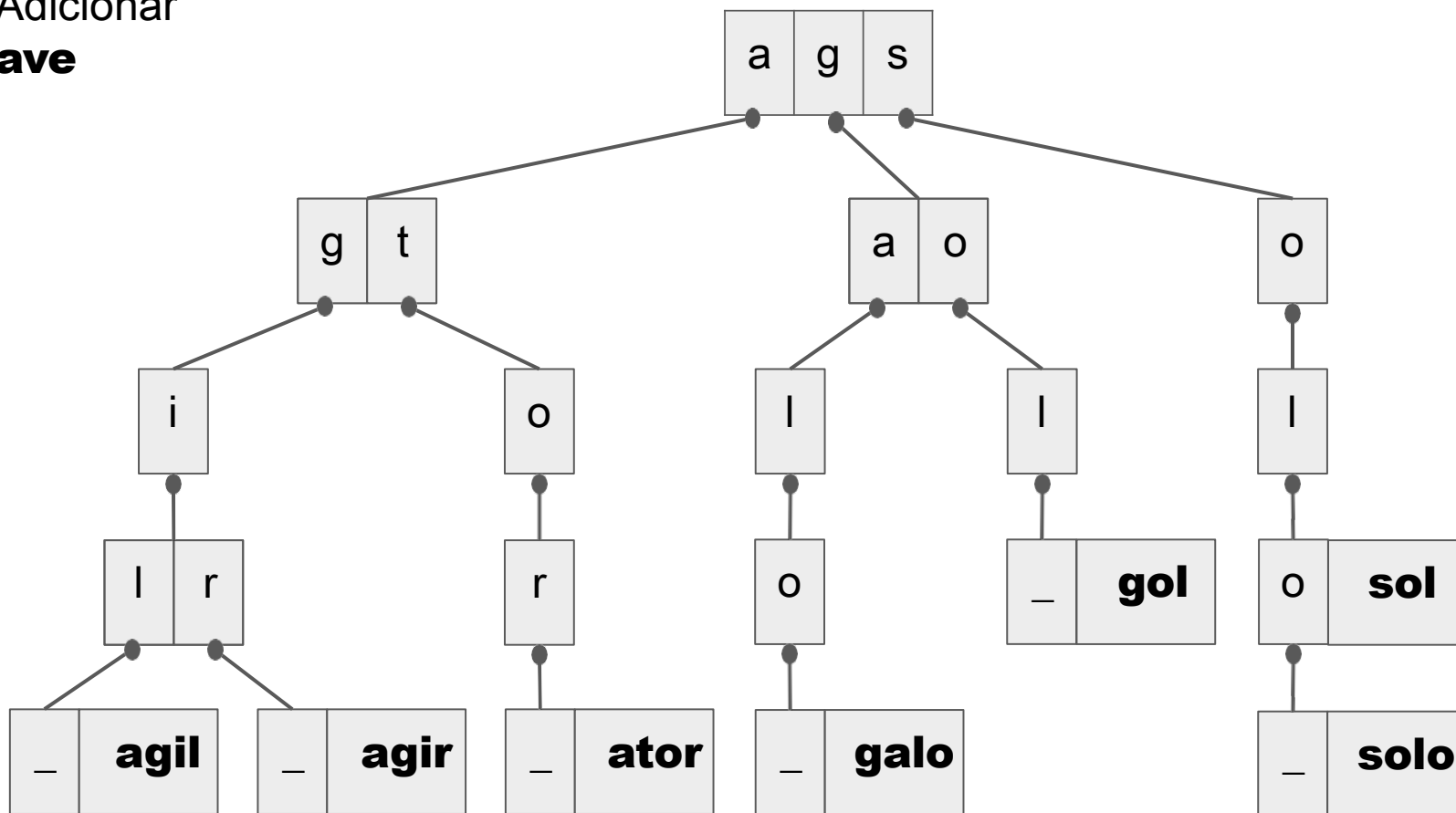
Árvores Trie

Adicionar
solo



Árvores Trie

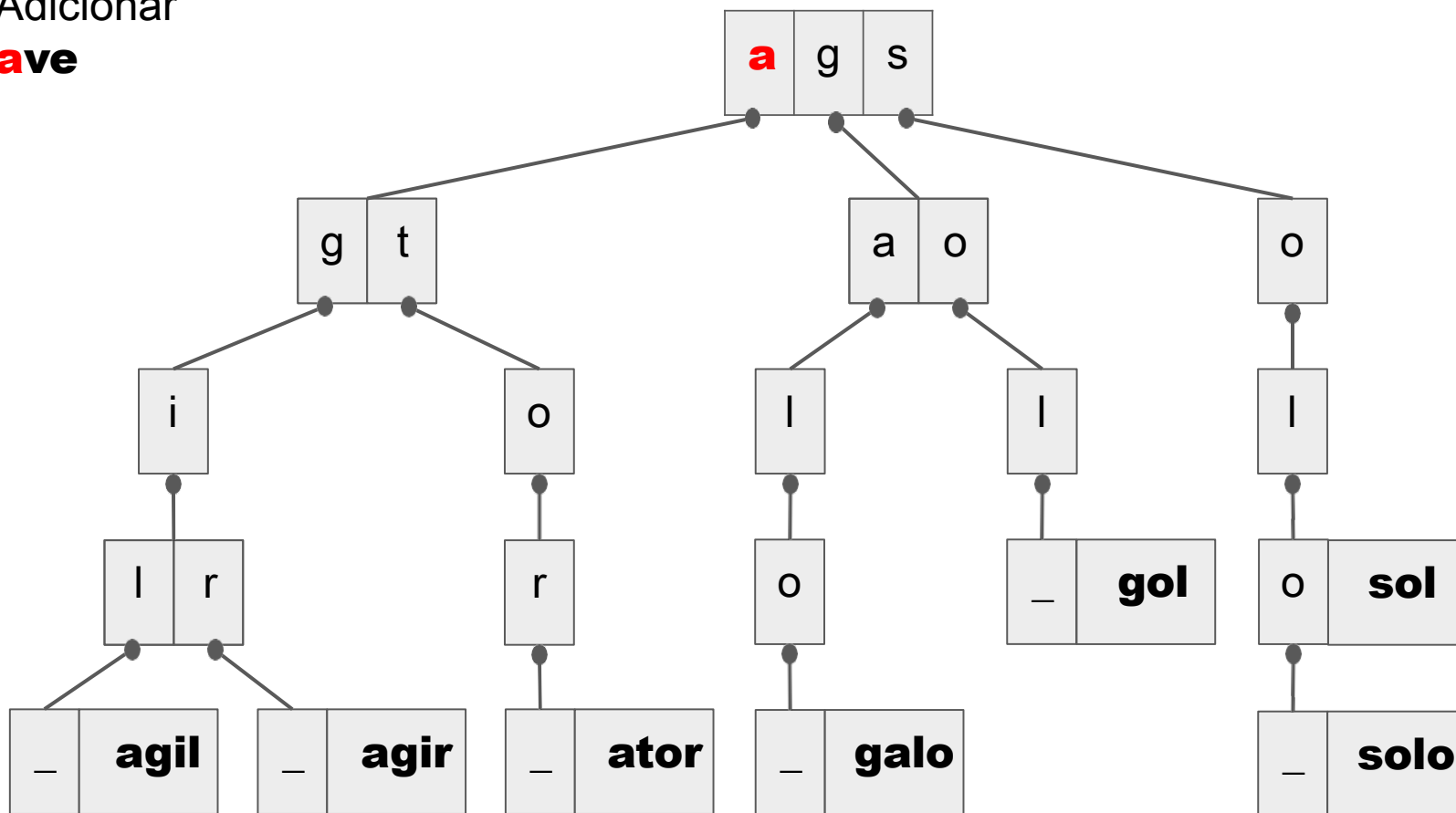
Adicionar
ave



Árvores Trie

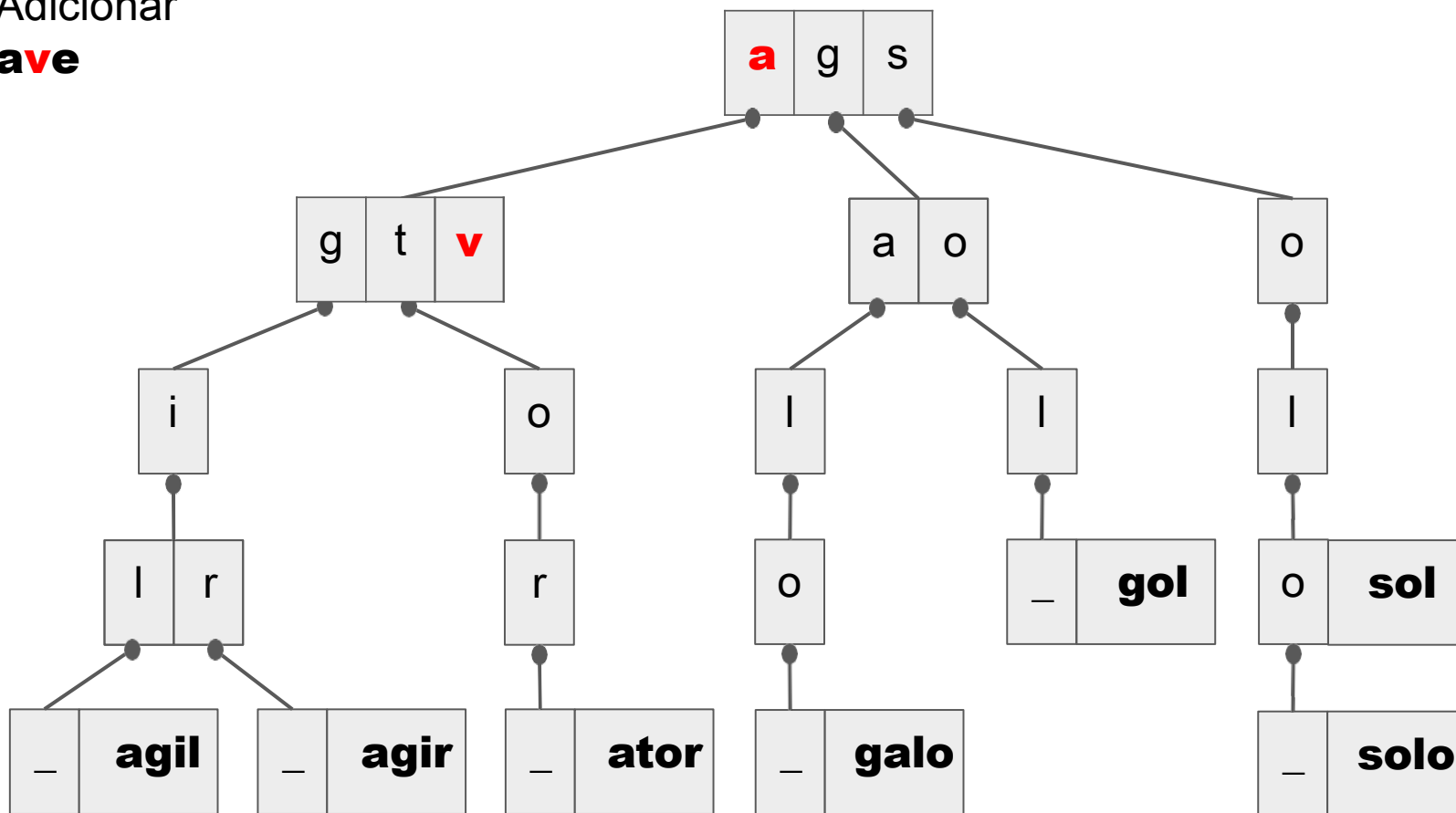
Adicionar

ave



Árvores Trie

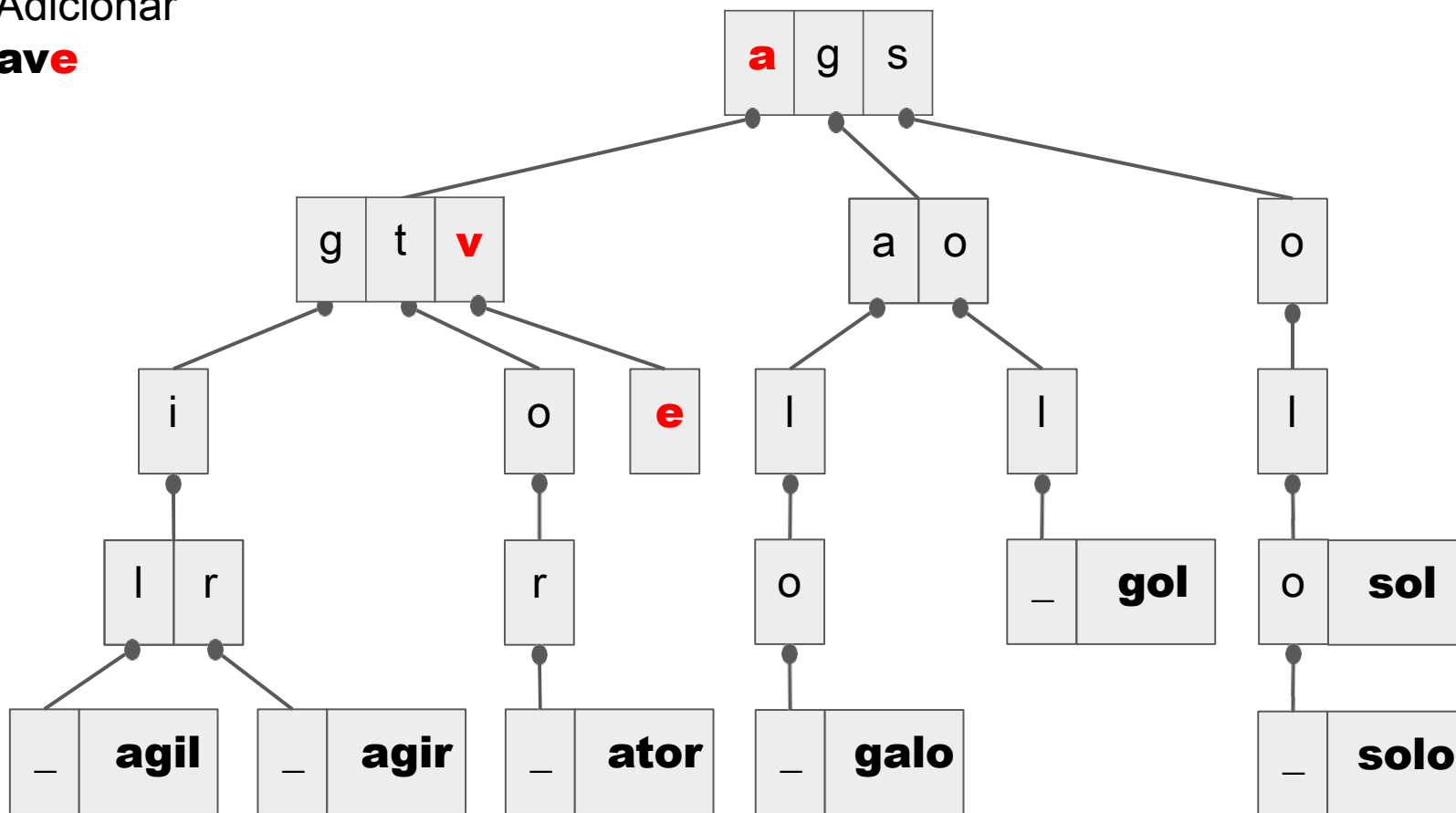
Adicionar
ave



Árvores Trie

Adicionar

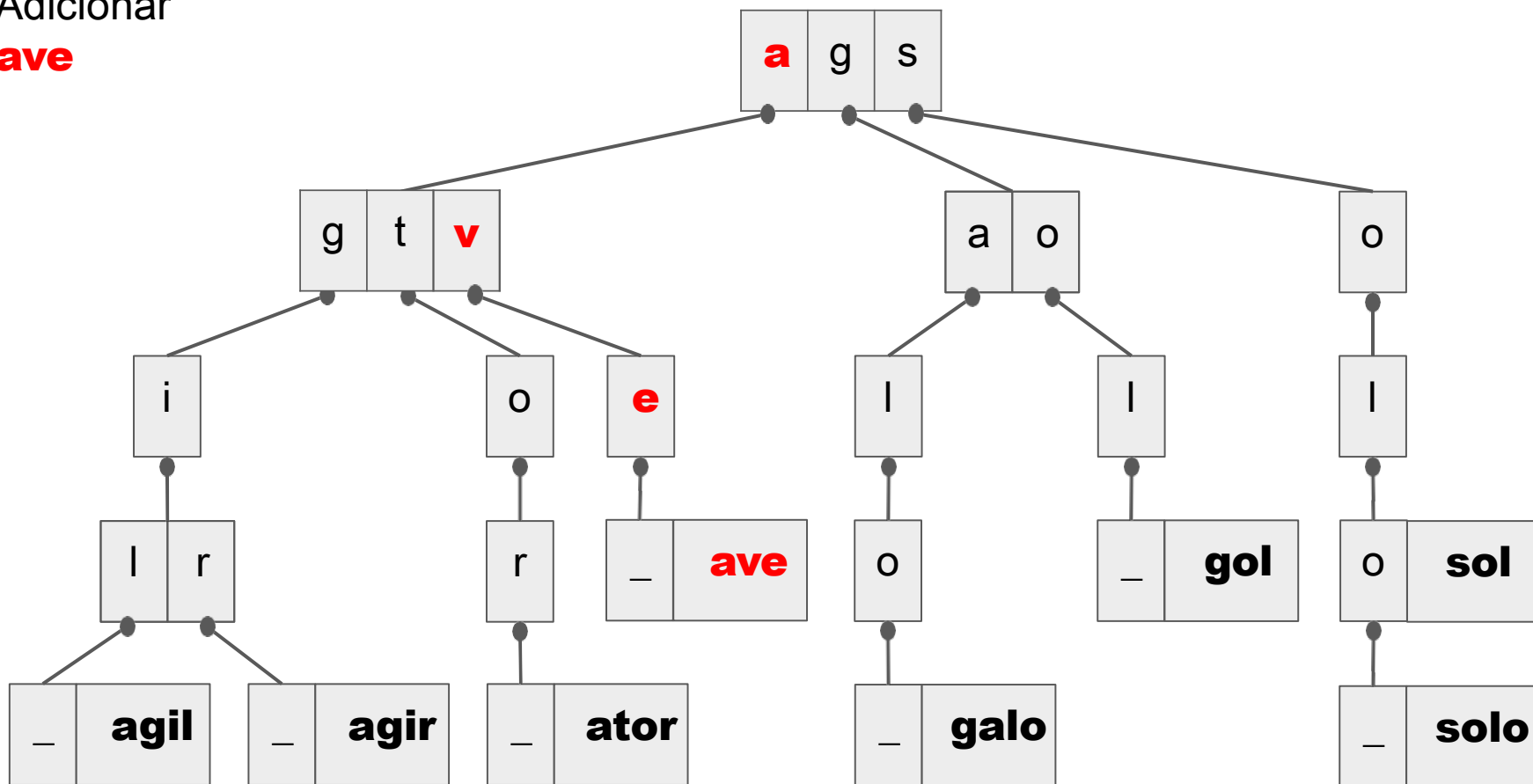
ave



Árvores Trie

Adicionar

ave

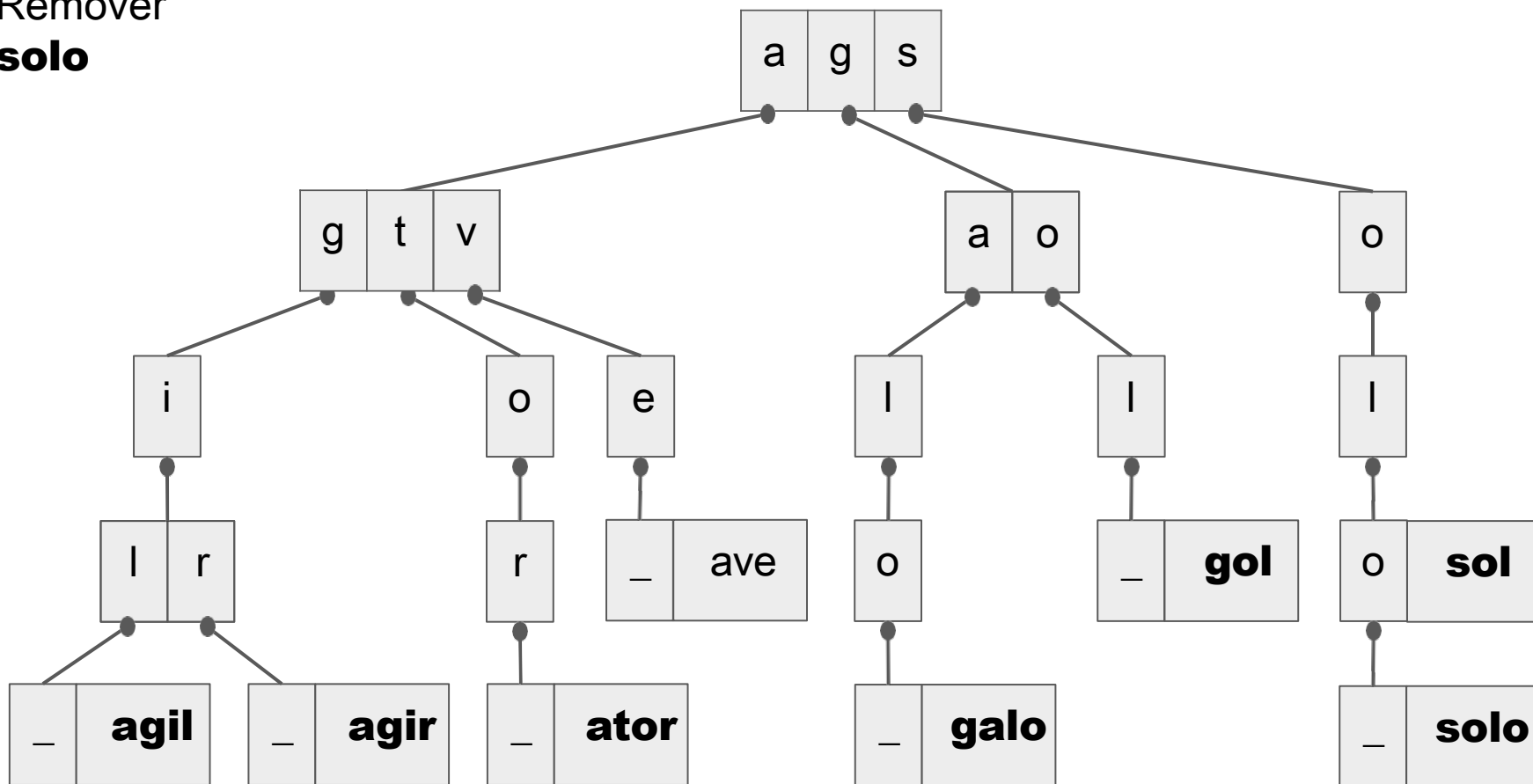


Operações sobre Árvores Trie

- Remoção
 - Realizada uma busca pela palavra a ser excluída
 - Ao encontrar o nó que representa o final da palavra a ser removida
 - Remove-se os nós que possuem apenas um filho no caminho ascendente
 - A remoção é concluída quando se encontra um nó com mais de um filho

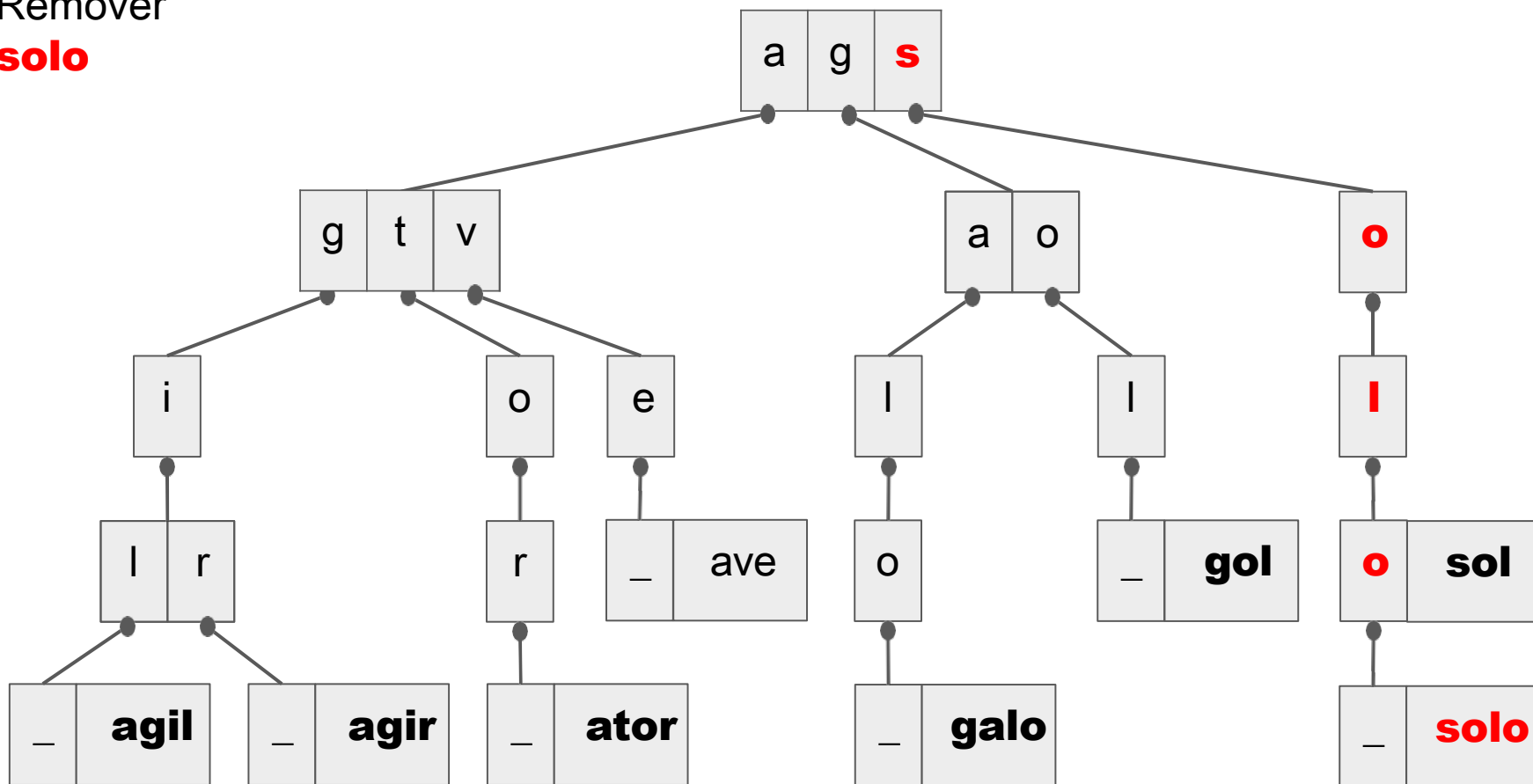
Árvores Trie

Remover
solo



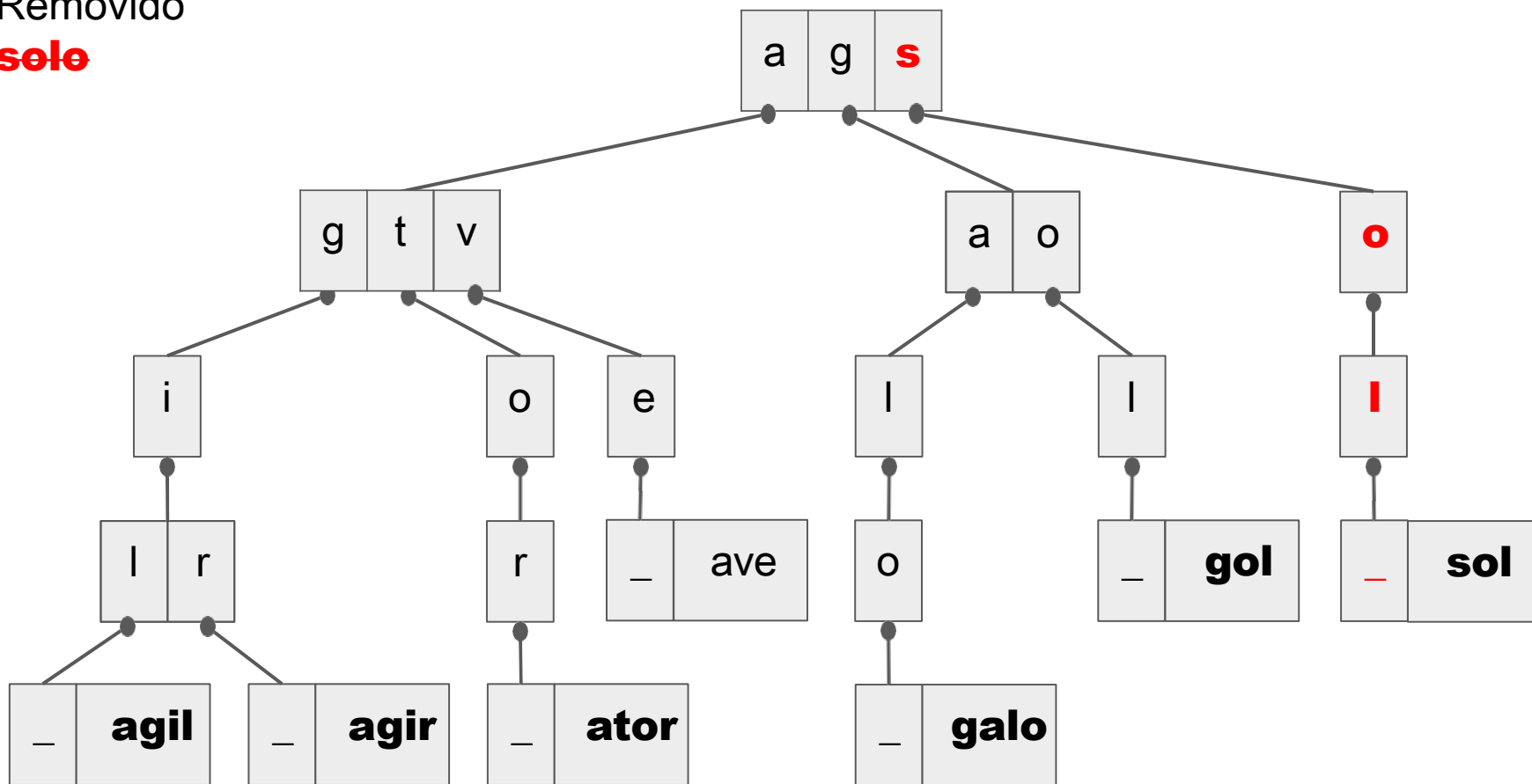
Árvores Trie

Remover
solo



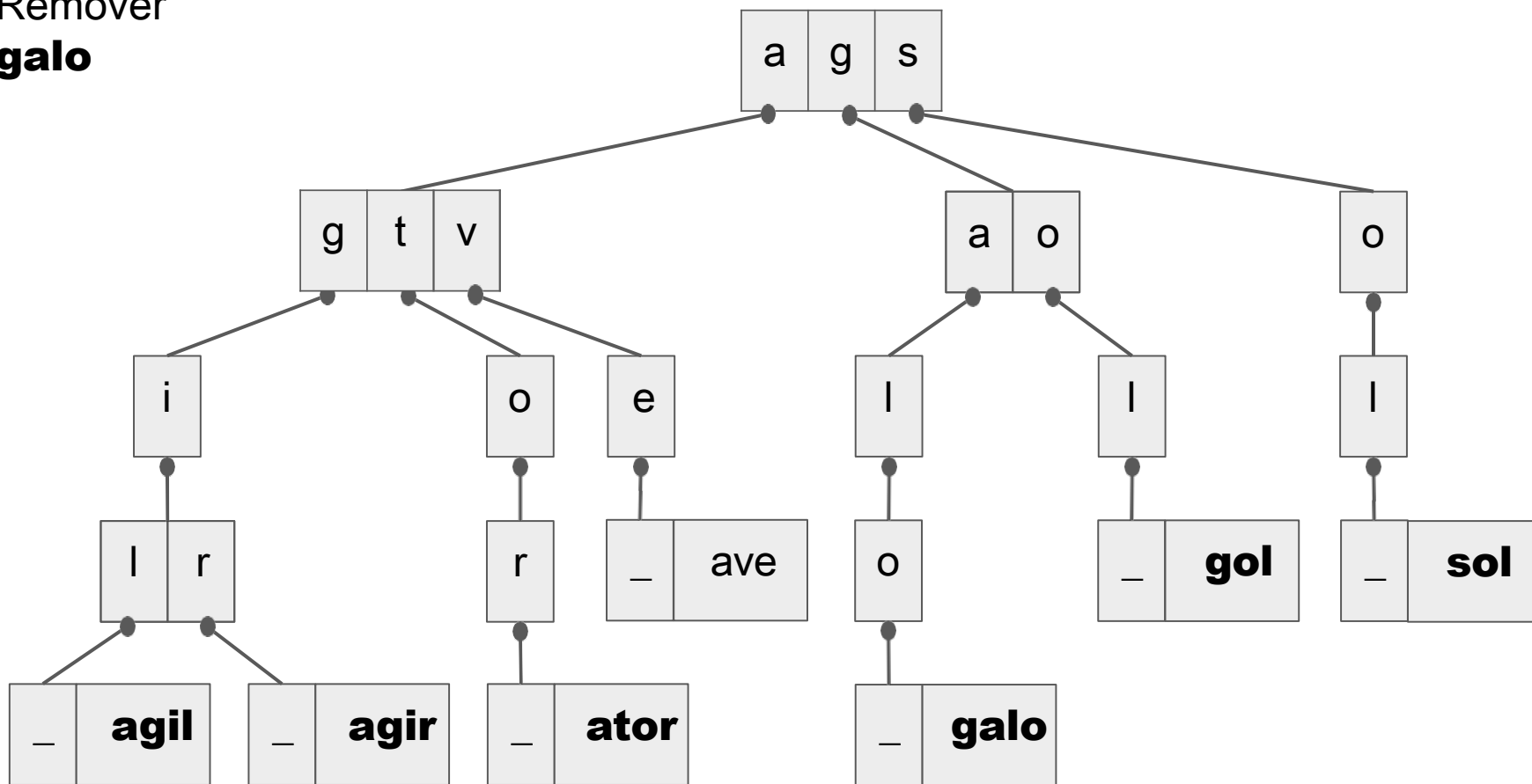
Árvores Trie

Removido
sole



Árvores Trie

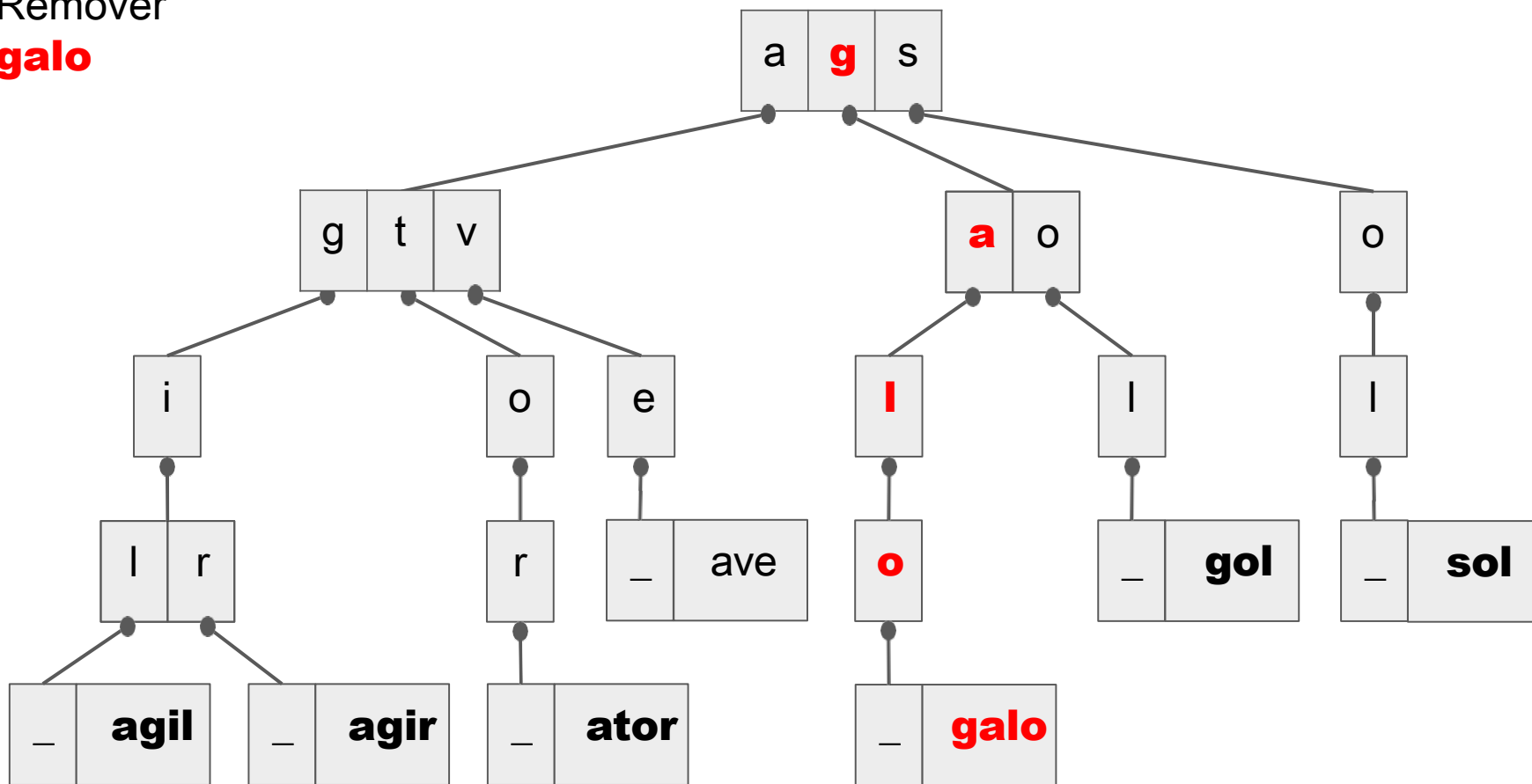
Remover
galo



Árvores Trie

Remover

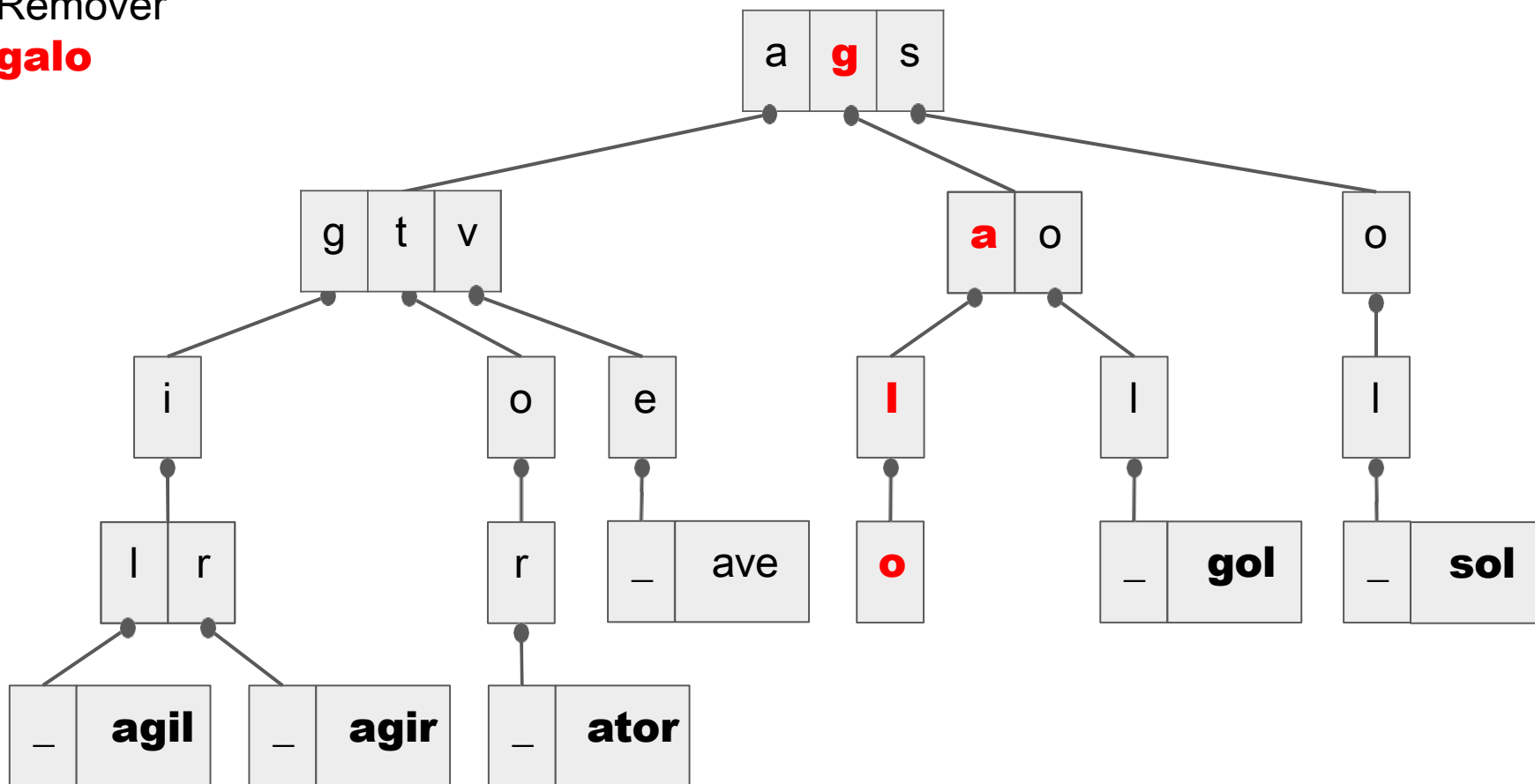
galo



Árvores Trie

Remover

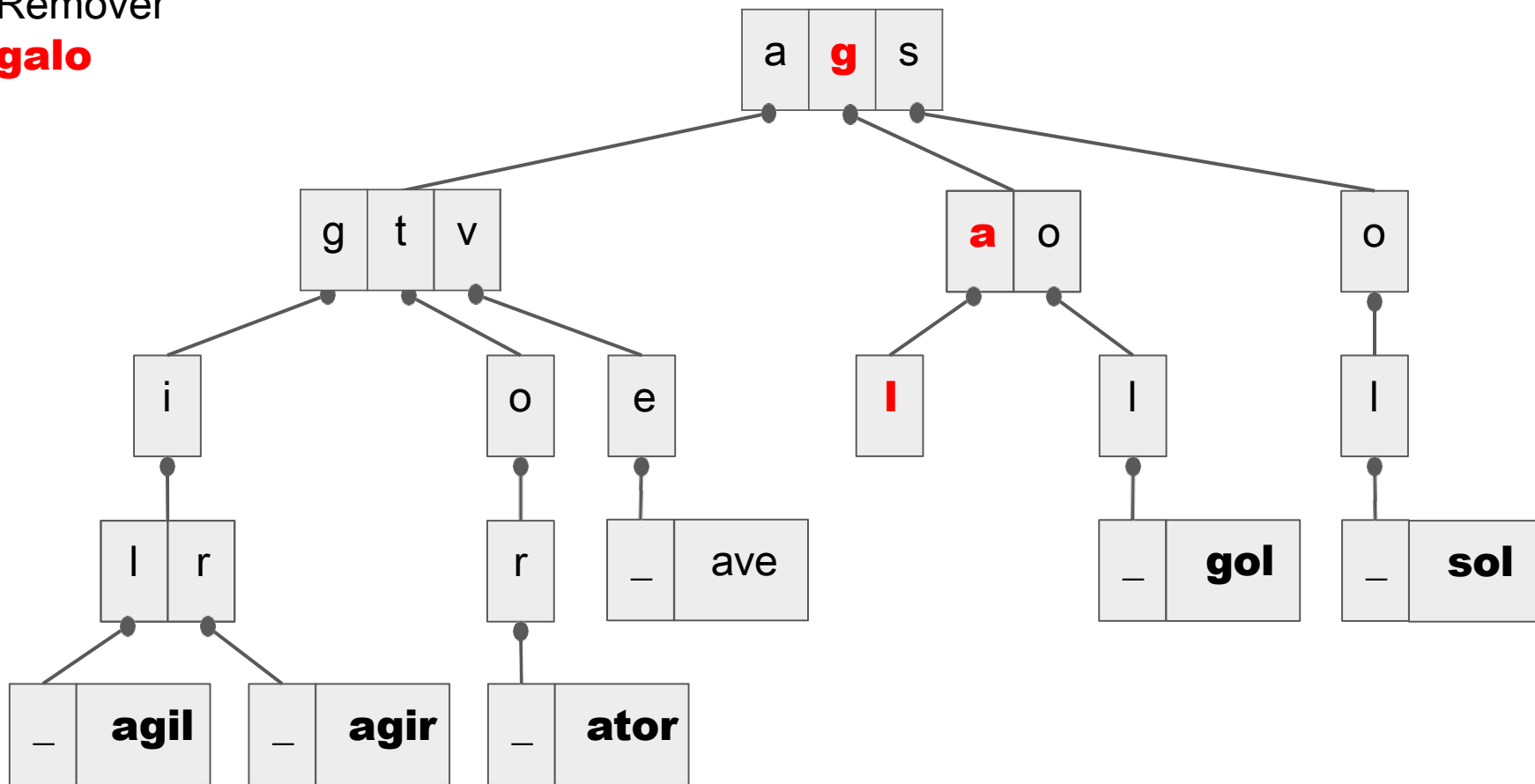
galo



Árvores Trie

Remover

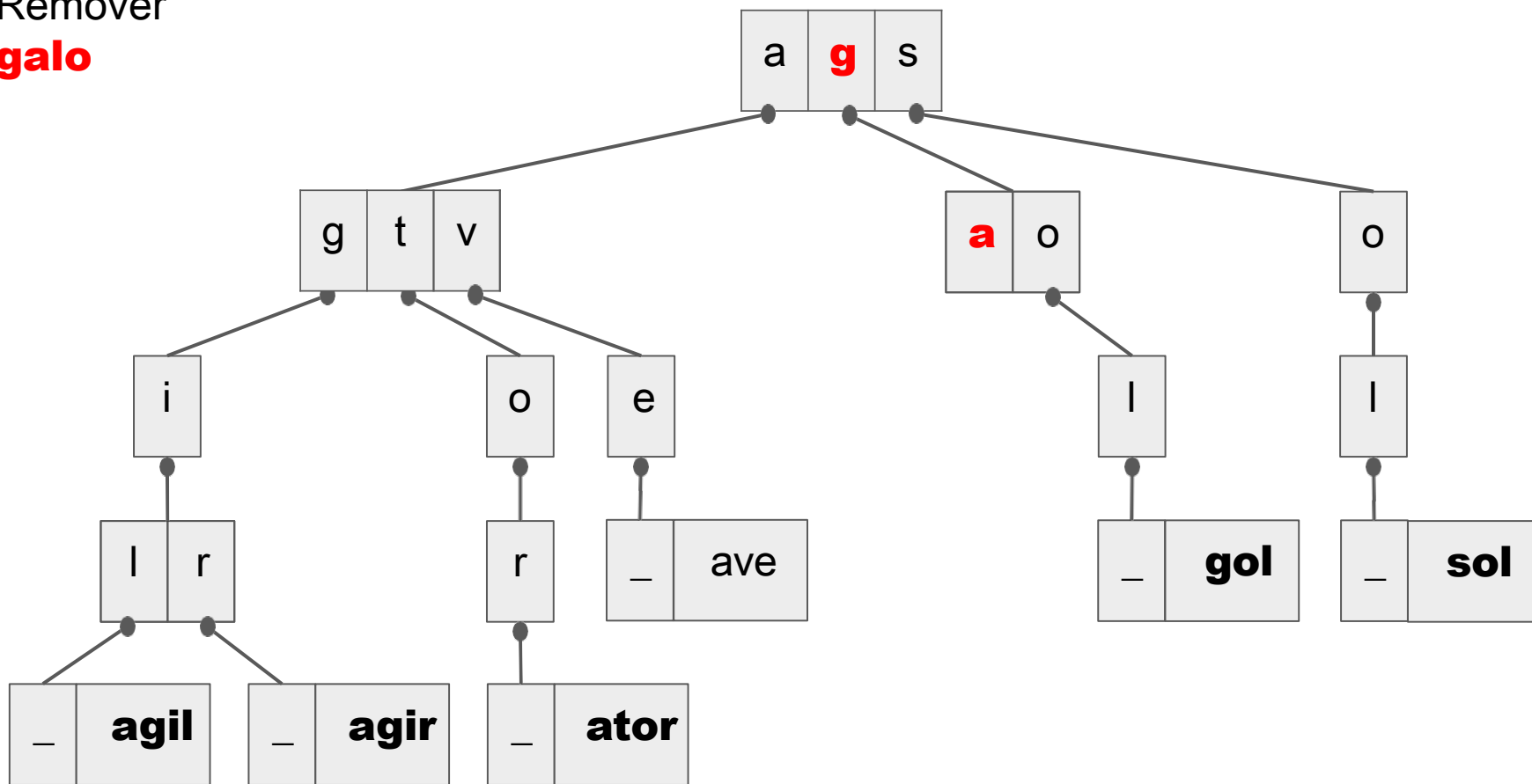
galo



Árvores Trie

Remover

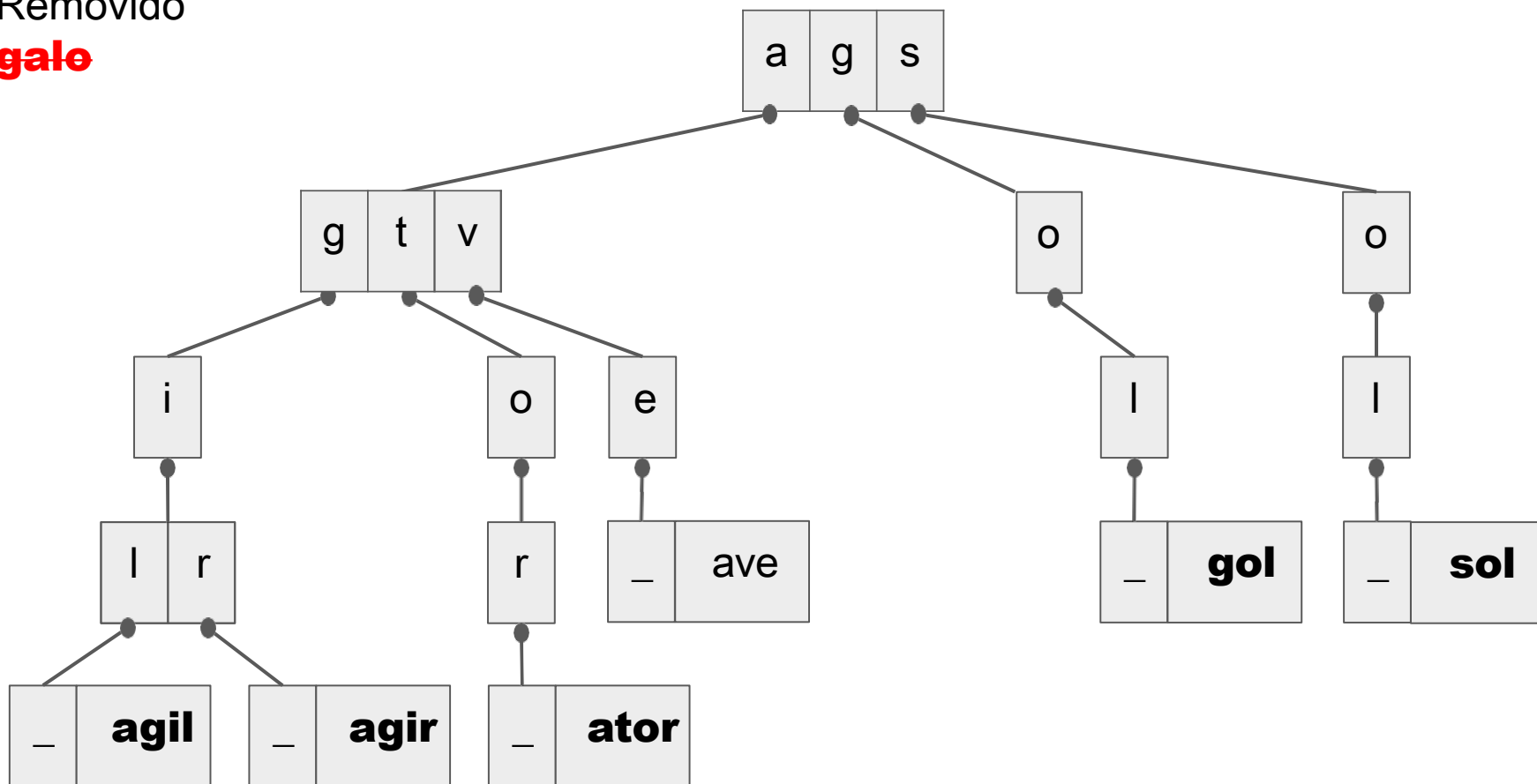
galo



Árvores Trie

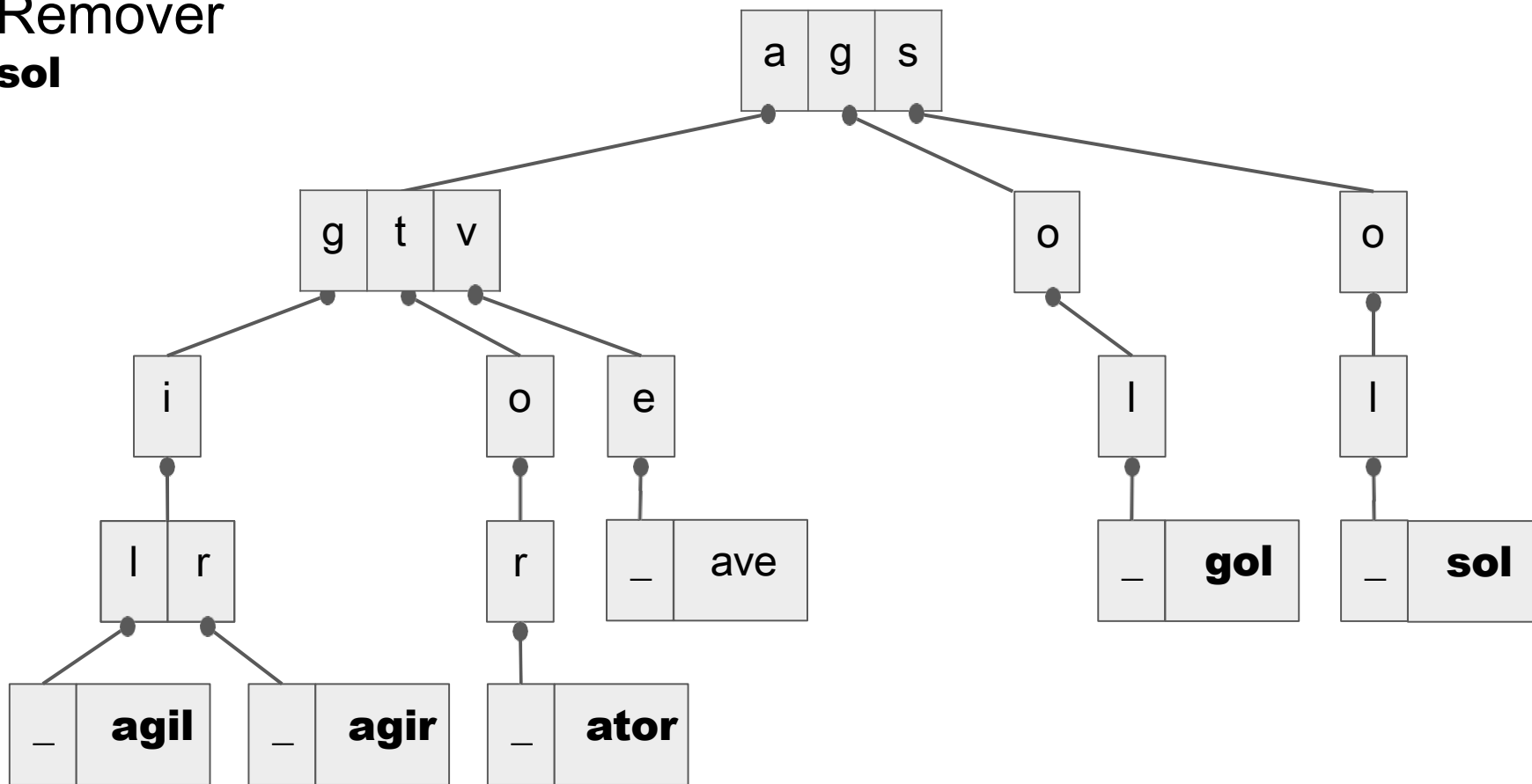
Removido

gale



Árvores Trie

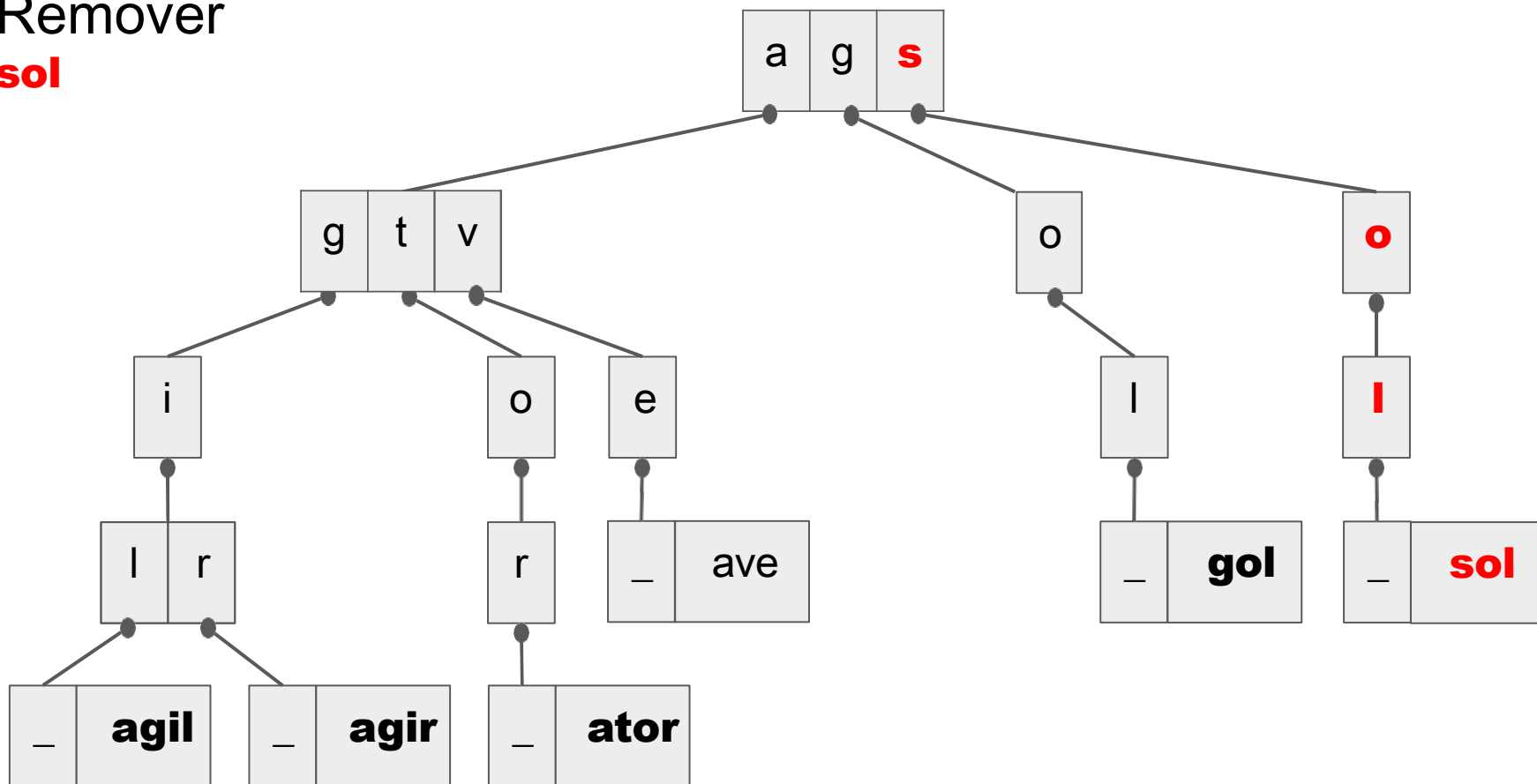
Remover
sol



Árvores Trie

Remover

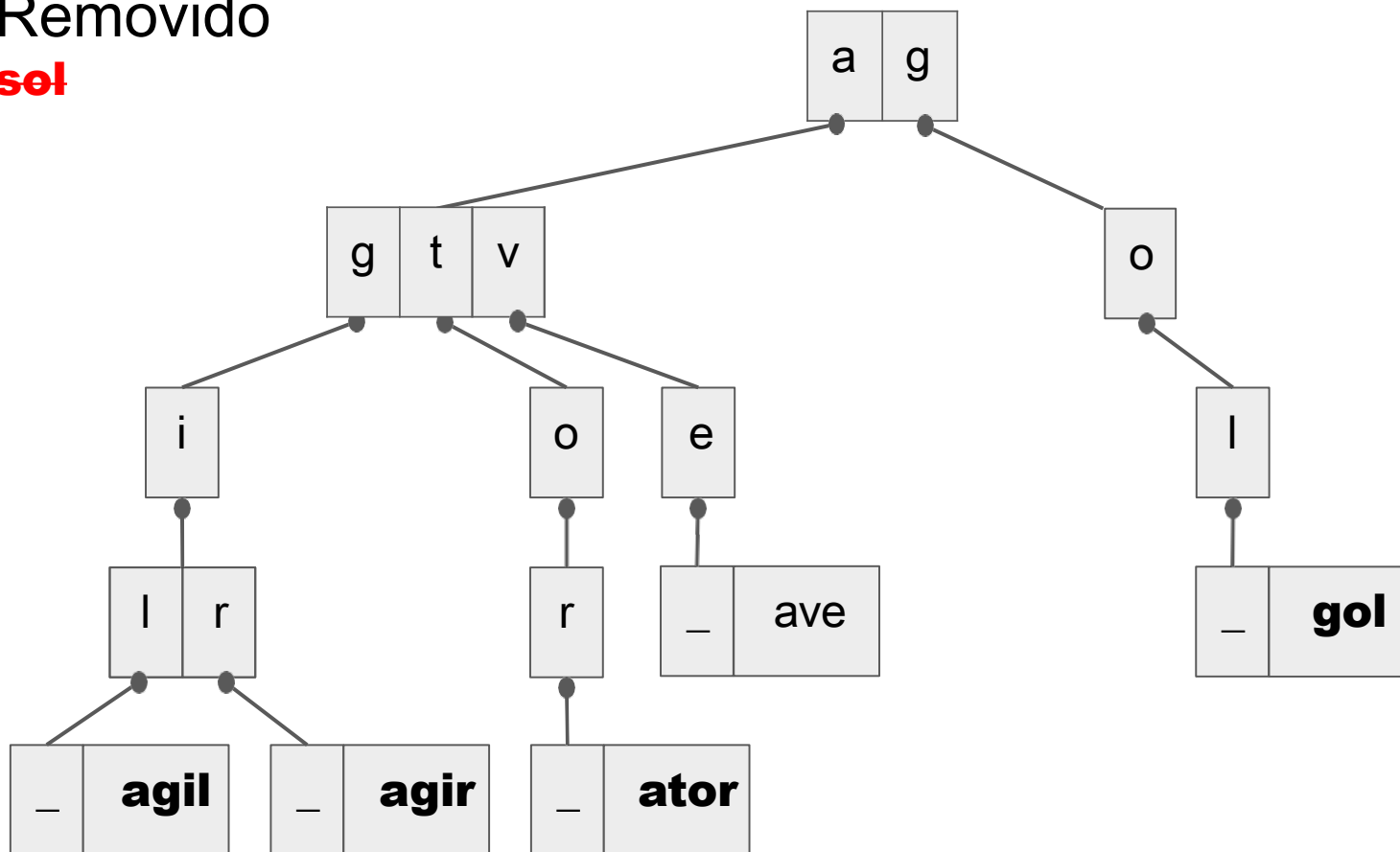
sol



Árvores Trie

Removido

sol





Trabalho Prático 3:

- Autocompletar com Árvore Trie
- Possíveis soluções para jogo Termo

Jogo de uma das atividades
<https://term.ooo>